Christof Teuscher

ECE 410/510: Hardware for AI and ML, Spring 2026

# Welcome and Introduction

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu


teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

# The most complex human-made systems ever created

Cerebras
Wafer Scale Engine 3

**WSE-3**

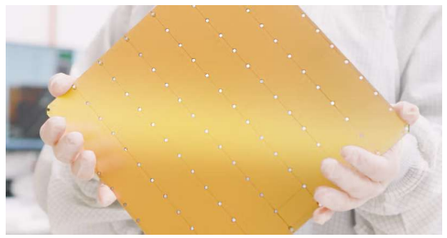
- **4 trillion** transistors
- **46,225 mm²** silicon
- **900,000** cores optimized for sparse linear algebra
- **125 petaFLOPS** of AI compute
- **44 GB** of on-chip memory
- **21 PB/s** memory bandwidth
- **214 Pb/s** fabric bandwidth
- **TSMC 5nm** process

CS-3: Twice the speed at same power & cost

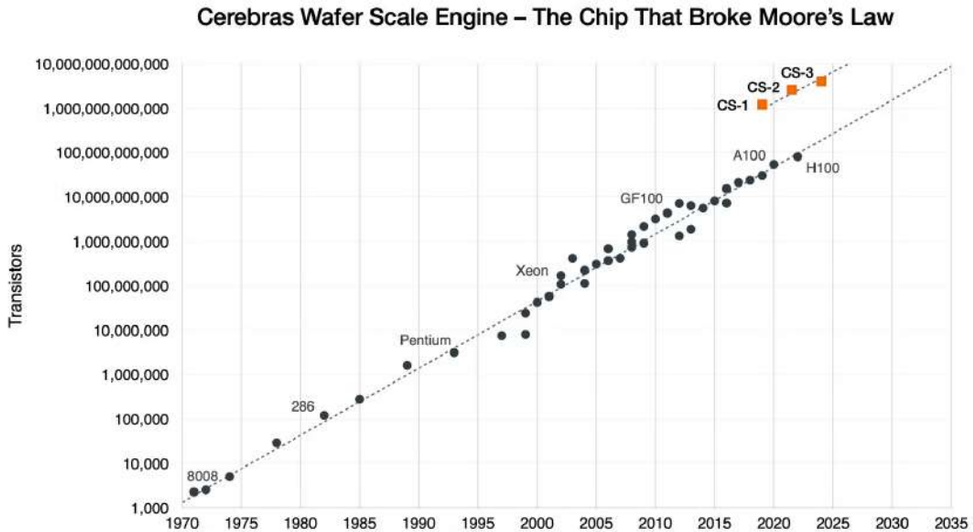
**Wafer Scale Engine-3 vs. H100**

| | Cerebras WSE-3 | Nvidia H100 | Cerebras Advantage |
|---|---|---|---|
| Chip size | 46,225 mm² | 814 mm² | 57x |
| Cores | 900,000 | 16,896 FP32 + 528 Tensor | 52x |
| On-chip memory | 44 Gigabytes | 0.05 Gigabytes | 880x |
| Memory bandwidth | 21 Petabytes/sec | 0.003 Petabytes/sec | 7,000X |
| Fabric bandwidth | 214 Petabits/sec | 0.0576 Petabits/sec | 3,715X |

cerebras



The human brain consists of 100 billion neurons and over 100 trillion synaptic connections.
https://www.cerebras.ai/blog/cerebras-cs3

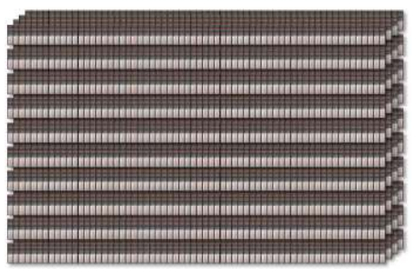



Train Llama2-70B in a Single Day

Meta GPU Cluster

~1 month

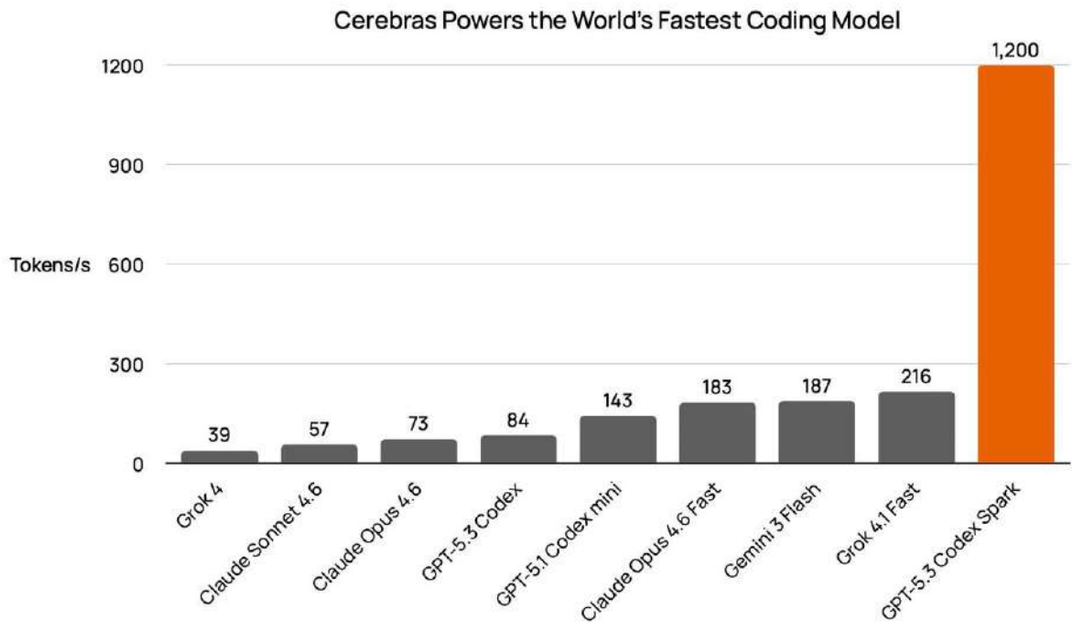
Cerebras CS-3 Cluster



~1 day

teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY





One of NVIDIA's big GTC announcements was a $20 billion bet on the same problem we solved 6 years ago.

Their next-gen inference chip - not available yet - has 140x less memory bandwidth than **Cerebras** has today.

To run a single 2 trillion parameter model, you need more than 2,000 Groq chips. On Cerebras, only 20 wafers.

Even paired with GPUs, in a solution that is scheduled to be delivered later this year, the solution maxes out at ~1,000 tokens per second.

We run at thousands of tokens per second today.
And every day. In production now.

When you connect 2,000 little chips together, every interconnect adds latency. Every cable has overhead. It doesn't matter what your memory bandwidth is on paper if you're bottlenecked by the wiring between thousands of tiny chips.

We solved this with wafer scale.
One integrated system.
Little interconnect tax.

**NVIDIA** told the world that fast inference is where the value is.

They are right - it's why leading AI companies and hyperscalers are choosing **Cerebras**.

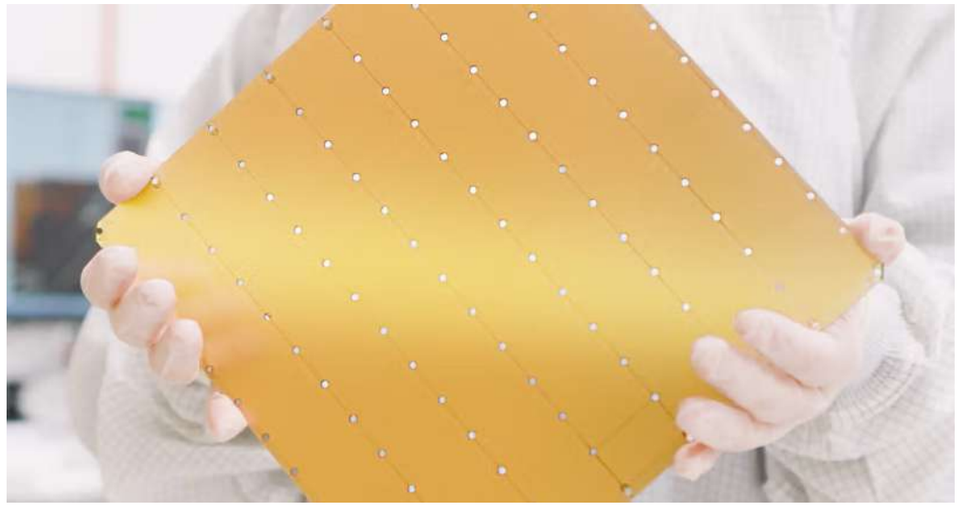

| | Cerebras CS-3 | Rubin + Groq 3 LPU | Cerebras Advantage |
|---|---|---|---|
| Memory | 44GB | 0.5GB | 90X |
| Chips Needed for 2T Parameter Model | 23 | 2000 | 90X |
| Memory Bandwidth | 21 Petabytes/s | 150 Terabytes/s | 140X |
| Availability | Now | End 2026 | |

# How much power does this chip use?

Cerebras
Wafer Scale Engine 3

**WSE-3**

- **4 trillion** transistors
- **46,225 mm²** silicon
- **900,000** cores optimized for sparse linear algebra
- **125 petaFLOPS** of AI compute
- **44 GB** of on-chip memory
- **21 PB/s** memory bandwidth
- **214 Pb/s** fabric bandwidth
- **TSMC 5nm** process



WSJ OPINION

Spring Sale Sign In

English Edition | Print Edition | Video | Audio | Latest Headlines | Puzzles | More

World Business U.S. Politics Economy Tech Markets & Finance Opinion Free Expression Arts Lifestyle Real Estate Personal Finance Health Style Sports

FREE EXPRESSION
A Daily Newsletter From WSJ Opinion

ABOUT FREE EXPRESSION | GET FREE EXPRESSION IN YOUR INBOX | BRITAIN'S JEWS ARE ON EDGE | SUCKERS FOR SOVIET COMMUNISM

OPINION COMMENTARY Follow

**The Microchip Era Is About to End**

The future is in wafers. Data centers will be the size of a box, not vast energy-hogging structures.

By George Gilder
Nov. 3, 2025 1:34 pm ET

Most Popular News

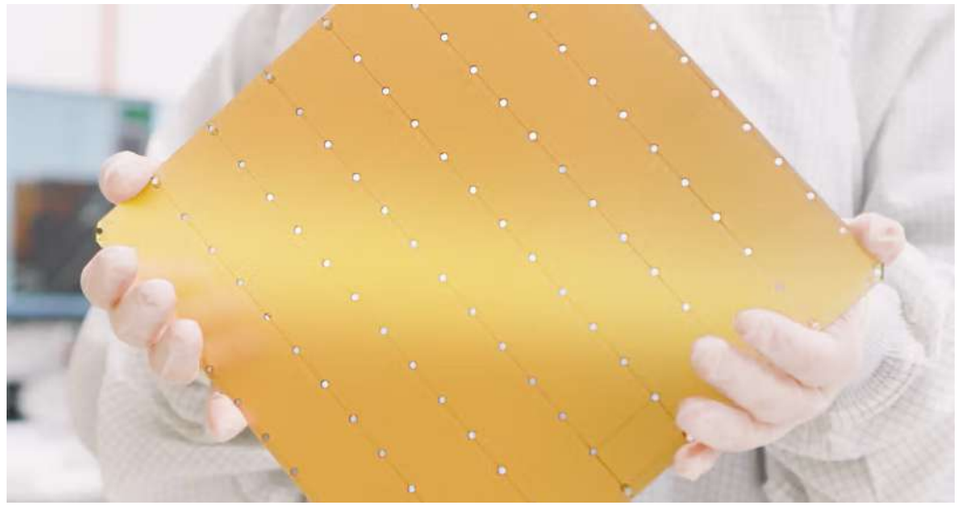
https://www.wsj.com/opinion/the-microchip-era-is-about-to-end-e71eb66a

# How much power does this chip use?

Cerebras
Wafer Scale Engine 3

**WSE-3**

- **4 trillion** transistors
- **46,225 mm²** silicon
- **900,000** cores optimized for sparse linear algebra
- **125 petaFLOPS** of AI compute
- **44 GB** of on-chip memory
- **21 PB/s** memory bandwidth
- **214 Pb/s** fabric bandwidth
- **TSMC 5nm** process



WSJ OPINION

Spring Sale Sign In

English Edition | Print Edition | Video | Audio | Latest Headlines | Puzzles | More

World Business U.S. Politics Economy Tech Markets & Finance Opinion Free Expression Arts Lifestyle Real Estate Personal Finance Health Style Sports

FREE EXPRESSION
A Daily Newsletter From WSJ Opinion

ABOUT FREE EXPRESSION | GET FREE EXPRESSION IN YOUR INBOX | BRITAIN'S JEWS ARE ON EDGE | SUCKERS FOR SOVIET COMMUNISM

OPINION COMMENTARY Follow

**The Microchip Era Is About to End**

The future is in wafers. Data centers will be the size of a box, not vast energy-hogging structures.

By George Gilder
Nov. 3, 2025 1:34 pm ET

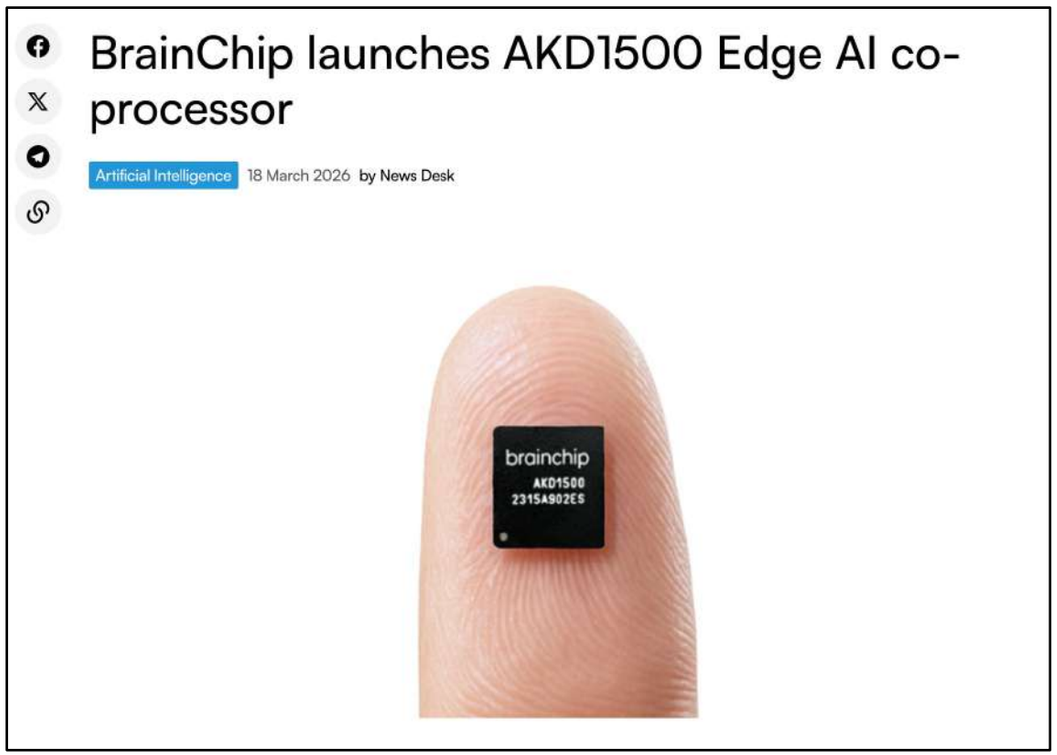
Most Popular News

https://www.wsj.com/opinion/the-microchip-era-is-about-to-end-e71eb66a

- **23 kW**
- **Closed internal water cooling**

- A typical U.S. house experiences peak power consumption between 5 and 10kW.

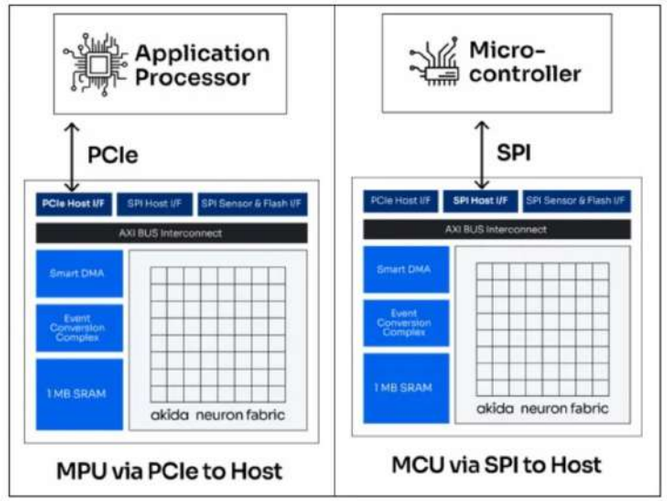
# BrainChip launches AKD1500 Edge AI co-processor


BrainChip launches AKD1500 Edge AI co-processor

Artificial Intelligence 18 March 2026 by News Desk



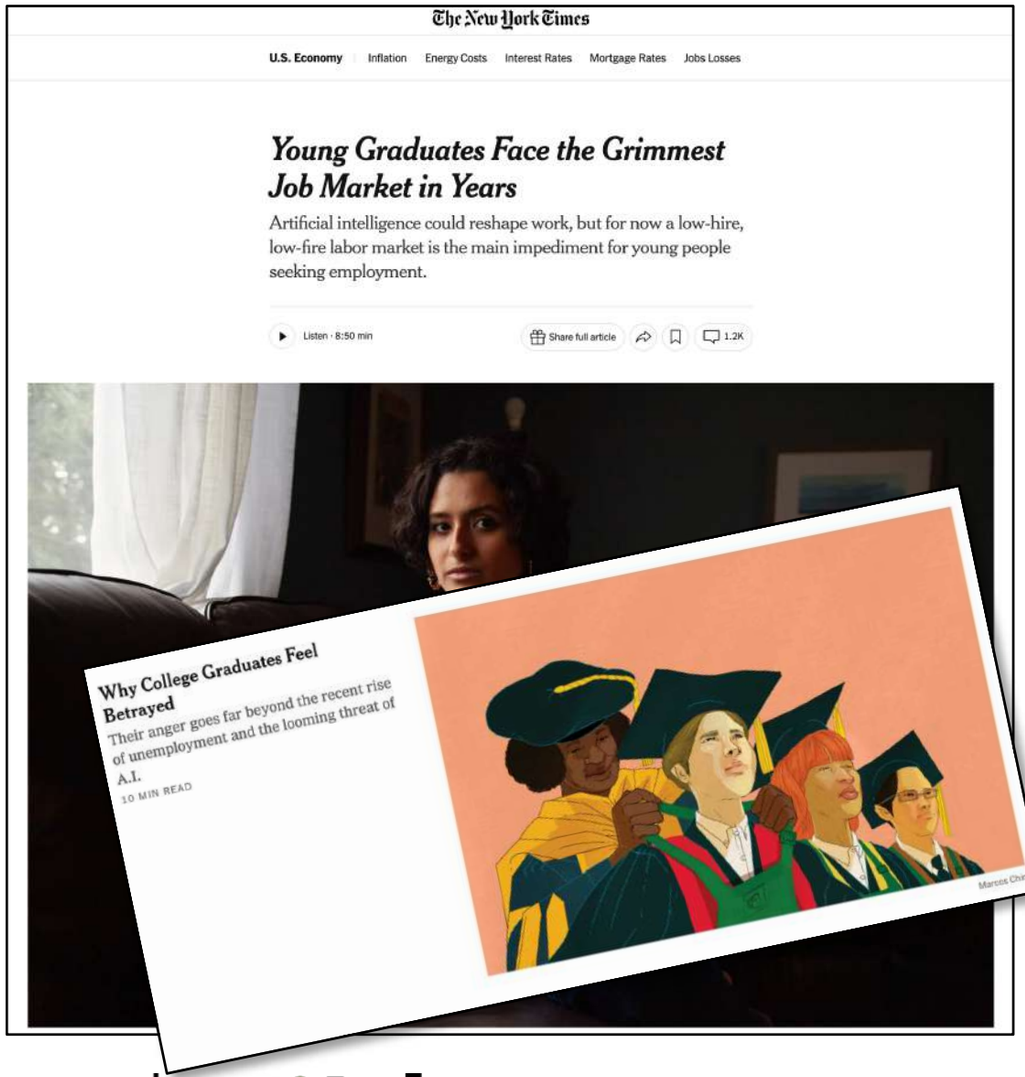
The AKD1500 achieves a breakthrough in efficiency for low power Edge applications by delivering slightly under **1 tera operations per second** (TOPS) while consuming less than **200mW** of power in serial mode and 300mW in PCIe mode.

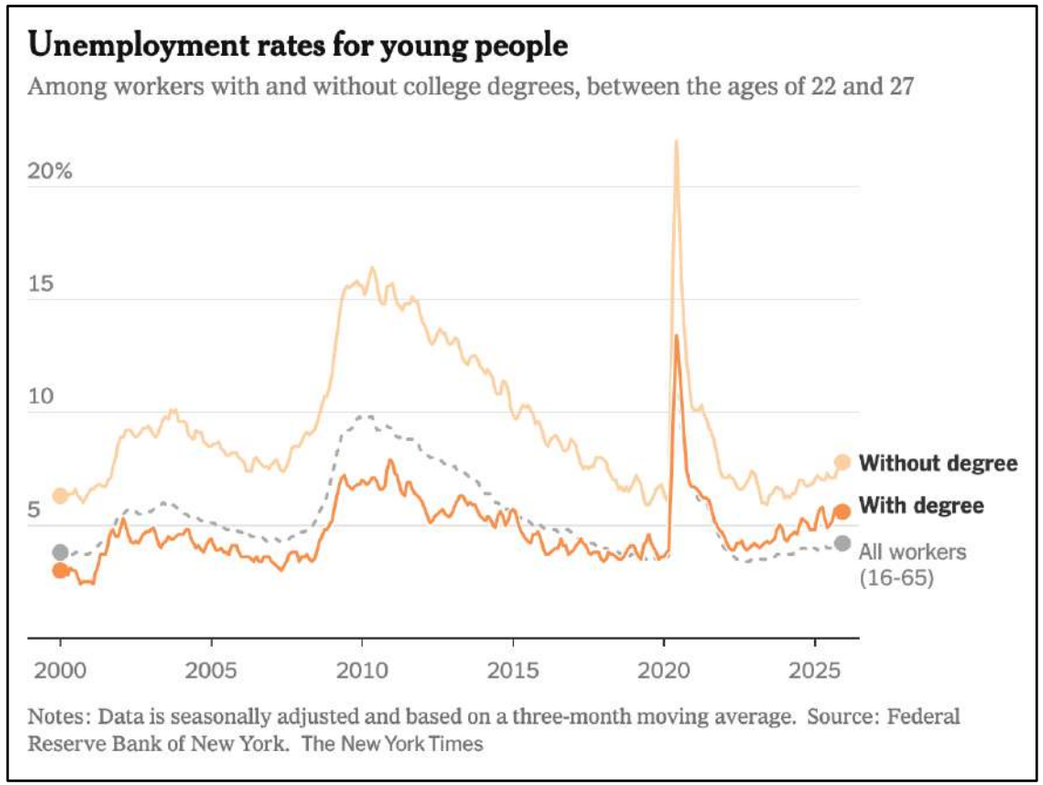



https://www.electronicspecifier.com/products/artificial-intelligence/brainchip-launches-akd1500-edge-ai-co-processor

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching




https://www.nytimes.com/2026/03/24/business/economy/college-graduates-job-market-hiring.html
https://www.nytimes.com/2026/03/27/business/college-graduates-economy-unemployment-.html

# AI proficiency is not optional anymore!

teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

Christof Teuscher • teuscher@pdx.edu



THE WALL STREET JOURNAL.
English Edition ▾ | Print Edition | Video | Audio | Latest Headlines | Puzzles | More ▾

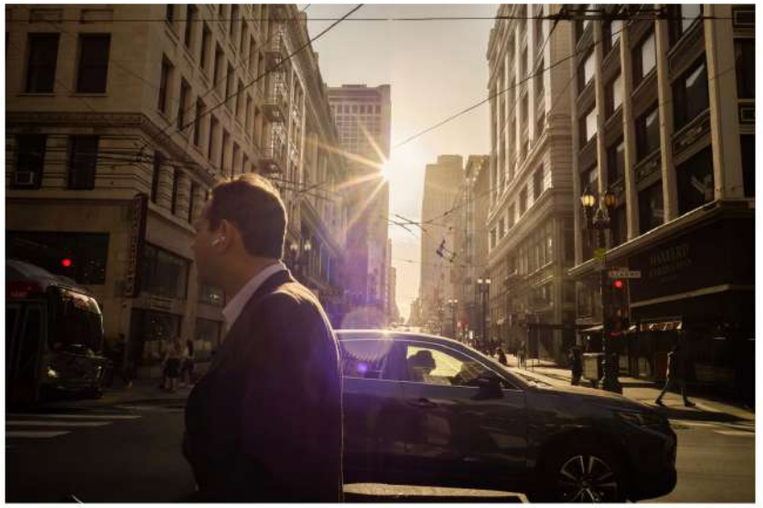
ECONOMY | JOBS | CAPITAL ACCOUNT

# Tech Has Never Caused a Job Apocalypse. Don't Bet on It Now.

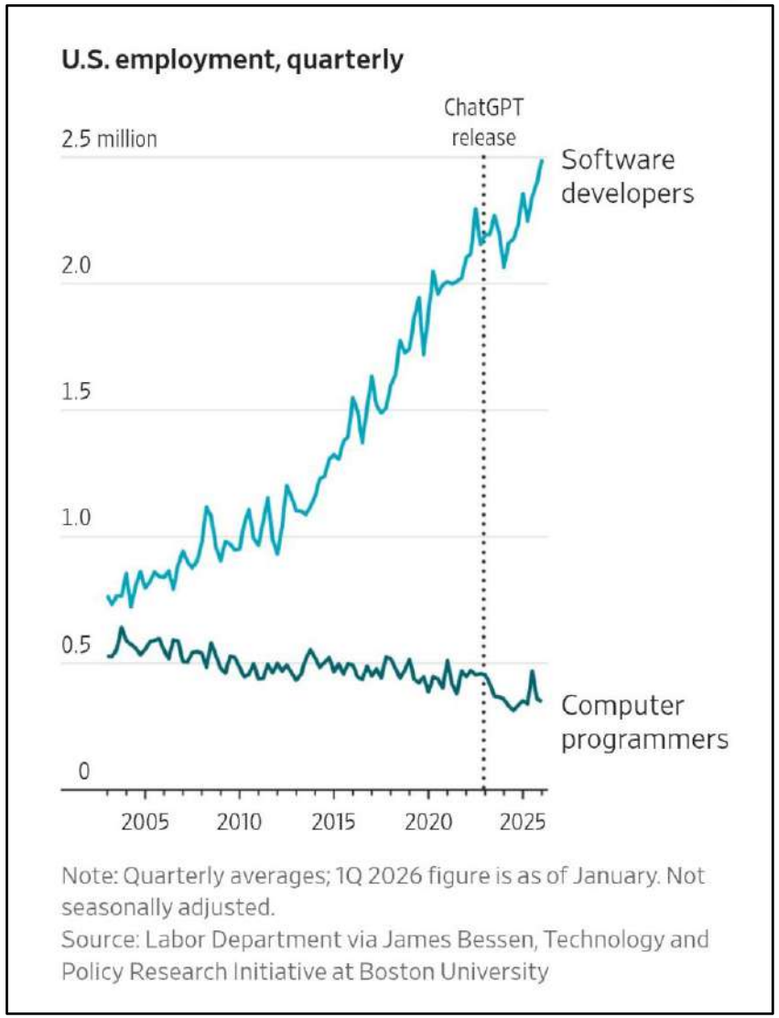
Neither theory, history nor the latest data suggests a recession driven by AI job dislocation is likely

By Greg Ip Follow
Feb. 27, 2026 5:30 am ET

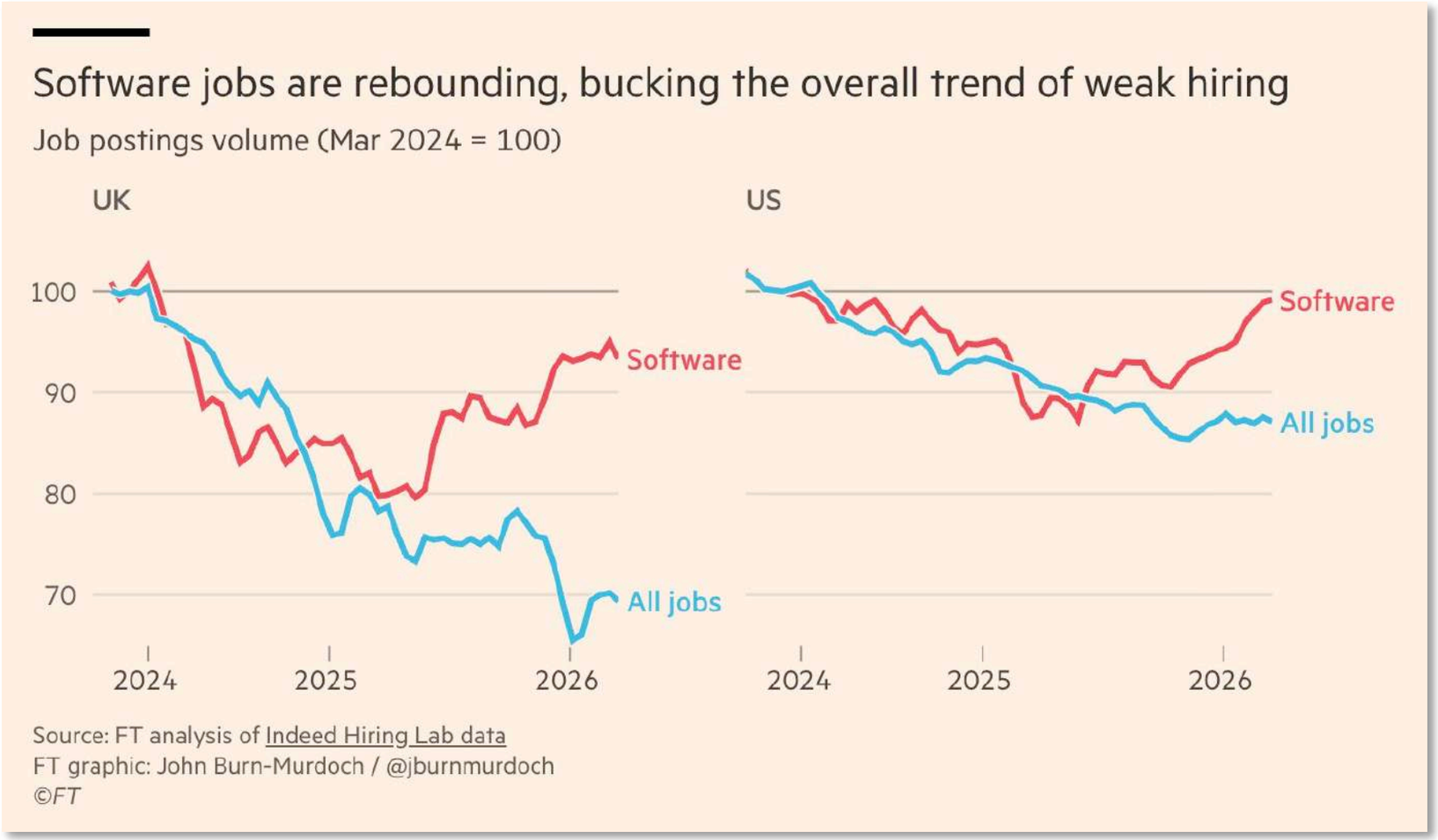
Share | Resize | 589 | Listen (1 min) | ⋮



People should still have jobs to commute to despite AI. BRIAN L. FRANK FOR WSJ




https://www.wsj.com/economy/jobs/tech-has-never-caused-a-job-apocalypse-dont-bet-on-it-now-d192b579




https://www.ft.com/content/7325e967-5f4e-40b1-af3f-7d2351781843?syn-25a6b1a6=1

EE Times

MY CART

HOME NEWS PERSPECTIVES DESIGNLINES PODCASTS EDUCATION SUPPLY CHAIN REP

DESIGNLINES | AI & BIG DATA DESIGNLINE

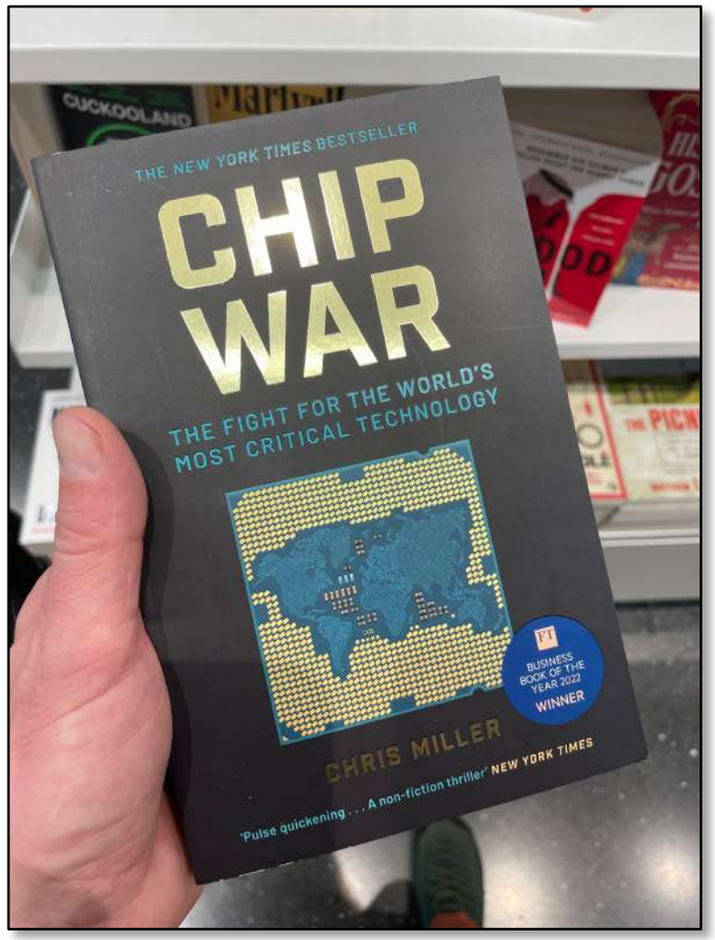
# TSMC Price Hikes End the Era of Cheap Transistors

By Pablo Valerio 10.01.2025 0

| Process Node | Year of HVM Introduction | Estimated Avg. Wafer Price (USD) | Node-over-Node Price Increase (%) | Key Customers at Launch |
|---|---|---|---|---|
| **7nm** | 2018 | ~$9,350 | - | Apple, AMD, Nvidia |
| **5nm** | 2020 | ~$17,000 | ~82% | Apple, AMD |
| **3nm** | 2022-2023 | ~$20,000 | ~18% | Apple |
| **2nm** | 2025 (est.) | ~$30,000+ | ~50%+ | Apple, Nvidia, AMD, MediaTek |

*Semiconductor Wafer Cost Evolution by Process Node (2020-2026E)*

https://www.eetimes.com/tsmc-price-hikes-end-the-era-of-cheap-transistors




## Geopolitical dimensions of the fight for AI dominance

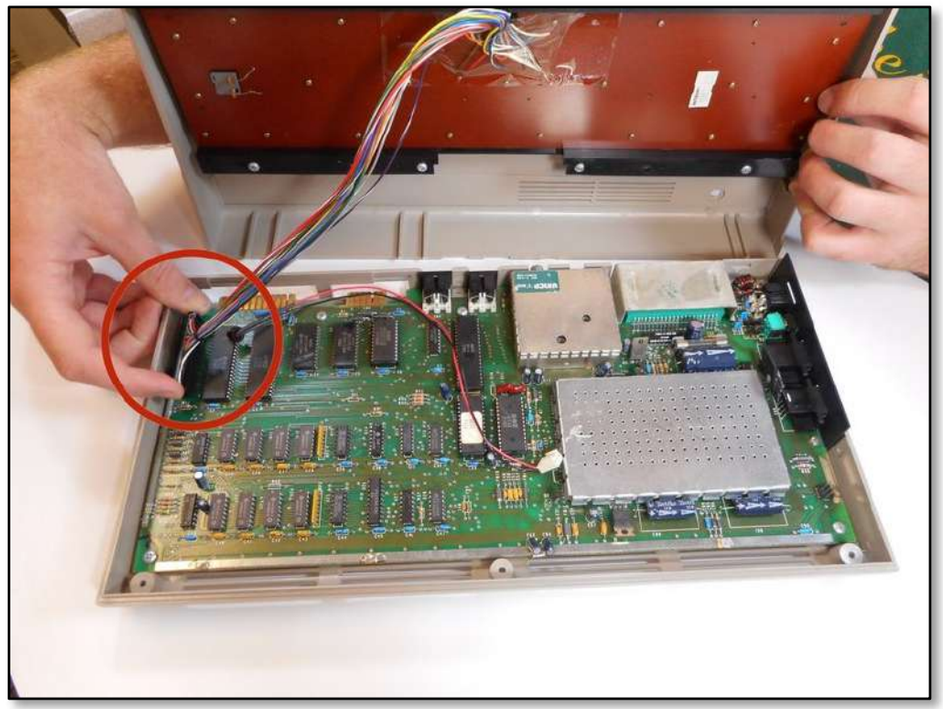
# Commodore C64

1982

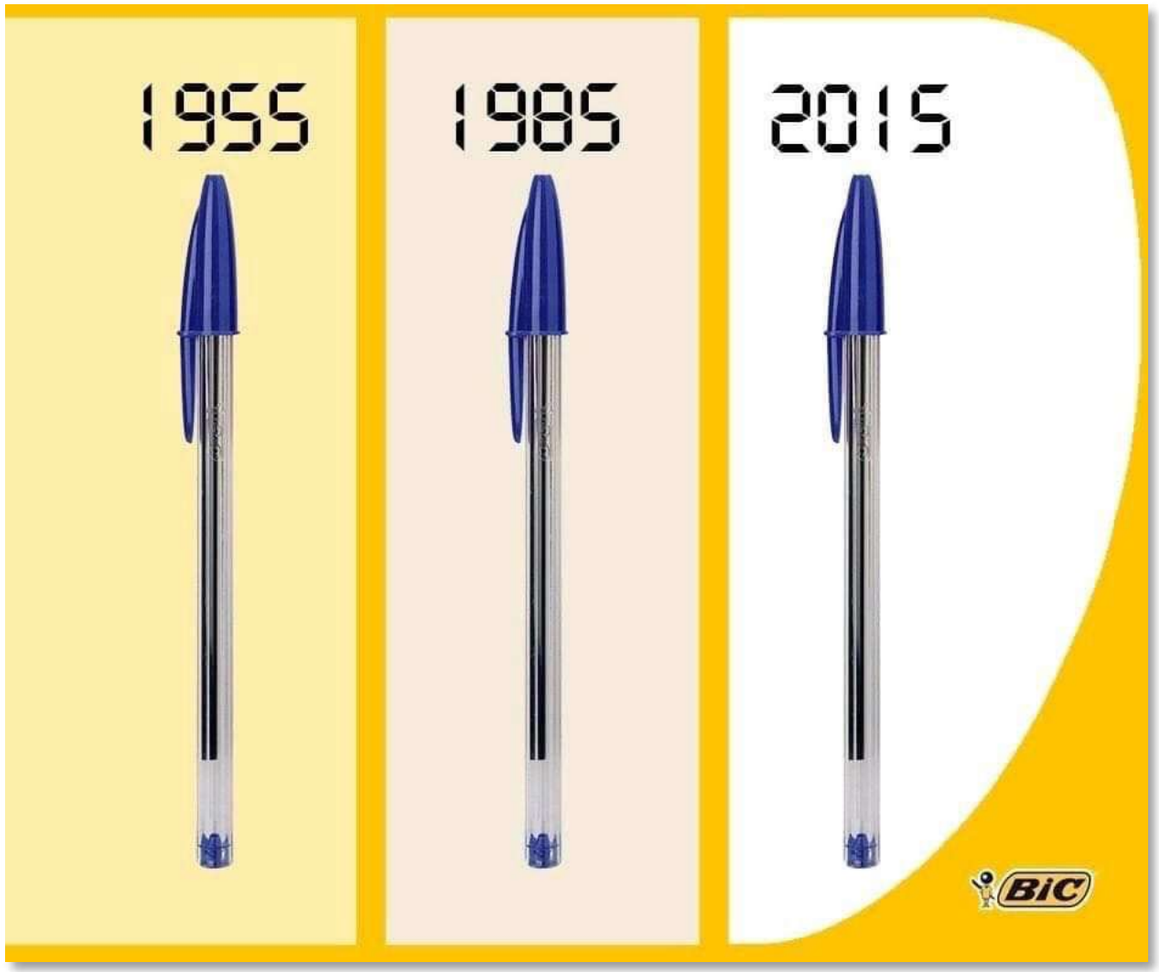

Radio-frequency identification (RFID) tag

Same amount of memory!

# The way we build, program, and use computers has fundamentally changed

MOS 6510, 1 MHz, 64 kB RAM, ~9000 transistors


1955
1985
2015
BIC

Daily Mail .com

# The perfect piece of toast: Scientists test 2,000 slices and find 216 seconds is the optimum time

By DAILY MAIL REPORTER
**UPDATED:** 04:03 EST, 22 July 2011



Scientists today revealed the mathematical formula for a perfect slice of toast, showing that it is best cooked for exactly 216 seconds.

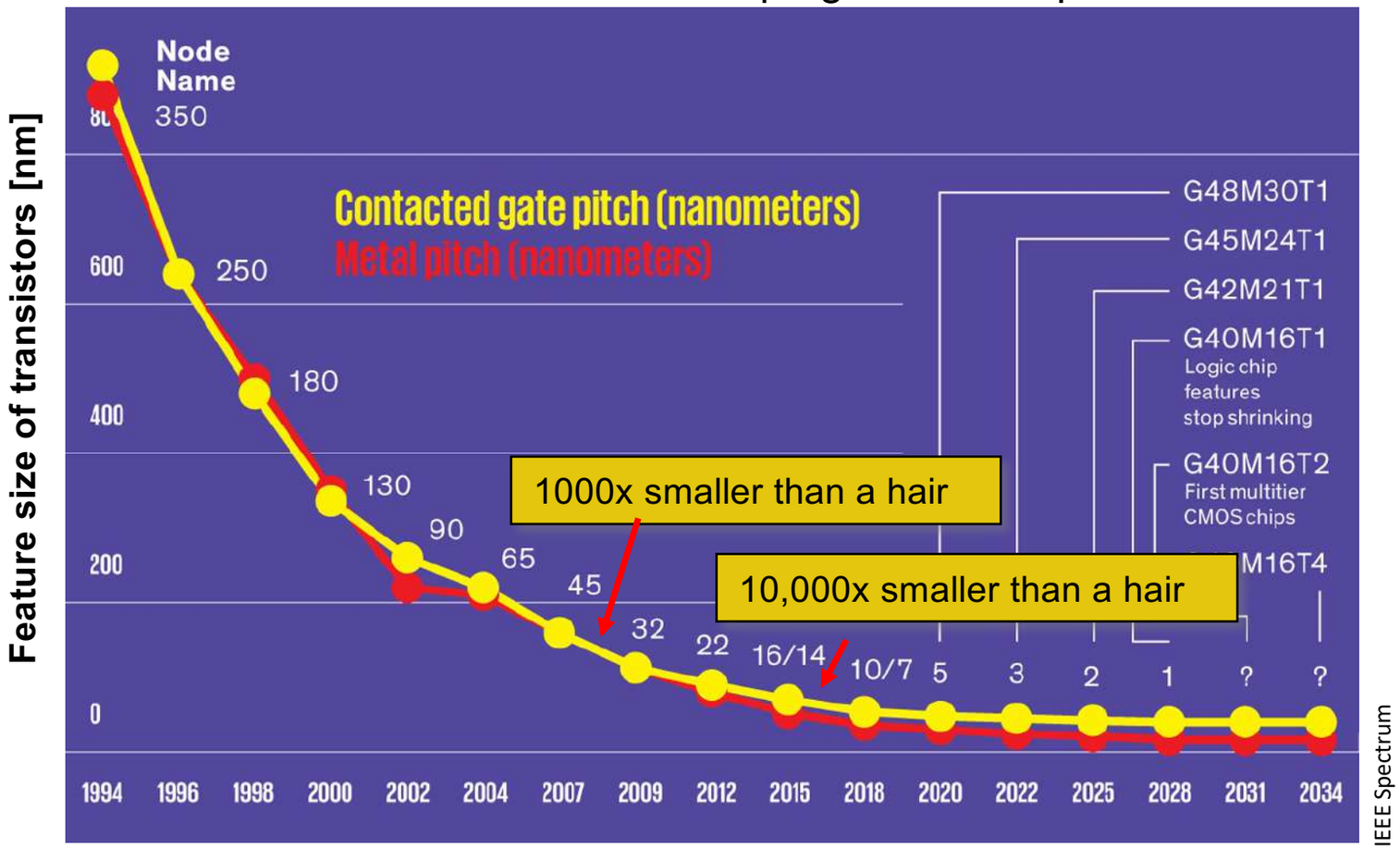
A team of researchers carried out a study which found the optimum thickness is 14mm and the ideal amount of butter is 0.44 grams per square inch.

The recommended cooking time gives the slice a 'golden-brown' colour and the 'ultimate balance of external crunch and internal softness'.


**Perfect: The ideal slice of toast is cooked for just over three minutes, according to scientists**

# Smaller smaller smaller smaller smaller smaller smaller smaller smaller

## Miniaturization: the cornerstone of progress in computer science

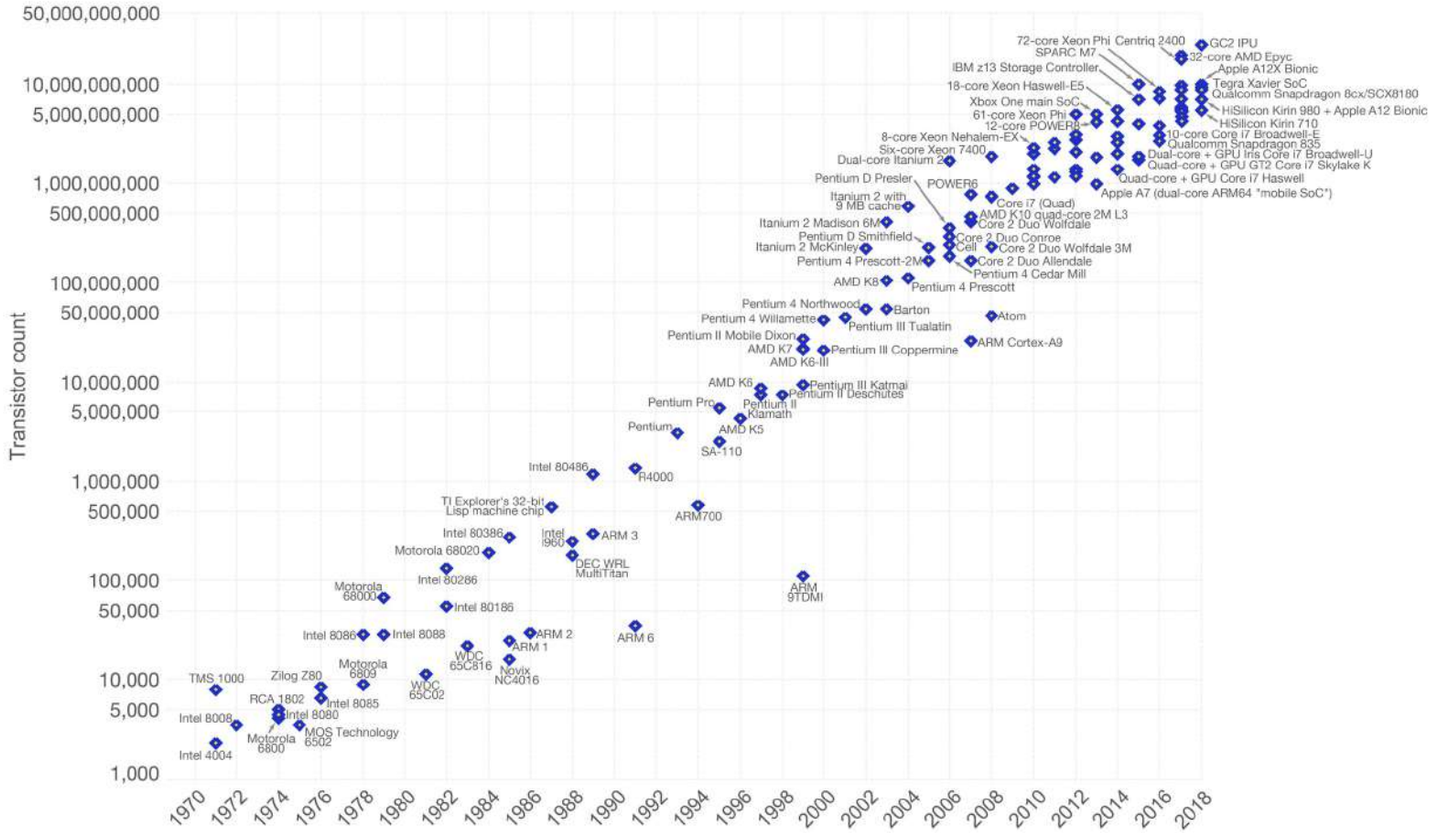



IEEE Spectrum

# Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

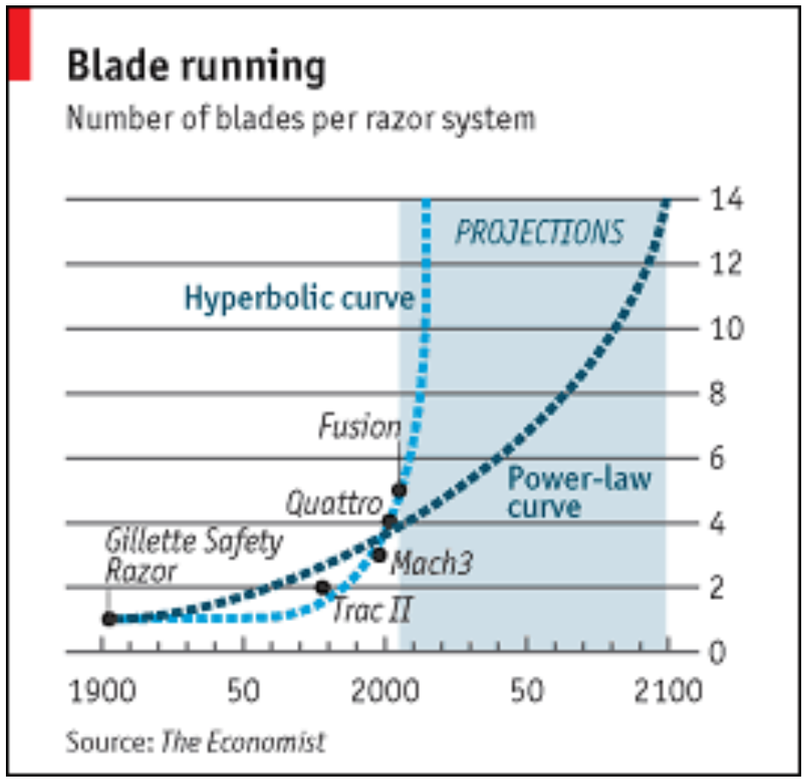
Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.

Our World in Data




Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

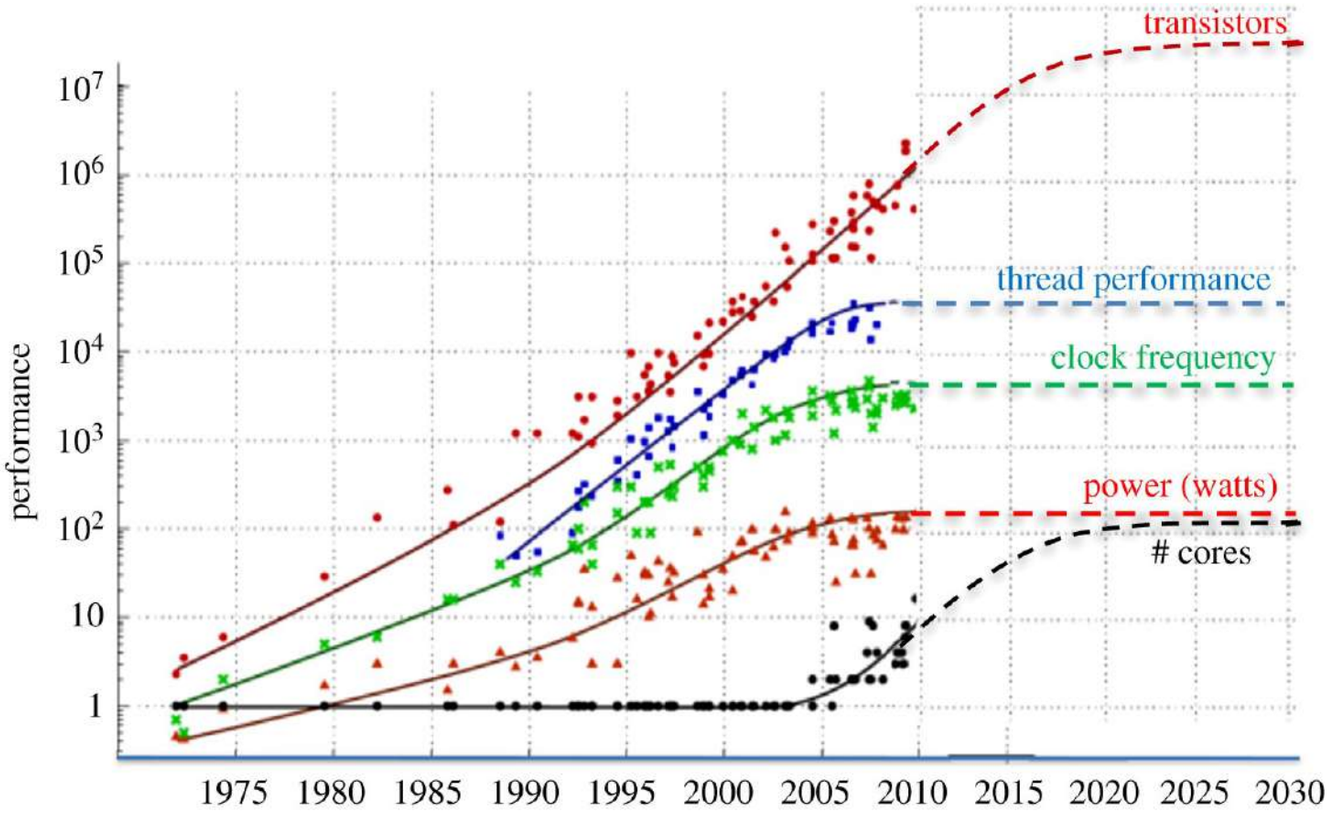
# The Cutting Edge: A Moore's Law for Razor Blades?

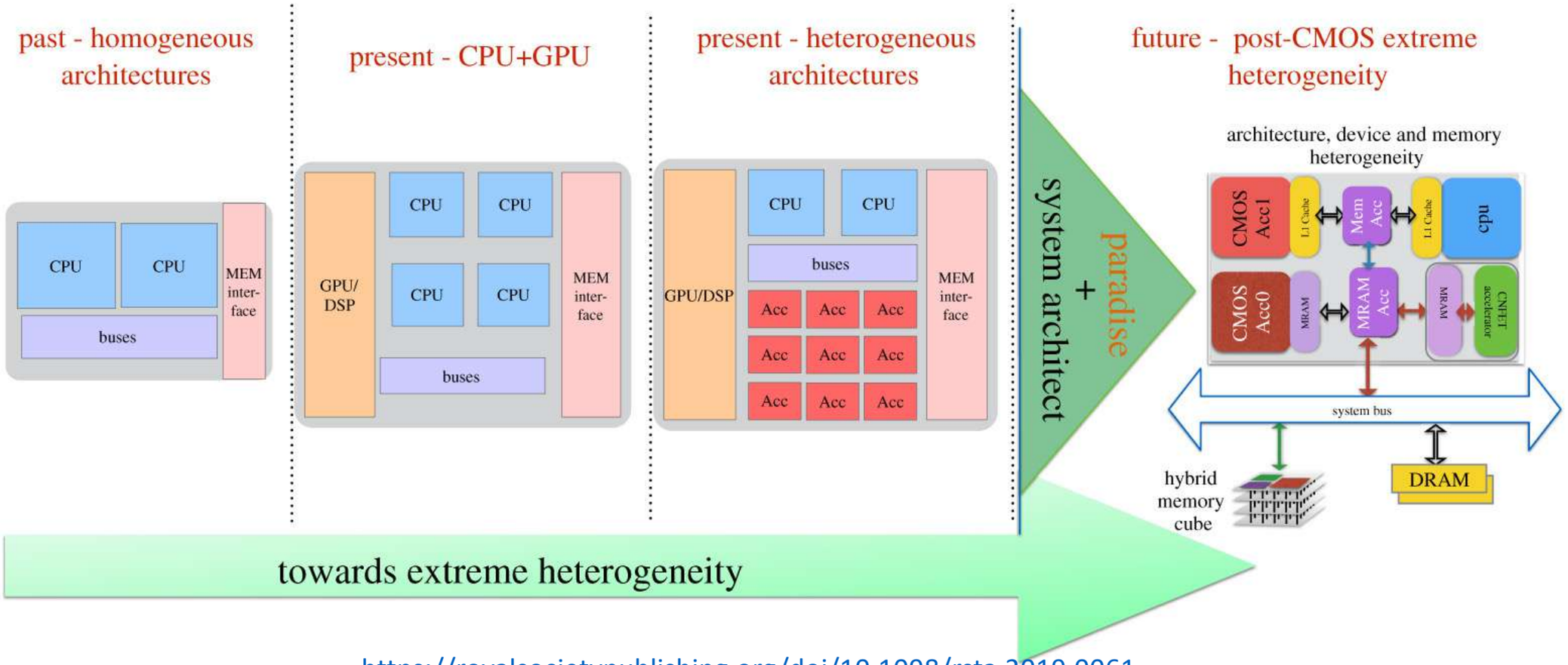


Source: The Economist, March 16, 2006

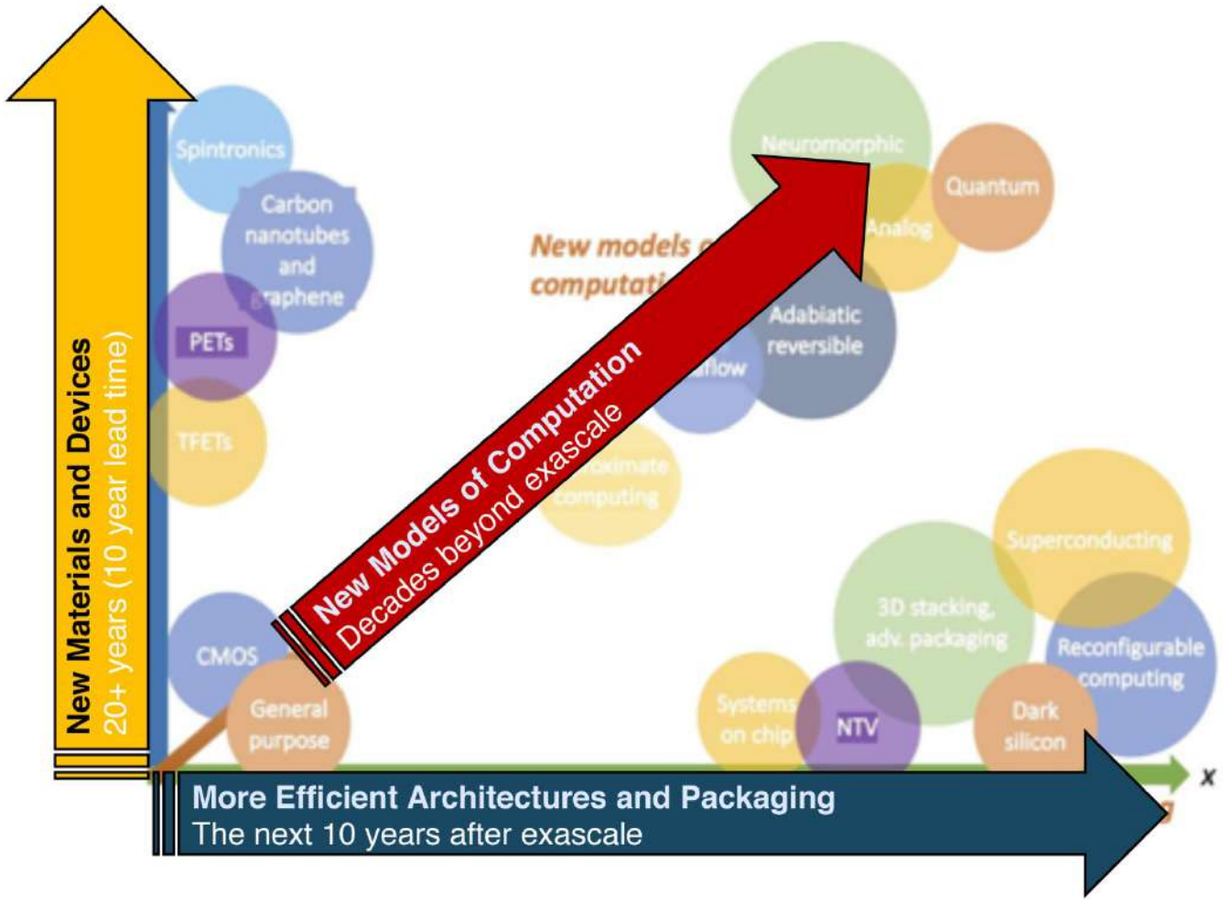
# More of Moore?




https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061



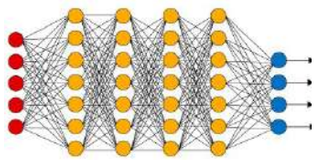

https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061



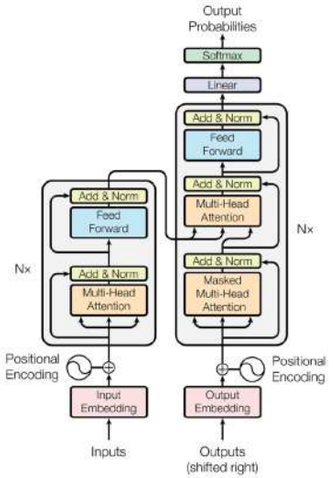

https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061

# How did we get here?

Alan Turing

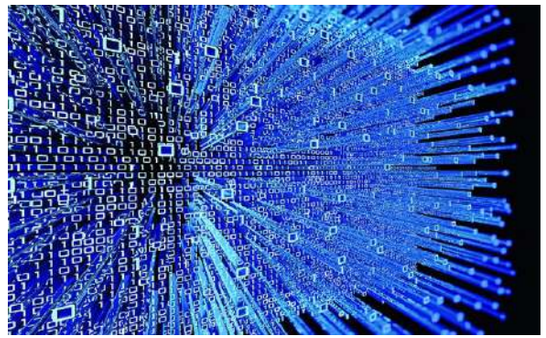


Deep neural networks

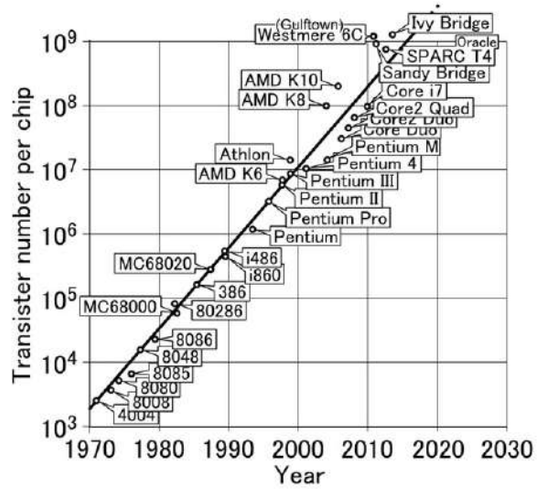



Transformers

Models that learn and adapt from data

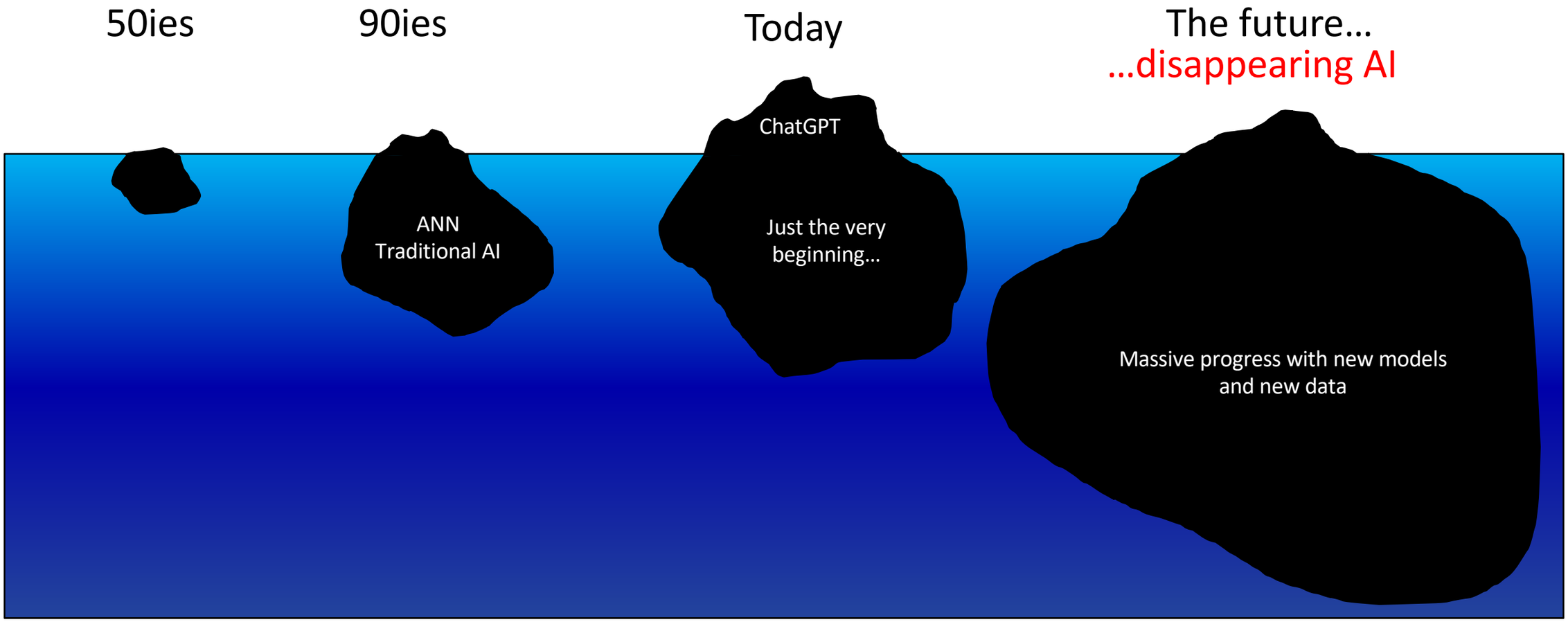


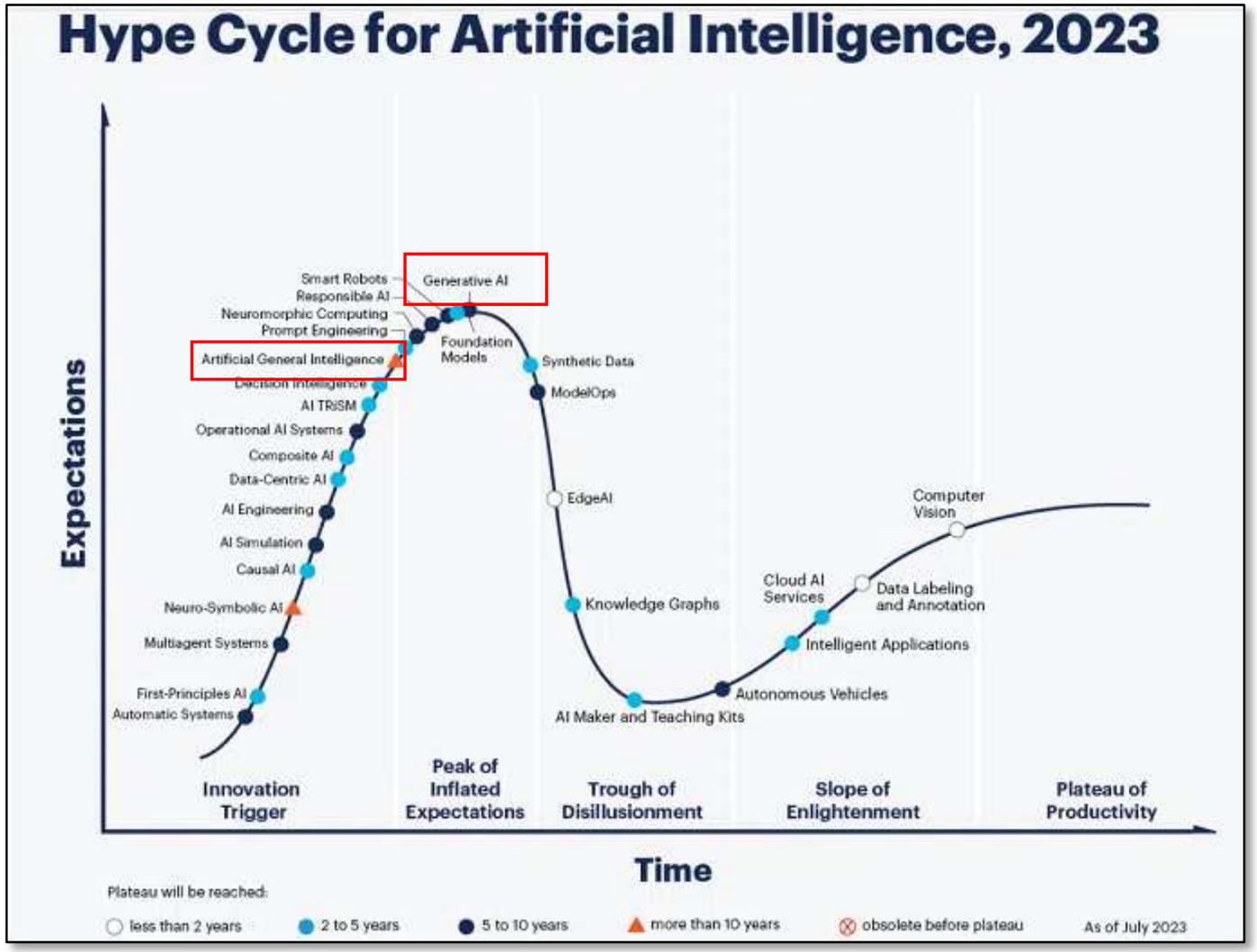
(Big) data

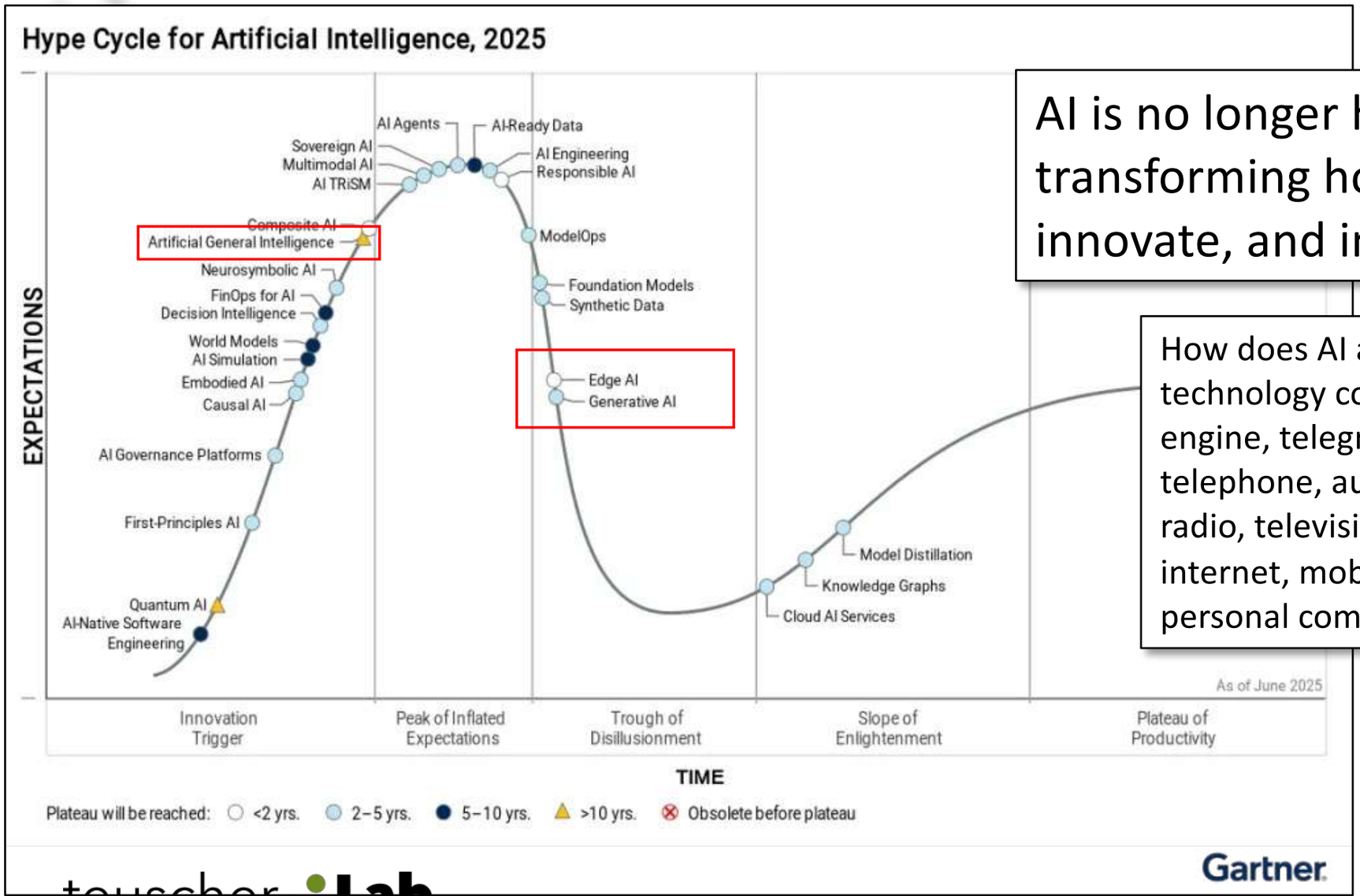



Moore's law

# Where is this going?




Hype Cycle for Artificial Intelligence, 2023
Expectations
Smart Robots
Responsible AI
Neuromorphic Computing
Prompt Engineering
Artificial General Intelligence
Decision Intelligence
AI TRiSM
Operational AI Systems
Composite AI
Data-Centric AI
AI Engineering
AI Simulation
Causal AI
Neuro-Symbolic AI
Multiagent Systems
First-Principles AI
Automatic Systems
Generative AI
Foundation Models
Synthetic Data
ModelOps
EdgeAI
Knowledge Graphs
AI Maker and Teaching Kits
Autonomous Vehicles
Cloud AI Services
Intelligent Applications
Data Labeling and Annotation
Computer Vision
Innovation Trigger
Peak of Inflated Expectations
Trough of Disillusionment
Slope of Enlightenment
Plateau of Productivity
Time
Plateau will be reached:
less than 2 years
2 to 5 years
5 to 10 years
more than 10 years
obsolete before plateau
As of July 2023




AI is no longer hype—it's transforming how we work, innovate, and invest.

How does AI as a revolutionary technology compare to the steam engine, telegraph, photograph, telephone, automobile, airplane, radio, television, computer, the internet, mobile phone, and the personal computer?

the ONION America's Finest News Source. Shop

HOME LATEST NEWS LOCAL POLITICS ENTERTAINMENT SPORTS OPINION

NEWS IN BRIEF

# Man Honestly Better Off For Having Turned Self Over To Algorithms

Published October 26, 2021

BOSTON—Noting the major improvements in his mood, taste, and overall outlook, friends and family members of local man Joseph Bennington told reporters Tuesday that he was honestly much better off for having turned his life over to the internet's algorithms. "He's always in a great mood from looking at funny videos, and those sweaters that Amazon suggested he buy are a huge upgrade on his old work outfits," said close friend Andy Wilcolm, explaining that Bennington's life had improved precipitously over the past year as he exchanged free will for the automated decisions handed to him by algorithms at TikTok, YouTube, Netflix, Facebook, Spotify, Google, and other major services. "He also has a constant influx of pornography, tailored exactly to his taste, which means he doesn't really have to go out and date. That had just been making him sad. But even if he did want to date, the algorithms at Tinder and Hinge would have him covered." Sources close to the man added that they had also seen a marked improvement in his social life after the algorithms helped him find community with the Proud Boys, 8_Chan, and the Ku Klux Klan.

Christof Teuscher • teuscher@pdx.edu

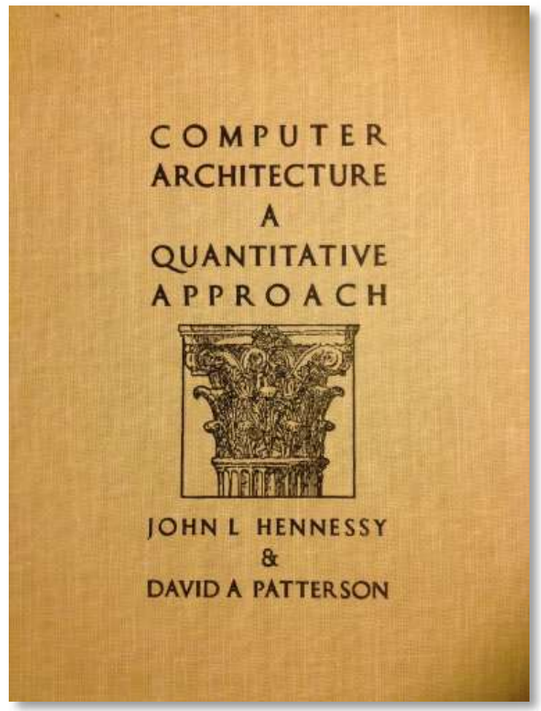


# Traditional computer engineering and architecture is dead

# AI/ML moves fast

- **“In AI, something breaks, ships, or goes viral every six hours.”**
- Timescale: ~weeks

- For comparison: Hennessy & Patterson, first edition, 1990
- I have a beautiful 2nd edition from 1996 that I bought for my computer architecture class.



## Synopsys lays out strategy for AI 'agents' to design computer chips

By **Stephen Nellis**

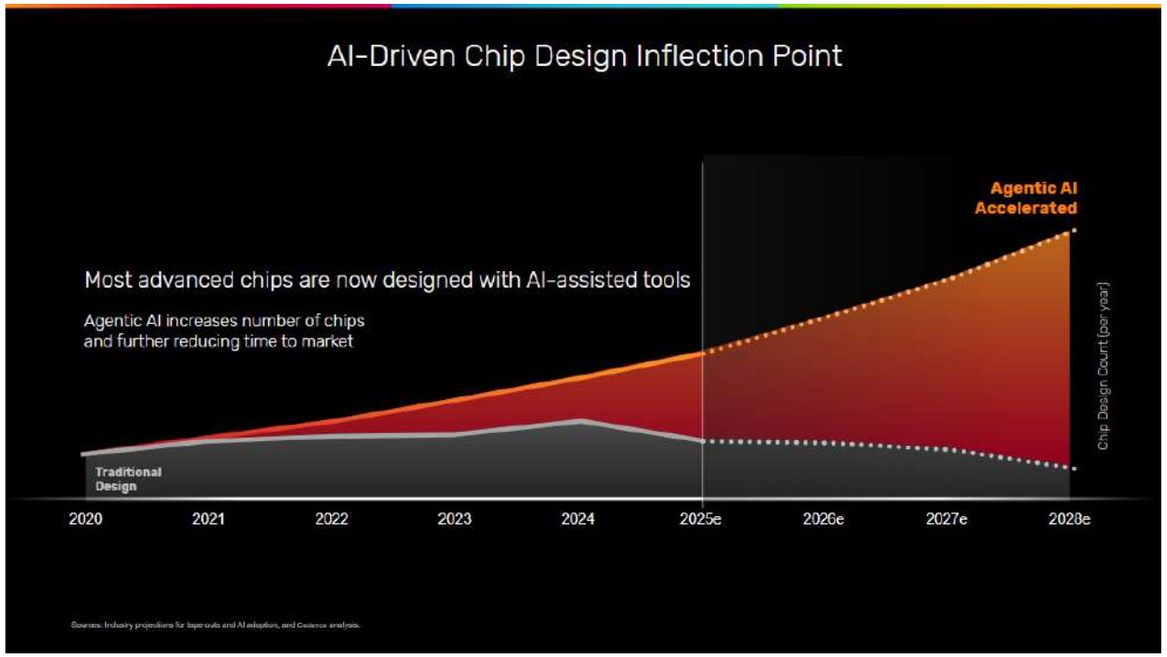
March 19, 2025 10:30 AM PDT · Updated 10 days ago


"[…] shift from designing single chips to AI server systems with hundreds or even thousands of chips in them while aiming to release a new server each year"

"These are very complex and difficult to design," Ghazi said of new AI computers. "The pressure engineers are feeling today is not only complexity, **it is complexity and the pace by when they need to deliver these products, as well as the cost**."

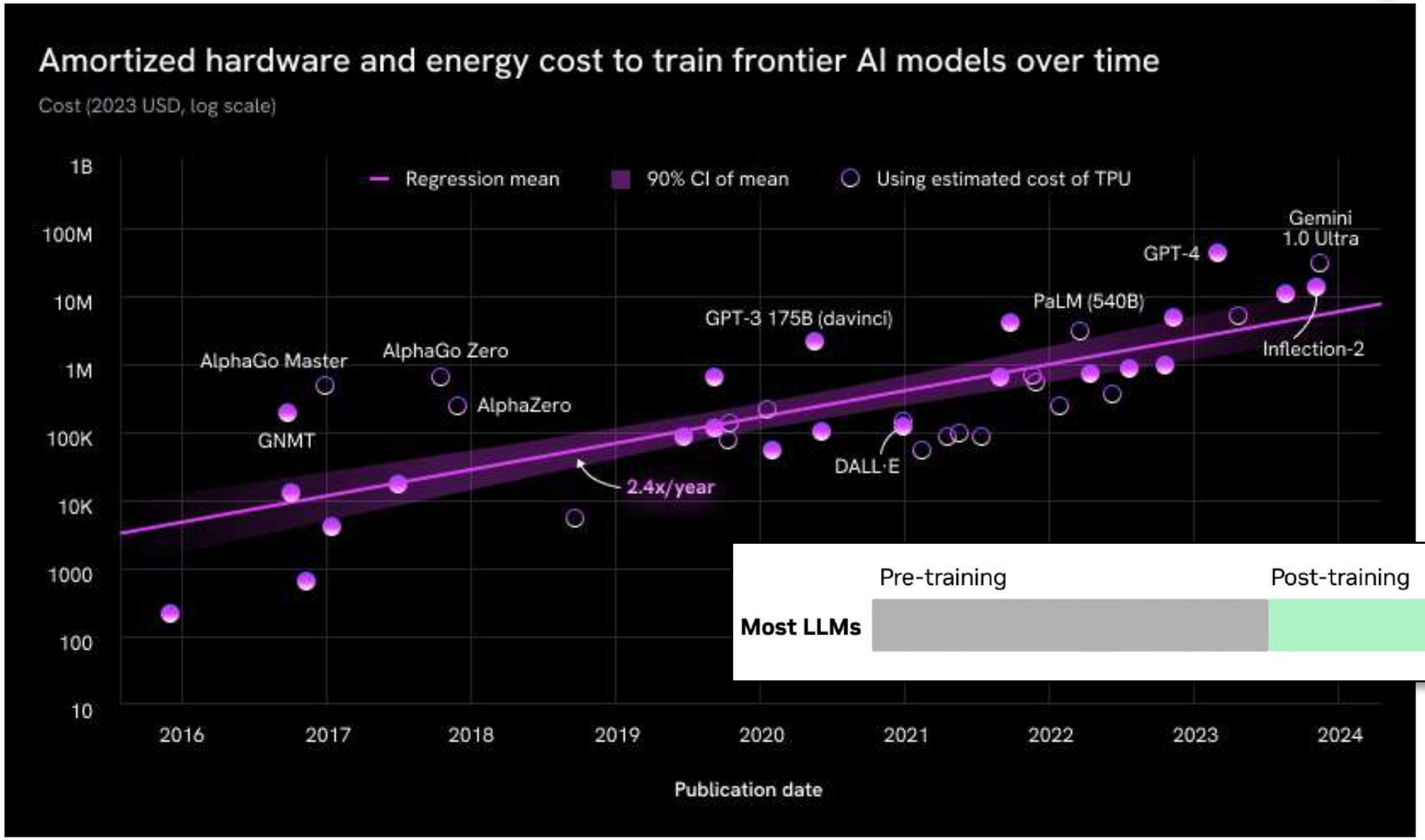
**AgentEngineer**: an "agent" that a human engineer can give instructions to.

https://www.reuters.com/technology/artificial-intelligence/synopsys-lays-out-strategy-ai-agents-design-computer-chips-2025-03-19/



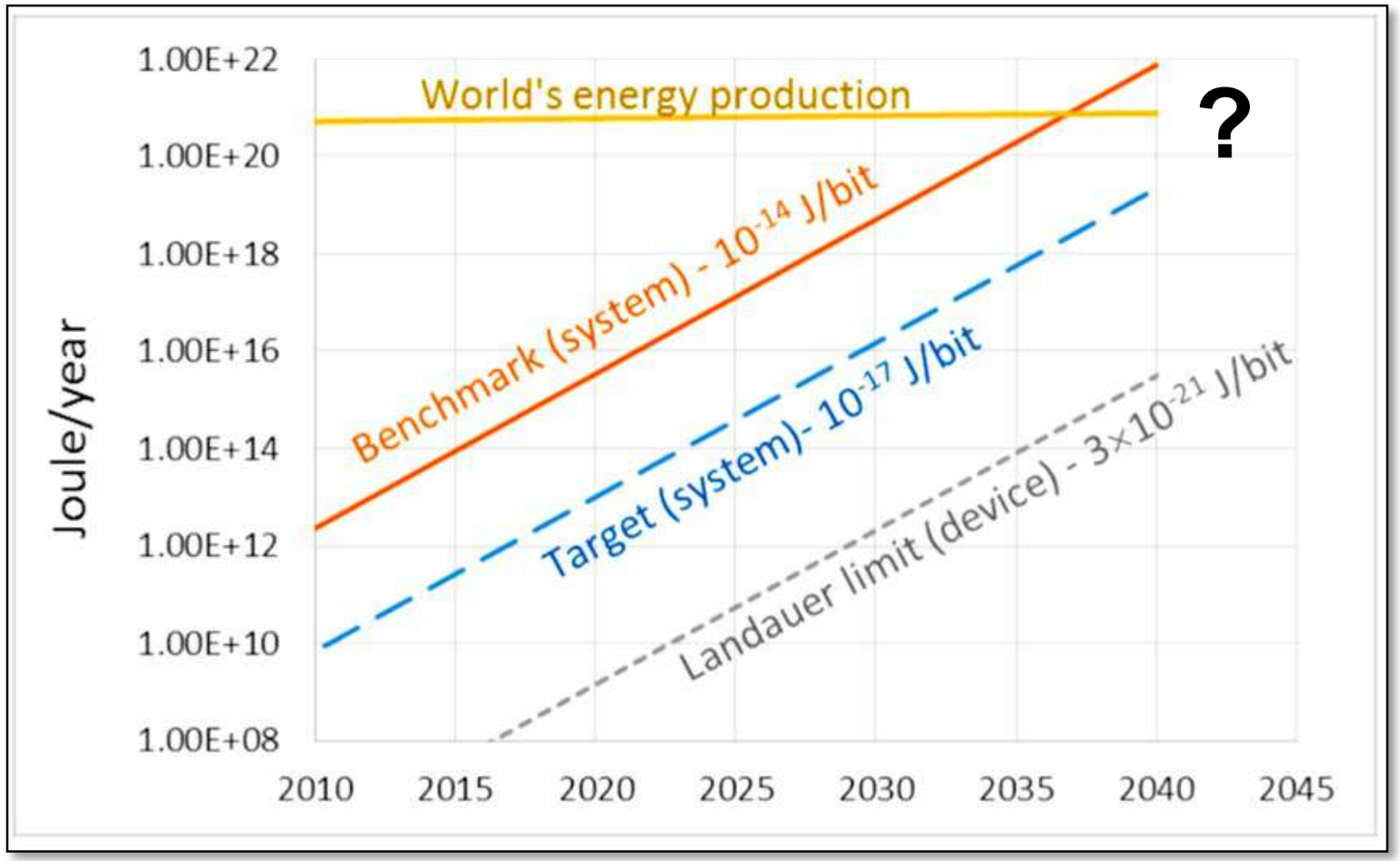


https://community.cadence.com/cadence_blogs_8/b/corporate-news/posts/agentic-ai-soc-system-engineering


Amortized hardware and energy cost to train frontier AI models over time
Cost (2023 USD, log scale)
1B
100M
10M
1M
100K
10K
1000
100
10
Regression mean
90% CI of mean
Using estimated cost of TPU
AlphaGo Master
AlphaGo Zero
AlphaZero
GNMT
GPT-3 175B (davinci)
PaLM (540B)
GPT-4
Gemini 1.0 Ultra
Inflection-2
DALL·E
2.4x/year
2016
2017
2018
2019
2020
2021
2022
2023
2024
Publication date
Pre-training
Post-training
Inference
Most LLMs

# What challenges do computer engineers face?

- A full stack understanding (algorithm to transistors)
- Co-design
- Deal with unseen complexities (server-scale AI machines, chiplets, trillions of transistors,…)
- Application-specific architectures for entirely new workloads
- Power barriers
- Shorter time to market
- Drastically increased need to keep up with state-of-the-art technological developments across the entire compute stack.
- Ethical issues
- …




Estimated total energy expenditure for computing, directly related to the number of raw bit transitions. Source: SIA/SRC.

# What skills do computer engineers have to have?

- **ML foundations**. A hardware engineer who cannot reason about tensor operations, quantization, sparsity, and memory access patterns cannot make good architectural decisions for ML workloads. Chip designers who use AI/ML as a black box will not be able to judge whether its outputs are reasonable or correct.
- **Hardware-software co-design**. This includes understanding how compilers, runtimes, and frameworks like TensorFlow or PyTorch map onto hardware primitives. The engineer needs to trace a workload from algorithm to register file.
- **Energy-aware design**. This spans from device-level techniques (voltage scaling, clock gating) through micro-architecture through system-level power management, with awareness of how workload characteristics interact with power budgets.
- **AI-assisted EDA**. Generative AI is now embedded in synthesis, verification, and physical design tools. Using these tools effectively requires enough domain knowledge to prompt them well and to audit their outputs. The shift from "run the tool" to "evaluate what the tool produced" is significant.
- **Verification at scale**. As SoC complexity grows and multi-die chiplet assemblies become common, verification is one of the hardest open problems. Engineers need both formal and simulation-based methods, and increasingly AI-assisted coverage analysis.

# Foundation vs applied

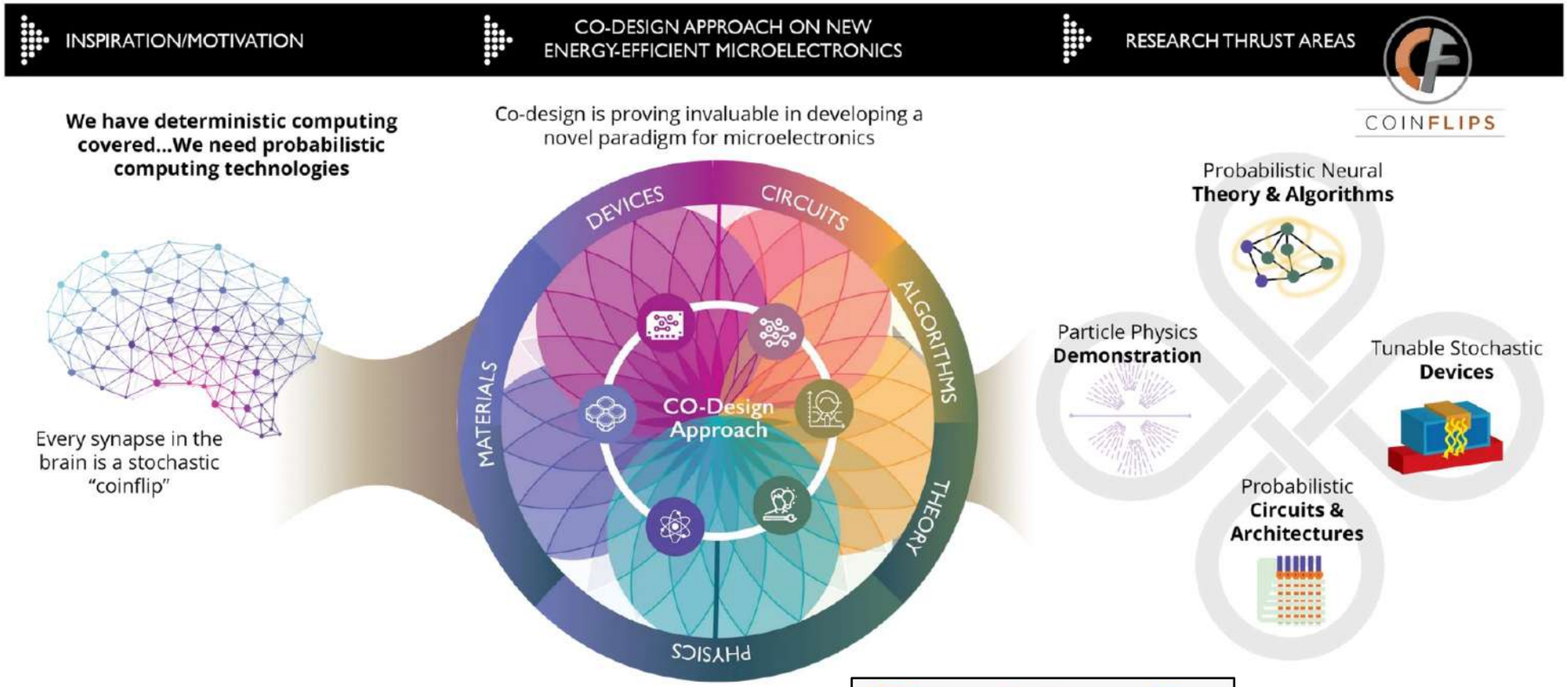
- By the end of this term, most tools you will have used will already have changed and/or will be obsolete.
- **What stays is the foundations.**
- Knowing state-of-the-art industry tools is NOT a career strategy.
- Being successful on the job market means you can adapt to new tools because you know the foundations.

# Progress in AI/ML is currently largely dependent on advances in hardware

**Thus this course.**

**That's why this is a timely topic of interest to any computer engineer.**

# Co-design




https://coinflipscomputing.org

Sandia National Laboratories · Oak Ridge National Laboratory · The University of Tennessee Knoxville · Temple University · Texas · NYU

teuscher Lab
teuscher-lab.com

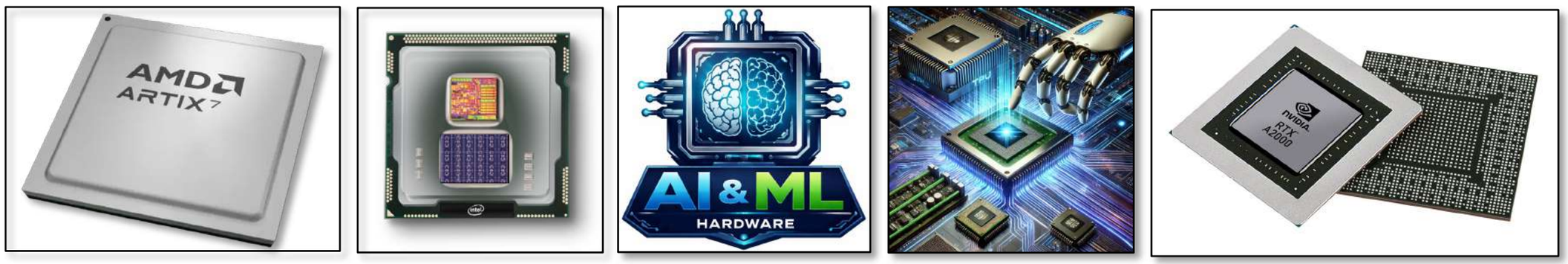
Portland State University

Christof Teuscher

ECE 410/510: Hardware for AI and ML, Spring 2026

# Course organization and details

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu

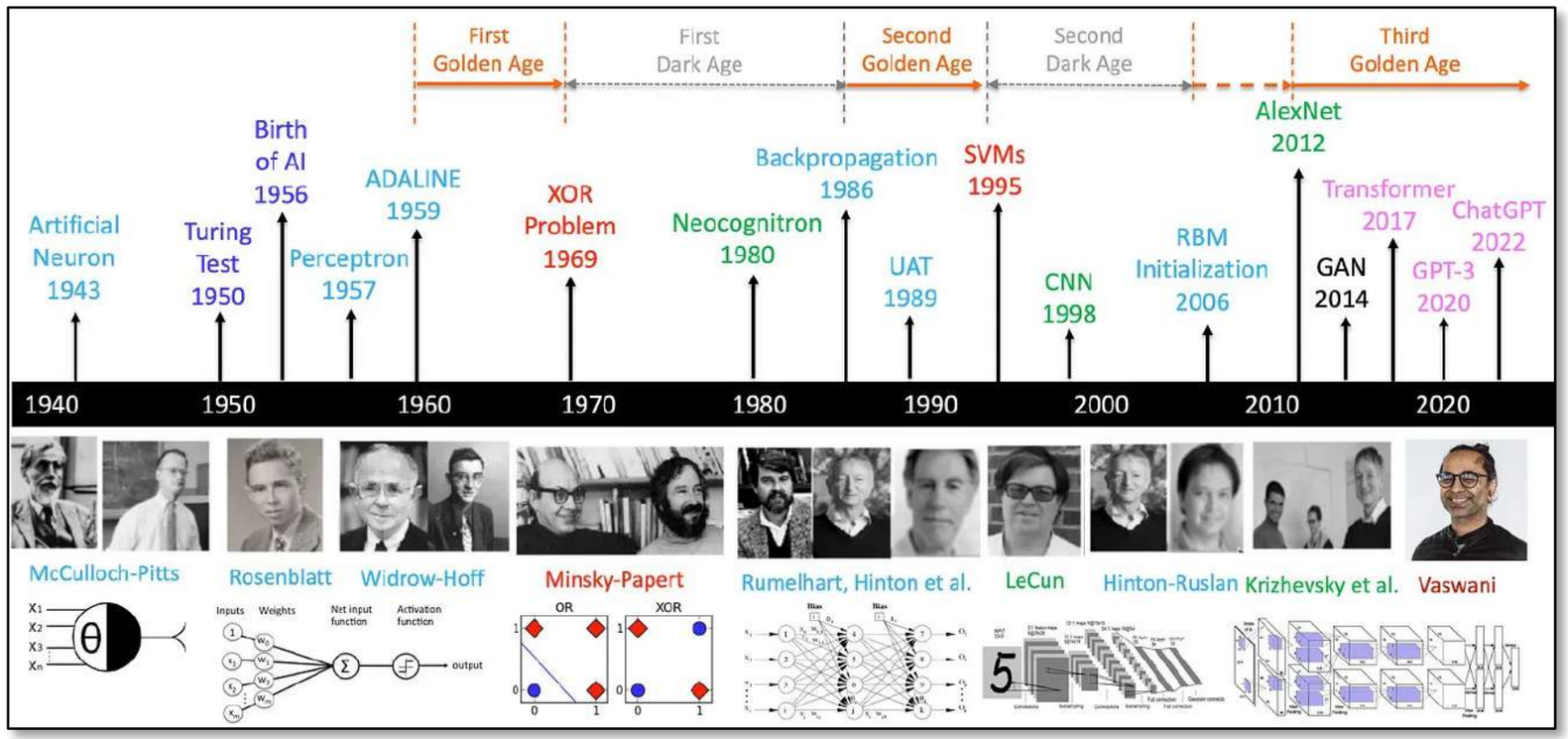



teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

# Once upon a time…

Systems Science & ECE professor George Lendaris retired in 2011. With him, the one and only neural network course we offered died because there was not enough interest.




https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b

ECE 486/586, Spring 2016

# Final Project: A Simple Pipelined Neuromorphic Processor

## Learning Goals

The main goal of this project is to expose you to the learning experience of designing a processor from the beginning to the end. You will explore different instruction set architectures for the task at hand, pick the most appropriate one, and then build a functional simulator of simple pipelined processor specialized to perform the operations for neural networks.

**Additional learning goals:**

- Explore design trade-offs.
- Learn how to build a functional simulator.
- Apply what we learned in class to a real-world problem.
- Propose a solution to a challenging problem you may not be familiar with.
- Expand your horizon, learn new things.
- Deal with ambiguity .
- Work on a team.
- Write a report.

## Introduction and Motivation

Neuromorphic chips are chips that model functions similar to what the human brain performs. The following two articles provide an initial overview:

- http://semiengineering.com/neuromorphic-chip-biz-heats-up
- https://www.technologyreview.com/s/526506/neuromorphic-chips

There are many ongoing projects, both nationally and internationally, with the goal to replicate some of the brain's functionality at a large scale. Some projects are based on general-purpose processors (e.g., ARM processors for the SpiNNaker project, http://apt.cs.manchester.ac.uk/projects/SpiNNaker/project), while other projects chose to build highly specialized architectures based on novel types of devices (e.g., IBM's TrueNorth chip, http://www.research.ibm.com/articles/brain-chip.shtml, or Intel's spintronic-based approach, https://www.technologyreview.com/s/428235/intel-reveals-neuromorphic-chip-design).

For the purpose of this project, we will be working with a very simplified version of a neural network: feed-forward linear threshold neural networks. Note that most of today's neuromorphic architectures are non-linear and recurrent, but we do not need to go to that level of complexity to learn about the basic trade-offs a computer architect faces with neuromorphic architectures.

**Your task is to design, model, simulate, test, and benchmark a simple pipelined neuromorphic processor for feed-forward linear threshold neural networks. The processor should be optimized for throughput, i.e., the more data you can feed through the neural network in a given time interval, the better.**

To model, simulate, and test your processor design, you will write your own functional simulator in a high-level language of your choice (such as C, C++, Java, Python, etc.). The goal is not to build a simulator that is hardware-accurate for each gate or circuit involved. However, your simulator needs to be cycle-accurate, i.e., be able to capture accurately the total number of clock cycles a program takes to execute. If you prefer (in case you are not sufficiently familiar with a high-level language), you can use a hardware description language, such as VHDL or Verilog, and build a more realistic circuit-level simulation. Note that this is neither required nor will it get you any extra points.



teuscher.:Lab
teuscher-lab.com
Emerging Computing Models and Technologies
Christof Teuscher
ECE 486/586
Spring 2016
Portland State UNIVERSITY

All the typical parts of the architecture need to be simulated, such as the **instruction decoder**, **ALU**, **registers** (if applicable), **pipeline** (if applicable), data and instruction **memory**, etc. The functional simulator that you will build needs to capture the effect of running the simulated program on the simulated machine state (which includes the typical parts listed above).

Your simulator needs to be able to load a text file that contains a memory image as an input. The memory image represents the initial state of the simulated program. It needs to contain the program that simulates your neural network, the network weights, and the data to be fed into the network. That means you also have to simulate a basic memory (see more details below). The output of the simulator needs to be another memory image (dump) that shows output data of the neural network. The output can be appended to the input file or written into a separate file.

It is up to the team to define the file format that is appropriate for their implementation. The following is simply an example:

- The data in the memory image should be 32-bits wide. Each line in the text file represents one word (4 bytes) of memory shown in the hexadecimal (base-16) format. The addresses start at "0," from the first line in the image and then increase by "4" at every next line. The addresses do not need to be specified in the memory image and are implicitly obvious from the line number in the image file. For example, if you want to read the contents of memory address $(20)_{10}$, then those contents can be found on the sixth line in the memory image.

Assume that the PC and all the general purpose registers (if applicable) are initialized to "0." As you simulate each instruction in the program, you will need to keep track of the machine state and make necessary changes to the state based on the impact of the simulated instruction. For example, if an instruction writes a new value to register R6, then you will update the register R6 with the instruction result. Each instruction will also update the PC, which will in turn cause a new instruction to be fetched and simulated. You will continue the simulation until you encounter some sort of "HALT" instruction.

## The Neural Network to be Simulated

Your task is to simulate a fixed feed-forward threshold neural network as depicted in Figure 1. The network has 4 input neurons, 4 hidden neurons, and 4 output neurons. Feed-forward means that the network does not have any cycles. In other words, it can be organized into layers that are only connected by "forward" connections as shown in Figure 1. There is no learning happening in this network. The weights are fixed and should be stored in the initial memory file.

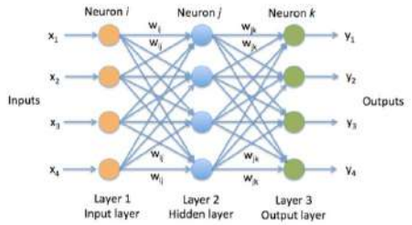



**Figure 1.** A 3-layer neural network with 4 input neurons, 4 hidden neurons, and 4 output neurons.

4/6/16 | rev 1

**2016**
**ECE 586**

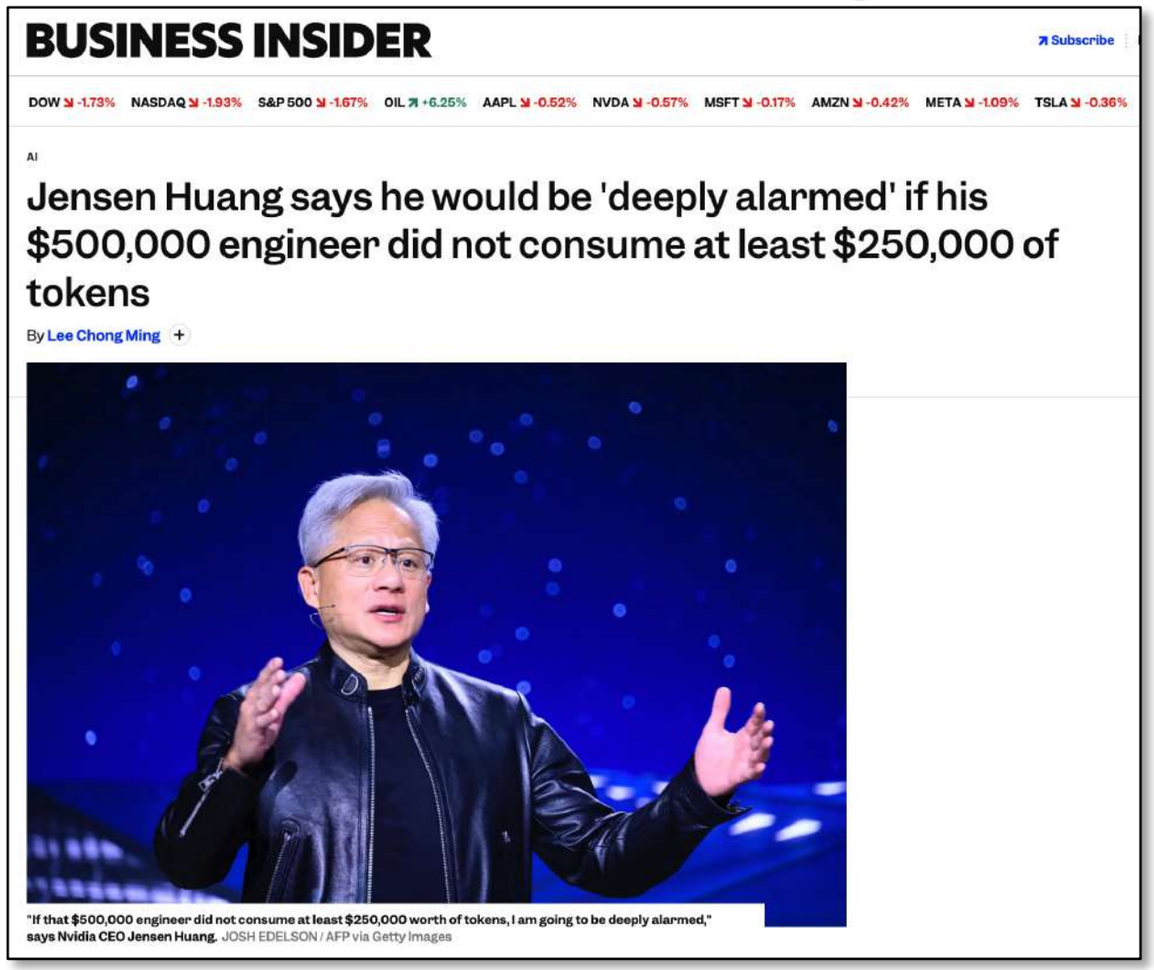
# Why this course?

Christof Teuscher • teuscher@pdx.edu



# AI proficiency is not optional anymore

- AI is everywhere
- AI won't go anywhere
- On your job, you will be expected to use AI
- You need to be experts in getting work done with AI



BUSINESS INSIDER

Subscribe

DOW -1.73% NASDAQ -1.93% S&P 500 -1.67% OIL +6.25% AAPL -0.52% NVDA -0.57% MSFT -0.17% AMZN -0.42% META -1.09% TSLA -0.36%

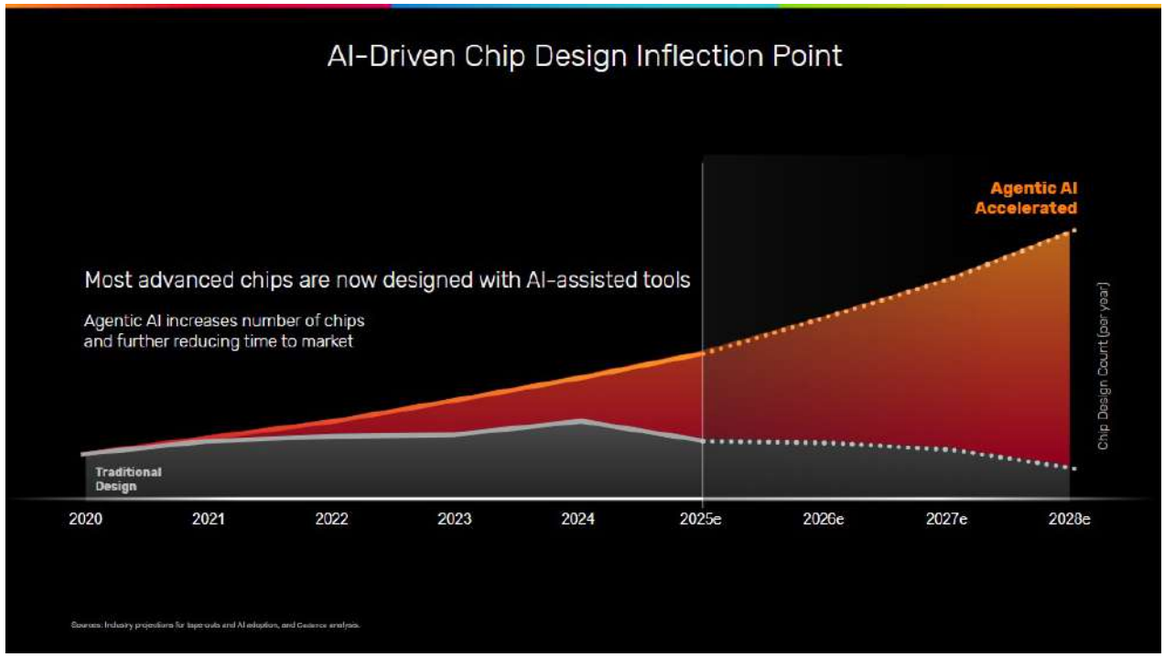
AI

Jensen Huang says he would be 'deeply alarmed' if his $500,000 engineer did not consume at least $250,000 of tokens

By Lee Chong Ming +

"If that $500,000 engineer did not consume at least $250,000 worth of tokens, I am going to be deeply alarmed," says Nvidia CEO Jensen Huang. JOSH EDELSON / AFP via Getty Images

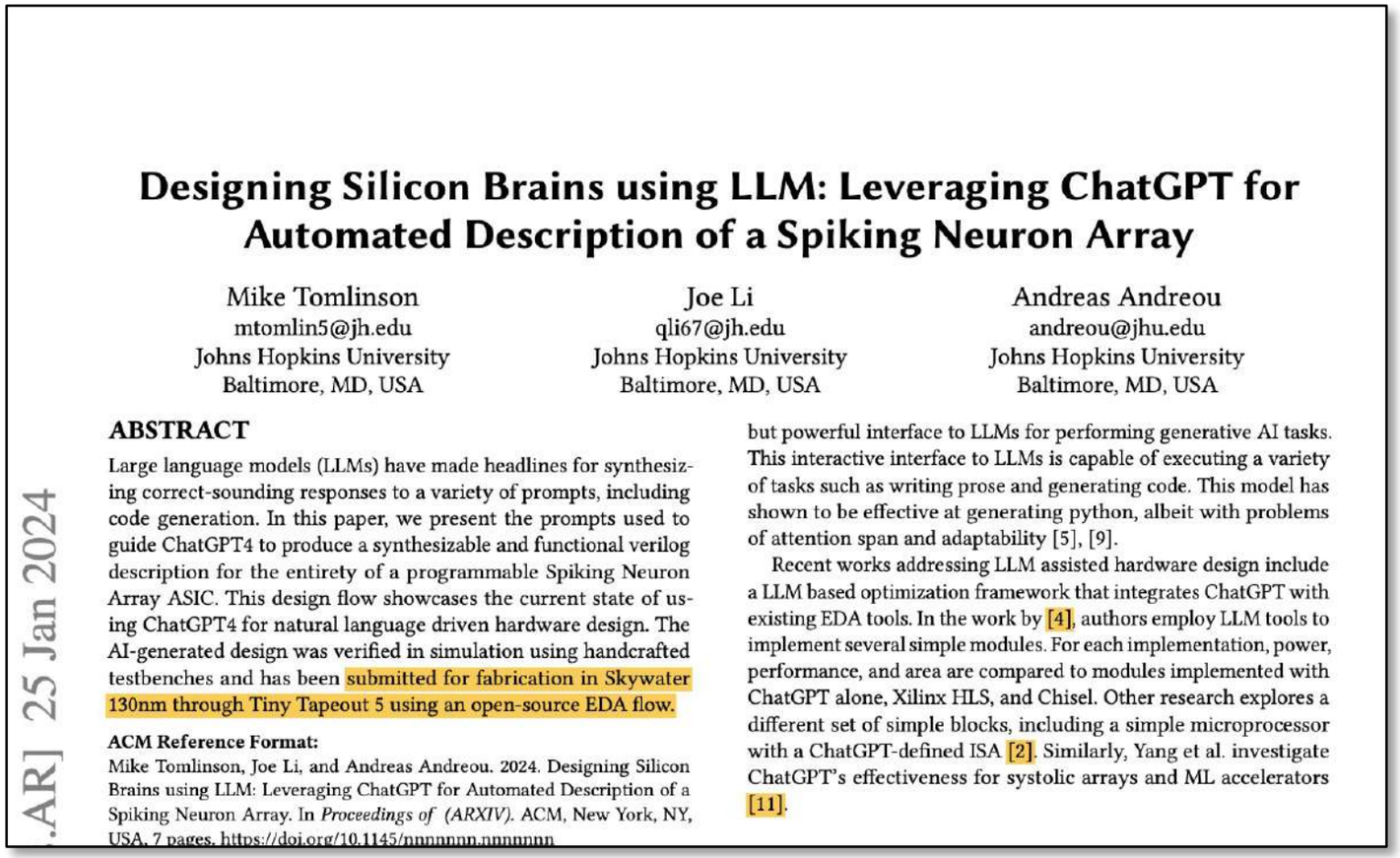
https://www.businessinsider.com/jensen-huang-500k-engineers-250k-ai-tokens-nvidia-compute-2026-3





https://community.cadence.com/cadence_blogs_8/b/corporate-news/posts/agentic-ai-soc-system-engineering

# Inspiration and course design



# Designing Silicon Brains using LLM: Leveraging ChatGPT for Automated Description of a Spiking Neuron Array

Mike Tomlinson
mtomlin5@jh.edu
Johns Hopkins University
Baltimore, MD, USA

Joe Li
qli67@jh.edu
Johns Hopkins University
Baltimore, MD, USA

Andreas Andreou
andreou@jhu.edu
Johns Hopkins University
Baltimore, MD, USA



**ABSTRACT**

Large language models (LLMs) have made headlines for synthesizing correct-sounding responses to a variety of prompts, including code generation. In this paper, we present the prompts used to guide ChatGPT4 to produce a synthesizable and functional verilog description for the entirety of a programmable Spiking Neuron Array ASIC. This design flow showcases the current state of using ChatGPT4 for natural language driven hardware design. The AI-generated design was verified in simulation using handcrafted testbenches and has been submitted for fabrication in Skywater 130nm through Tiny Tapeout 5 using an open-source EDA flow.

**ACM Reference Format:**
Mike Tomlinson, Joe Li, and Andreas Andreou. 2024. Designing Silicon Brains using LLM: Leveraging ChatGPT for Automated Description of a Spiking Neuron Array. In *Proceedings of (ARXIV).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

but powerful interface to LLMs for performing generative AI tasks. This interactive interface to LLMs is capable of executing a variety of tasks such as writing prose and generating code. This model has shown to be effective at generating python, albeit with problems of attention span and adaptability [5], [9].

Recent works addressing LLM assisted hardware design include a LLM based optimization framework that integrates ChatGPT with existing EDA tools. In the work by [4], authors employ LLM tools to implement several simple modules. For each implementation, power, performance, and area are compared to modules implemented with ChatGPT alone, Xilinx HLS, and Chisel. Other research explores a different set of simple blocks, including a simple microprocessor with a ChatGPT-defined ISA [2]. Similarly, Yang et al. investigate ChatGPT's effectiveness for systolic arrays and ML accelerators [11].

https://arxiv.org/abs/2402.10920

# How do we teach cutting-edge new technology?

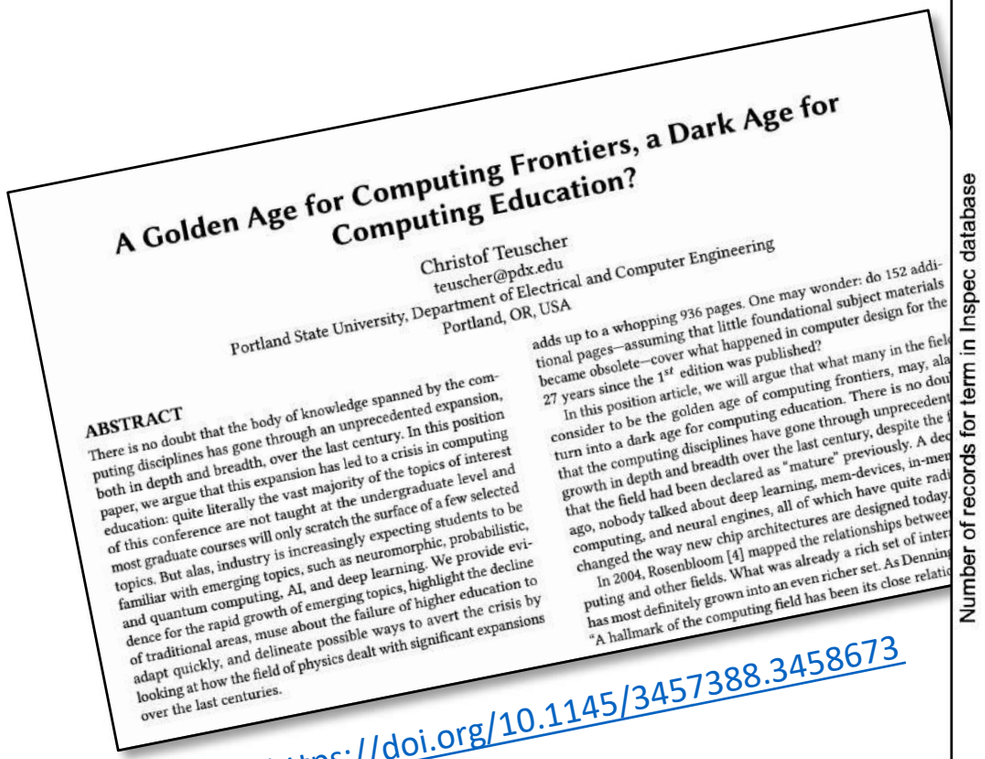
Nothing seems linear and predictable

Everything is complex

Everything is connected

Everything moves fast

# The need for new educational approaches



A Golden Age for Computing Frontiers, a Dark Age for Computing Education?

Christof Teuscher
teuscher@pdx.edu
Portland State University, Department of Electrical and Computer Engineering
Portland, OR, USA

ABSTRACT

There is no doubt that the body of knowledge spanned by the computing disciplines has gone through an unprecedented expansion, both in depth and breadth, over the last century. In this position paper, we argue that this expansion has led to a crisis in computing education: quite literally the vast majority of the topics of interest of this conference are not taught at the undergraduate level and most graduate courses will only scratch the surface of a few selected topics. But alas, industry is increasingly expecting students to be familiar with emerging topics, such as neuromorphic, probabilistic, and quantum computing, AI, and deep learning. We provide evidence for the rapid growth of emerging topics, highlight the decline of traditional areas, muse about the failure of higher education to adapt quickly, and delineate possible ways to avert the crisis by looking at how the field of physics dealt with significant expansions over the last centuries.

adds up to a whopping 936 pages. One may wonder: do 152 additional pages—assuming that little foundational subject materials became obsolete—cover what happened in computer design for the 27 years since the 1st edition was published?

In this position article, we will argue that what many in the field consider to be the golden age of computing frontiers, may, ala turn into a dark age for computing education. There is no doubt that the computing disciplines have gone through unprecedented growth in depth and breadth over the last century, despite the fact that the field had been declared as "mature" previously. A decade ago, nobody talked about deep learning, mem-devices, in-memory computing, and neural engines, all of which have quite radically changed the way new chip architectures are designed today.

In 2004, Rosenbloom [4] mapped the relationships between computing and other fields. What was already a rich set of interactions has most definitely grown into an even richer set. As Denning stated, "A hallmark of the computing field has been its close relation

https://doi.org/10.1145/3457388.3458673

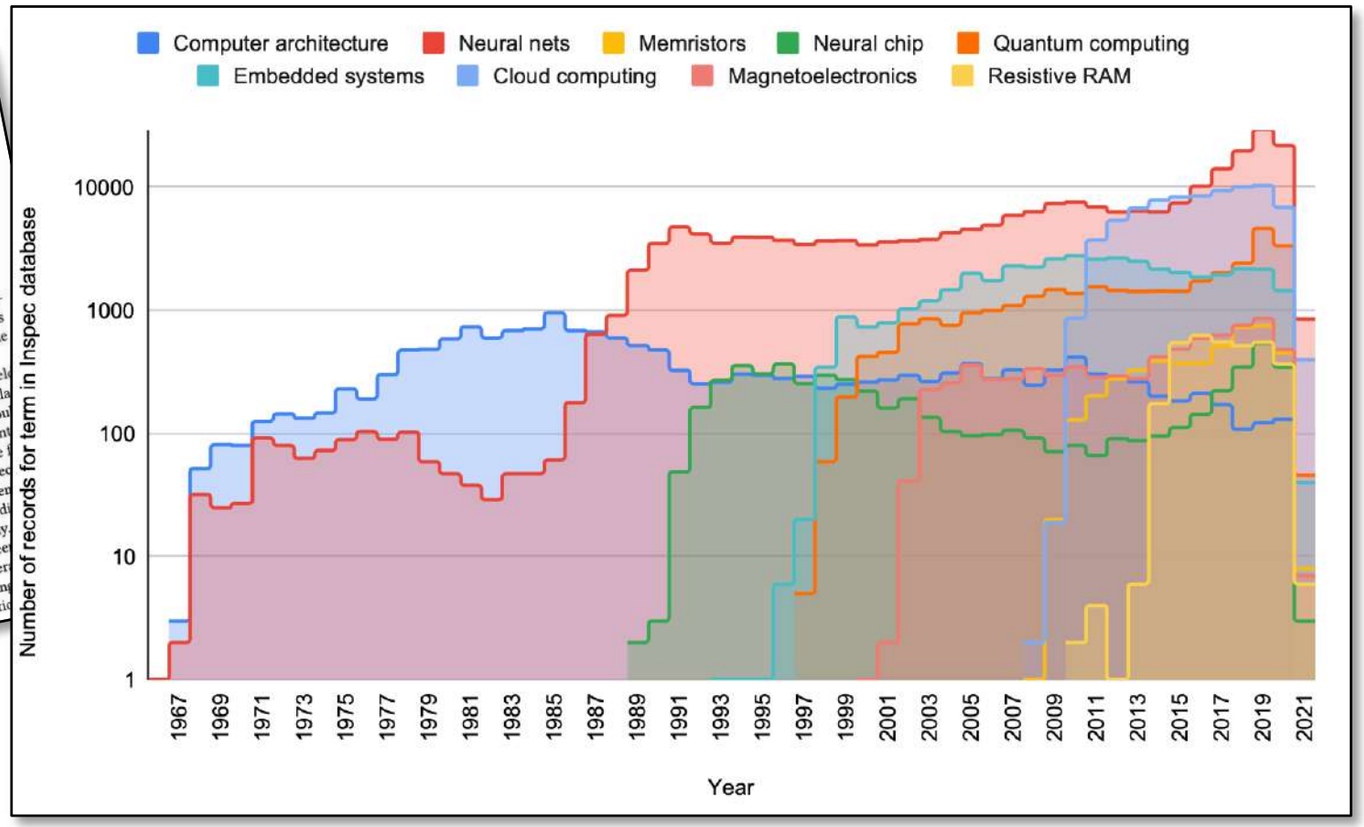


**Table 2: List of graduate electives for an M.S. degree in computer engineering with a specialization in hardware and computer architecture at Boston University.**

| |
|---|
| EC 513 Computer Architecture |
| EC 527 High-Performance Programming with Multicore and GPUs |
| EC 535 Introduction to Embedded Systems |
| EC 551 Advanced Digital Design with Verilog and FPGA |
| EC 561 Error-Control Codes |
| EC 571 VLSI Principles and Applications |
| EC 580 Modern Active Circuit Design |
| EC 582 RF/Analog IC Design Fundamentals |
| EC 713 Parallel Computer Architecture |
| EC 749 Interconnection Networks for Multicomputers |
| EC 752 Theory of Computer Hardware Testing |
| EC 753 Fault-Tolerant Computing |
| EC 757 Advanced Microprocessor Design |
| EC 772 VLSI Graduate Design Project |

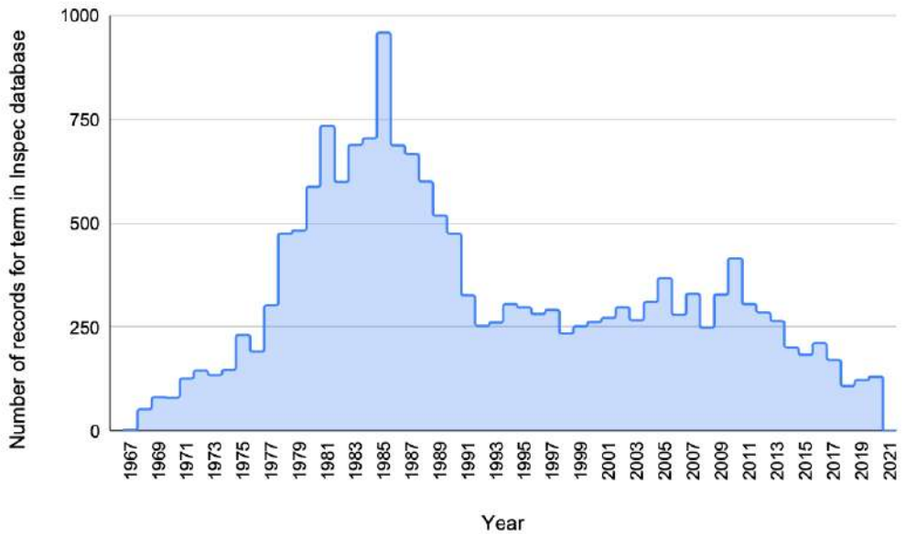



**Figure 1: The number of records over the years that were found for the "computer architecture" thesaurus term. The field had its peak around 1985 and has since been in decline. Data source: Elsevier Engineering Village thesaurus.**

# Vibe Your Chip: An Unconventional Approach to Hardware Design Education in the Age of LLMs

Niklas Anderson[1*], Brandon Hippe[1,2], Tucker Mastin[1], Christof Teuscher[1]

[1*]Department of Electrical and Computer Engineering (ECE), Portland State University, 1900 SW 4th Ave, Portland, 97201, OR, US.
[2]Department of Electrical and Computer Engineering (ECE), Villanova University, 800 E. Lancaster Ave, Villanova, 19085, PA, US.

*Corresponding author(s). E-mail(s): niklas2@pdx.edu;
Contributing authors: bhippe@villanova.edu; tmastin@pdx.edu; teuscher@pdx.edu;

**Abstract**

The rapid proliferation of AI and machine learning applications has driven unprecedented innovation in computing hardware, from neuromorphic processors and memristive devices to quantum accelerators and in-memory computing architectures. However, traditional computer engineering curricula remain anchored in conventional digital design principles, creating a widening gap between industry demands and graduate competencies. This paper presents a pedagogical experiment for integrating emerging AI hardware paradigms into undergraduate and graduate computing education by relying heavily on Large Language Models (LLMs) for the design of AI accelerator hardware. Our assessments show that **89%** of surveyed students reported positive knowledge improvement ($M = 30.6$ points on a **100**-point scale), with consistent learning gains across graduate, undergraduate, and accelerated B+M students. Students produced substantial code artifacts, with a median of **2,847** lines of code per repository, and **90.5%** of repositories contained both Python and SystemVerilog or Verilog, reflecting genuine engagement with the hardware-software co-design pipeline. Students who entered with little or no hardware design experience produced working implementations of AI/ML accelerators and custom ASIC designs. Besides traditional lectures, the course design included "codefests," weekly presentations, self-grading, and unconventional incentives for students. All course materials

**Last year's class**

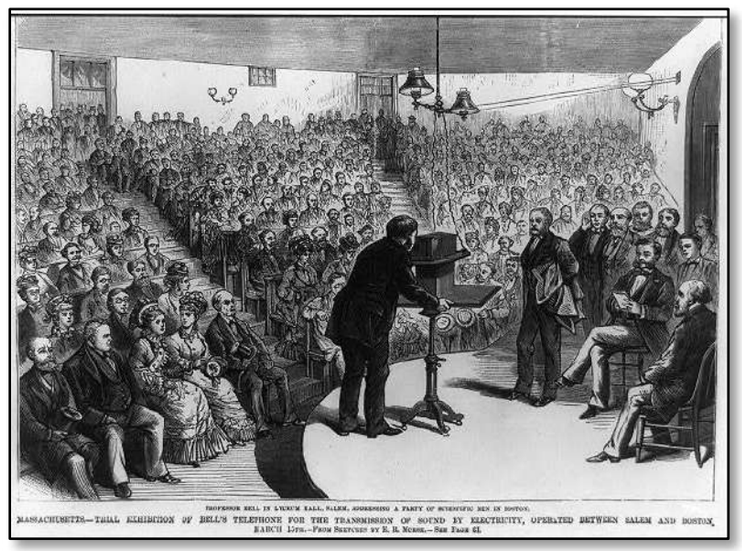
# What's the role of education?

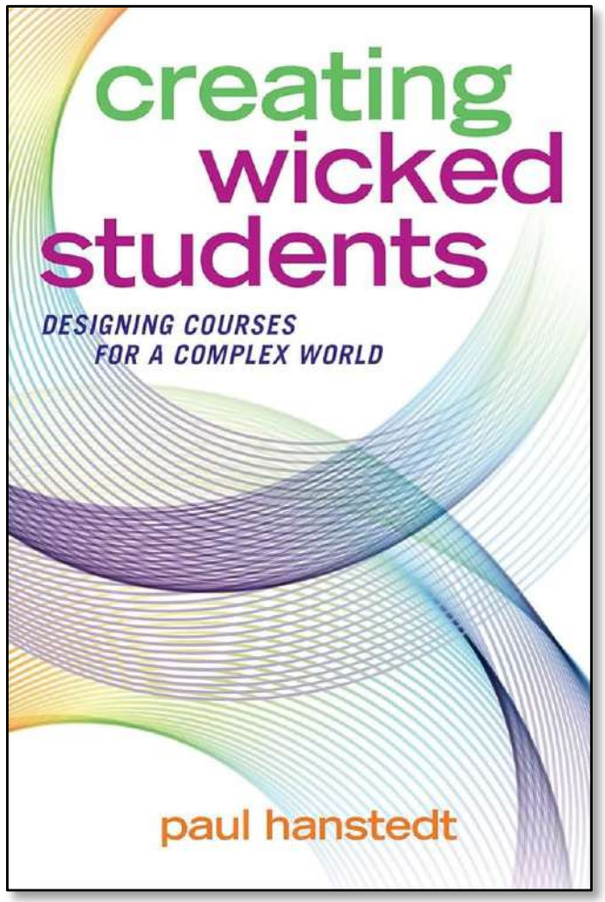
- **The old model**:
  - `knowledge + skills = success`
  - Domain-specific compartmentalization + professor with authority + lectures + fixed curriculum + assessment → **terminal degree**
- **The new model**:
  - creative thinking
  - complex problem solving
  - the ability to learn
  - emotional intelligence

    → **learning never ends**

    → **be invested in your own learning**

    → **AI is your friend**

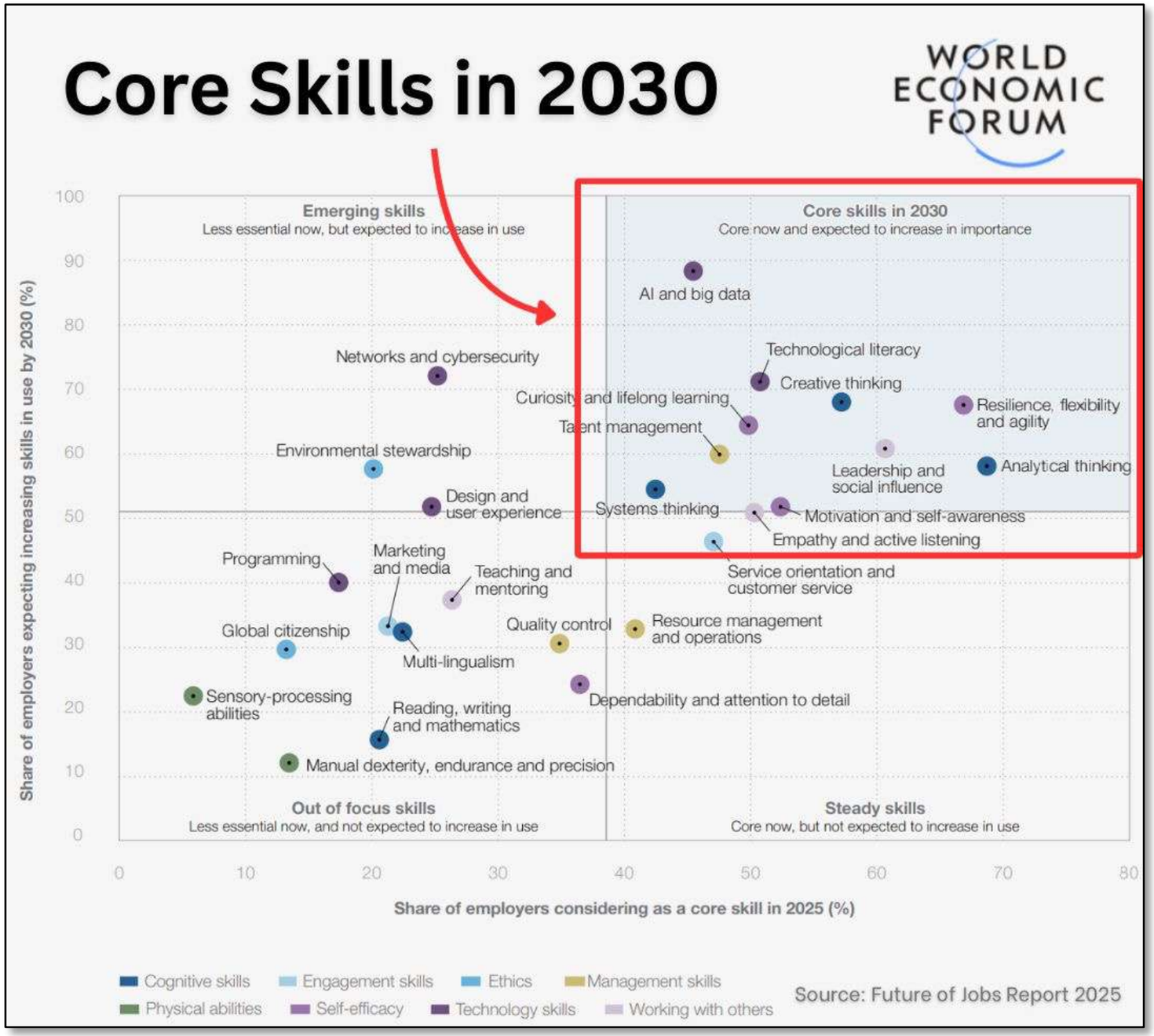





A "wicked student," as coined by Paul Hanstedt, is a learner who takes authority over their own education, **capable of handling complex, ambiguous, and "wicked" problems, rather than just memorizing facts for a test**. They are **creative**, **adaptable**, and **comfortable with failure** as part of the learning process.

# Core Skills in 2030

WORLD ECONOMIC FORUM




Source: Future of Jobs Report 2025

# Let’s talk about the fine print…

# My assumptions and expectations

- Intrinsically motivated to learn about this subject.
- Willing to be challenged, to fail on the way, and to get out of your comfort zone.
- Willing to show up and put in the work.
- Willing to not take shortcuts.
- Willing to always do a little more (rather than the bare minimum).
- Willing to assume responsibility for your actions (and/or non-actions).
- Willing to share and to engage.
- Willing to contribute to a positive learning environment

# If you are here to

- …cheat
- …take shortcuts
- …take the easy route
- ...not be here

**Well, you are the one who is missing out!**

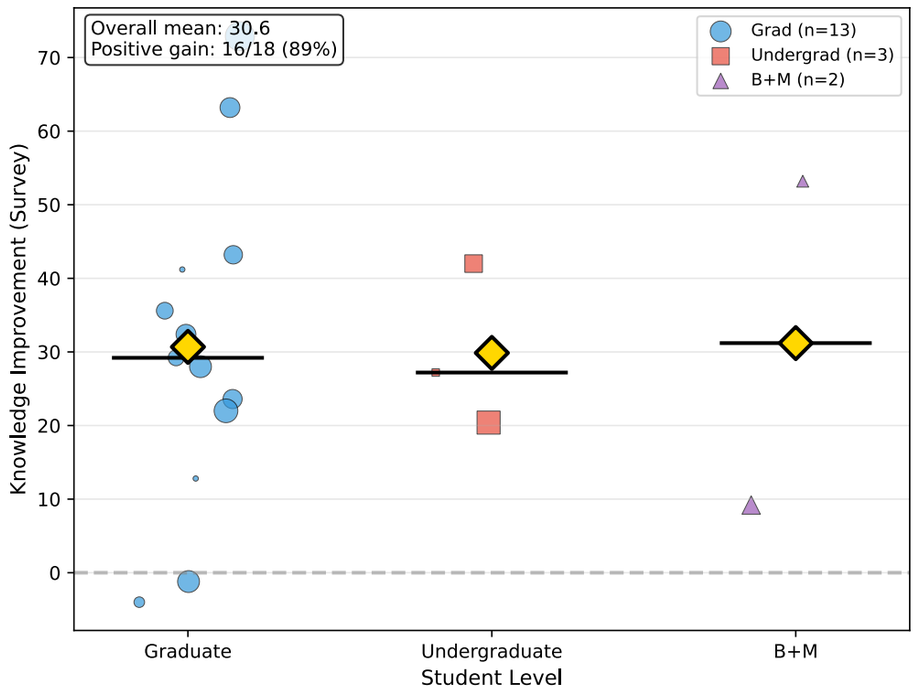
# A word about prerequisites

## Recommendations:

- Undergraduate: ECE 371 and ECE 351
- Graduate: ECE 485

## Ideal knowledge:

- Python or other language
- Github
- Advanced computer architecture, incl. GPUs
- FPGAs
- Analog and digital circuits
- Verilog, SystemVerilog
- Experience with LLMs and vibe coding.



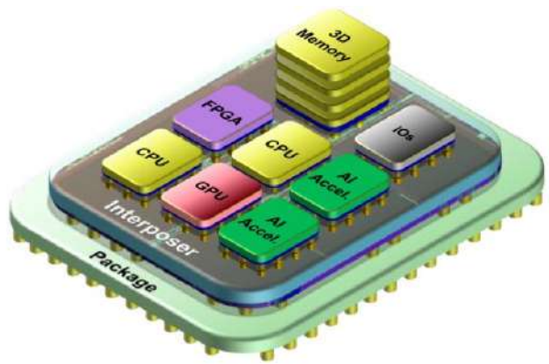
# Learning goals

- Apply the mathematical foundations underlying AI/ML algorithms (linear algebra, calculus, probability) in the context of hardware design and optimization.
- **Explain the principles of HW/SW co-design and justify partitioning decisions for AI/ML workloads.**
- **Map AI/ML algorithms onto specialized hardware through co-design methodologies**.
- Evaluate and optimize HW designs through co-design for computational power, energy efficiency, and flexibility.
- Explain the foundations of neural networks and Large Language Models (LLMs), including CNNs, DNNs, and transformer architectures.
- Describe the architectures of specialized AI/ML hardware including GPUs, TPUs, FPGAs, neuromorphic processors, and in-memory computing systems.
- **Design, implement, and verify a custom hardware accelerator for an AI/ML algorithm using a hardware description language (Verilog or SystemVerilog).**
- **Use modern software and hardware tools, including LLM-assisted design flows, for designing and deploying specialized AI/ML hardware.**
- Critically evaluate LLM-generated hardware descriptions for correctness, efficiency, and synthesizability.
- Use the ASIC design flow (RTL to GDSII) including synthesis, placement, routing, and timing analysis.
- Benchmark hardware implementations against software baselines using appropriate performance metrics.
- Explain emerging computing paradigms, including in-memory computing, memristive devices, and their applications to AI/ML workloads.
- Communicate design decisions, tradeoffs, and results through professional documentation.

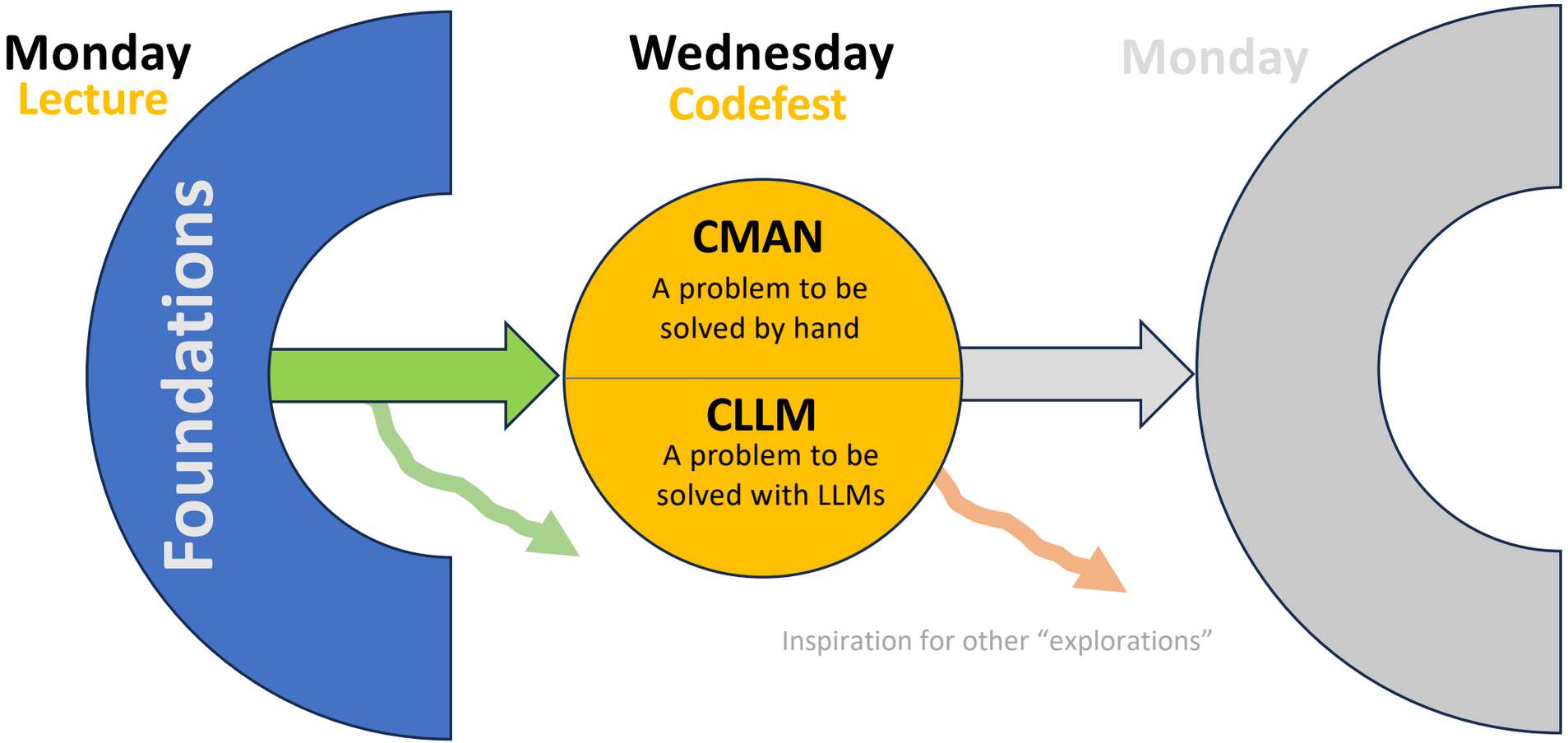
# What this class is not

- Not a class about LLM, AI, ML, ANN, etc.
- …

You will each (co-)design, test, evaluate, etc. an AI/ML accelerator for a chiplet or for a standalone chip.



# A typical week

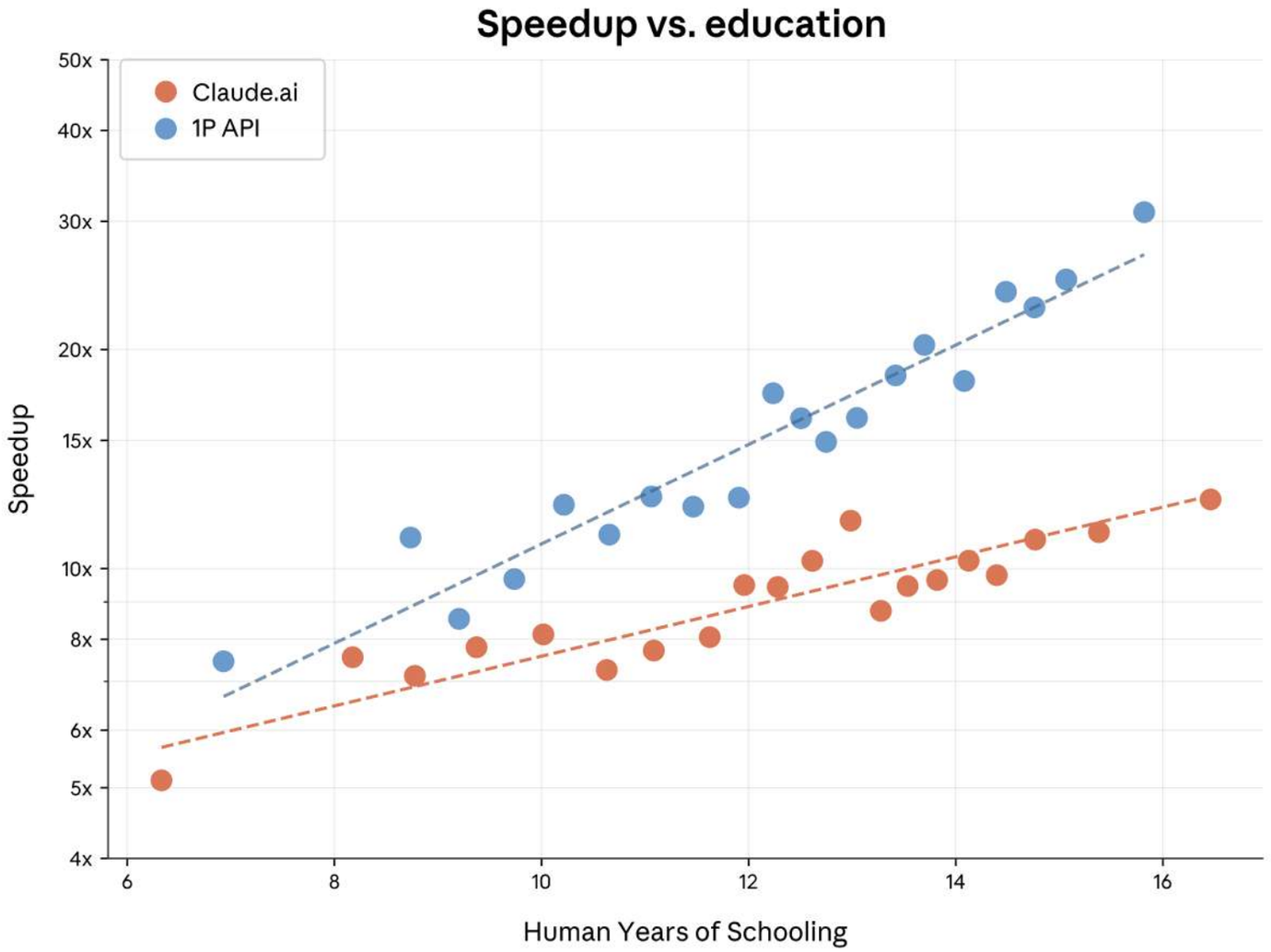


# Pedagogical foundations (1)

Grounded in the literature.

**The foundations are what lasts:**

- Teach foundations, then multiply and scale them up them up with AI.




- The more you know, the more efficient you can use AI.
- AI is a **multiplier**, not a shortcut.
- **The foundations matter!**

**How humans prompt is how Claude responds**

We find a very high correlation between human and AI education, i.e. the number of years of education required to understand a human prompt or the AI's response (countries: $r = 0.925$, $p < 0.001$, $N = 117$; US states: $r = 0.928$, $p < 0.001$, $N = 50$). This highlights the importance of skills and suggests that how humans prompt the AI determines how effective it can be. This also highlights the importance of model design and training. While Claude is able to respond in a highly sophisticated manner, it tends to do so only when users input sophisticated prompts.

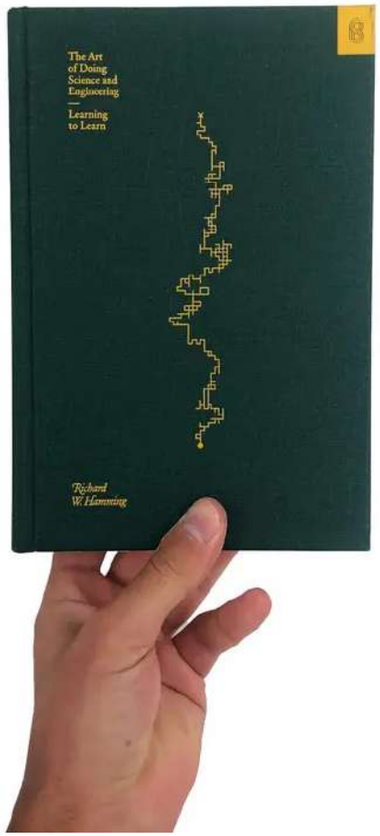


https://www.anthropic.com/research/anthropic-economic-index-january-2026-report

# Why learn if my LLM can do it for me?

- You can't just learn how to craft a prompt for an LLM without first having the experience, exposure and, yes, education to know what the heck you are doing.
- **The more sophisticated your prompt, the more sophisticated your answer**. Or in other words: Garbage in, garbage out.
- Learning is a messy, nonlinear human development process that resists efficiency. AI cannot replace it.

# Does one need to understand AI-assisted designs?




"In science, if you know what you are doing, you should not be doing it.

In engineering, if you do not know what you are doing, you should not be doing it."

— Richard W. Hamming, **The Art of Doing Science and Engineering: Learning to Learn**.

# Pedagogical foundations (2)

**Problem-based or project-based learning**

- Open-ended, real-world problems demand rigorous thinking (you can't fake a working solution) while allowing wildly different approaches.
- The constraint is the outcome quality, not the method.
- I'm not micromanaging how you get to the "solution."

# Pedagogical foundations (3)

**Structured autonomy**:

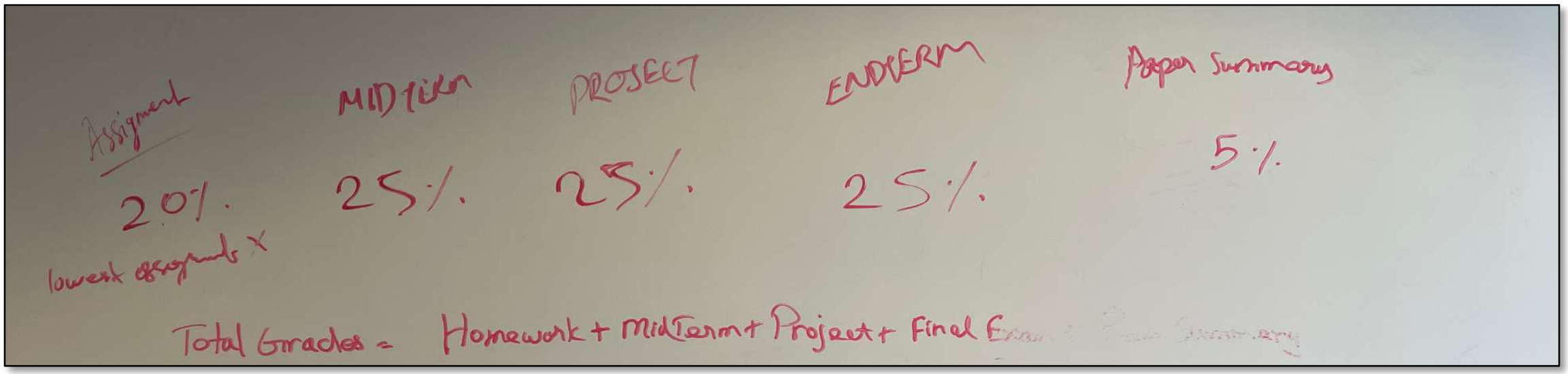
- Scaffold with clear learning objectives, non-negotiable competency thresholds, explicit criteria.
- You choose how to get there.
- Preserves accountability without micromanaging the path.
- Provide flexibility while keeping rigor.

# Pedagogical foundations (4)

**Low-stakes iteration, high-stakes demonstration**.

- Keep practice low-pressure and revisable.
- Reserve high standards for culminating assessments.
- This gives students room to fail productively during learning while still holding a firm line at the end.

# Traditional grading



# This course uses specifications grading

- This course uses specifications grading.
- Each graded deliverable is evaluated as **Satisfactory (S)** or **Not Yet (NY)** against clearly defined specifications.
- Students who receive **Not Yet** can revise and resubmit.
- Final course grades are determined by which bundle of satisfactory work a student completes, not by accumulating points.

- Each student begins the term with **3 revision tokens**. A token is spent each time a student revises a **Not Yet** deliverable beyond the one free revision allowed per codefest deliverable and per milestone. Tokens cannot be used for Milestone 4 or the final examination.
- Additional tokens can be earned in the following ways:
  - Spot-check explanations: the instructor conducts 1–3 spot checks with every student during codefests across the term. A clear, correct verbal explanation over all spot checks earns 1 token.
  - Codefest presentations: every student presents their work at least once during the term. A strong presentation earns 1 token.
  - Consistent Slack contributions throughout the term, as assessed by the instructor, earns 1 token.

# Codefest deliverables

- Each Wednesday codefest produces a **CMAN+CLLM deliverable pair** submitted to your GitHub repository by the following Sunday at 11:59 pm.

- **CMAN must be completed without AI assistance** and is submitted as a handwritten photo or plain-text file with all working shown.
- **CLLM scales the same concept to real-world complexity where AI** tooling is permitted and required.
- Each CMAN+CLLM pair is graded S/NY. Students receiving NY may revise and resubmit within one week of receiving feedback by using a revision token.

- Every student will present their work to the class at least once during the term; the instructor will schedule presentations across sessions.

# Two Canvas quizzes

- **Week 5**: Quiz 1: Canvas-based, open-note, individual. Covers lecture material through Week 4.
- **Week 9**: Quiz 2: Canvas-based, open-note, individual. Covers lecture material through Week 8.
- Graded S/NY with an 80% threshold

# Project milestones

- **The term-long co-design project is assessed at four milestones**. Each milestone is graded S/NY. Milestones 1 through 3 may be revised once with a revision token.

- **Milestone 1** (due Sun Apr 13): Refined Heilmeier answers, software profiling results, roofline analysis, HW/SW partition proposal, hardware interface selection with bandwidth justification, initial SW benchmark.
- **Milestone 2** (due Sun May 4): First working simulation with testbench; interface module in HDL with functional testbench; precision choice documented.
- **Milestone 3** (due Sun May 24): Synthesis attempt with timing and area report; interface integrated with compute core; co-simulation demonstrating end-to-end data flow; or justified scope adjustment.
- **Milestone 4** (due Sun Jun 7): Full deliverable package including source code, synthesis results, benchmark comparison, roofline plot, and design justification report. **This is final and cannot be revised.**

# Final exam

- During week 10, every student submits their M4 package.
- An LLM examiner is provided the documentation and creates a set of exam questions on architectural decisions, tradeoffs, and applied concepts.
- Students are also assessed on their ability to identify when the AI examiner's questions are incorrect or misleading.

- The final exam format will be announced at a later stage.

- The final exam is graded **Pass with Distinction** / **Pass** / **No Pass**.
- **No retake is permitted.**

**Undergraduate grade bundles (ECE 410)**

| Grade | Codefest deliverables (of 10) | Project milestones (of 4) | Final exam | Quizzes (of 2) |
|---|---|---|---|---|
| A | 8+ S | All 4 S | Pass w/ Distinction | Both S |
| A− | 7+ S | All 4 S | Pass | Both S |
| B+ | 6+ S | 3+ S | Pass | Both S |
| B | 5+ S | 3+ S | Pass | 1+ S |
| B− | 4+ S | 2+ S | Pass | 1+ S |
| C+ | 3+ S | 2+ S | Pass | 0+ S |
| C | 2+ S | 1+ S | Pass | 0+ S |
| C− | 1+ S | 1+ S | Pass | 0+ S |
| D+ | 0 S | 0+ S | Pass | 0+ S |
| D | 0 S | 0+ S | Pass | 0+ S |
| F | 0 S | 0 S | No pass | 0 S |

**Graduate and B+M grade bundles (ECE 510)**

| Grade | Codefest deliverables (of 10) | Project milestones (of 4) | Final exam | Quizzes (of 2) |
|---|---|---|---|---|
| A | 9+ S | All 4 S | Pass w/ Distinction | Both S |
| A− | 8+ S | All 4 S | Pass | Both S |
| B+ | 7+ S | 3+ S | Pass | Both S |
| B | 6+ S | 3+ S | Pass | 1+ S |
| B− | 5+ S | 2+ S | Pass | 1+ S |
| C+ | 4+ S | 2+ S | Pass | 0+ S |
| C | 3+ S | 1+ S | Pass | 0+ S |
| C− | 2+ S | 1+ S | Pass | 0+ S |
| D+ | 1+ S | 0+ S | Pass | 0+ S |
| D | 1+ S | 0+ S | Pass | 0+ S |
| F | 0 S | 0 S | No pass | 0 S |

If M4 or the final examination is *No Pass,* the bundle grade drops by **2 letter grades** (e.g., A becomes B+). If both M4 and the final examination are No Pass, the grade drops by **4 letter grades** (e.g., A becomes B-). No revision is permitted for either deliverable.


"*A plumber doesn't get credit for effort; he gets credit if the faucet stops leaking*." – Seth Godin

# Tentative course plan

| Wk | Date | Day | Type | Topic / activity | Project / milestone |
|---|---|---|---|---|---|
| 1 | Mar 30 | Mon | Lecture | Introduction: What is computation? What is a computer? Course organization. AI/ML landscape overview. | Identify candidate algorithms; review application list. |
| 1 | Apr 1 | Wed | Codefest | CMAN (no AI): MAC/parameter/activation memory count by hand for a 3-layer MLP. CLLM (AI OK): Environment setup (GitHub, Slack). Profile ResNet-18 with torchinfo. Draft Heilmeier answers. Select project topic. | Draft Heilmeier answers; identify algorithm; run initial profiler; confirm SW baseline. Topic selected. |
| 2 | Apr 6 | Mon | Lecture | HW/SW co-design fundamentals: partitioning, profiling bottlenecks, interface design, tradeoff analysis. Roofline model: arithmetic intensity, compute-bound vs. memory-bound regimes, ridge point. | Revisit HW/SW partition with roofline framing. |
| 2 | Apr 8 | Wed | Codefest | CMAN (no AI): Roofline construction by hand; classify GEMM and vector-add kernels. CLLM (AI OK): CNN inference profiling; plot kernels on roofline; HW/SW partition proposal. | **M1 DUE Sun, Apr 12**: Heilmeier answers, profiling, roofline, HW/SW partition, interface selection, SW benchmark. |
| 3 | Apr 13 | Mon | Lecture | GPU architecture: SIMT execution, warp scheduling, memory hierarchy, CUDA programming model. Memory wall: DRAM vs. SRAM, bandwidth constraints, energy cost of data movement. | Identify which operations in your algorithm map to SIMT or tensor-core patterns. |
| 3 | Apr 16 | Wed | Codefest | CMAN (no AI): DRAM traffic analysis for naive vs. tiled 64×64 matrix multiply. CLLM (AI OK): CUDA GEMM naive vs. tiled; Nsight profiling; roofline placement. | Draft HW architecture due: block diagram, data path sketch, first HDL scaffold. |

…see syllabus for the rest

# What you will need in this course

- **A "pro" subscription to ChatGPT, Gemini, or Claude.**
- My recommendation: **Claude**
- Will it be worth it? 3x$20 = $60 (Pro). 3x$100 = $300 (Max)
- Ability to bring and work on a **laptop for codefests (Wed).**

> Regarding the question of where this leaves human grad students, my advice to students at all levels (and in any field) is to take LLMs seriously. Do not fall into the hallucination trap: "I asked the LLM X and it made something up, so I'm just going to wait for it to improve." Instead, get to know these models. Learn what they are good at and what they fail at. Buy the $20 subscription. It will change your life.

https://www.anthropic.com/research/vibe-physics

# How to get help?

- **Post question on Slack**
- **E-mail:** teuscher@pdx.edu
- **Office hours:** https://www.teuscher-lab.com/officehour

- How do you like to be called?
  - Christof
  - Dr. Teuscher
  - Dr. T
  - Professor Teuscher
  - ~~Hey Teuscher!~~

Christof Teuscher • teuscher@pdx.edu



# Live the “campsite rule”

## Your presence should make this classroom a better place.

- **Engage**: listen, participate, contribute, do your share. Play fair.
- **Focus**: no side conversations, no laptop and phone addiction, no work for other classes.
- **Be present**: show up, be on time, act like a professional.
- **Challenge yourself**: do always a little more instead of just the bare minimum. Up your game, set high standards for yourself. Produce work that you can be proud of.
- **Help others** to be successful. Share and care.
- **Willingness to learn**: have a growth mindset, intellectual humility and ambition.
- **Be in charge**: Put yourself in the driver’s seat, assume responsibility for your actions.


Leaving the world a better place

Add languages

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

**Leaving the world a better place**, often called the **campground rule**, or just **leaving things better than you found them**, is an ethical proposition that individuals should go beyond trying not to do harm in the world, and should try to remediate harms done by others.

teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

# Why show up?

- "*Showing up is 80 percent of life*." — Woody Allen
- It's the key to success
- The easiest way to stay on top of things.
- The easiest way to get help.
- The easiest way to learn from others.
- The easiest way to calibrate your work.
- Nothing can replace the connections you build in person.
- Please be on time. It's a matter of professionalism.

# Acknowledging AI tools

- In this course, the use of AI tools does not need to be acknowledged. It is understood that you will be using them.
- However, proper credit must be given for any other source your use.
- You must be able to explain any code in your repository. The final examination is designed to verify this. LLM-generated code that you do not understand and cannot debug is a liability, not an asset.

# Prep for Wednesday Codefest

1. Bring your laptop
2. Register for a “pro” ChatGPT, Gemini, or Claude subscription.
3. Set up your IDE and vibe coding workflow.
4. Accept the Slack invite.
5. Create a Github, post the link on Canvas (see spreadsheet)
6. Complete the pre-course assessment survey.

Christof Teuscher

ECE 410/510: Hardware for AI and ML, Spring 2026

# Hardware for AI and ML: an Overview

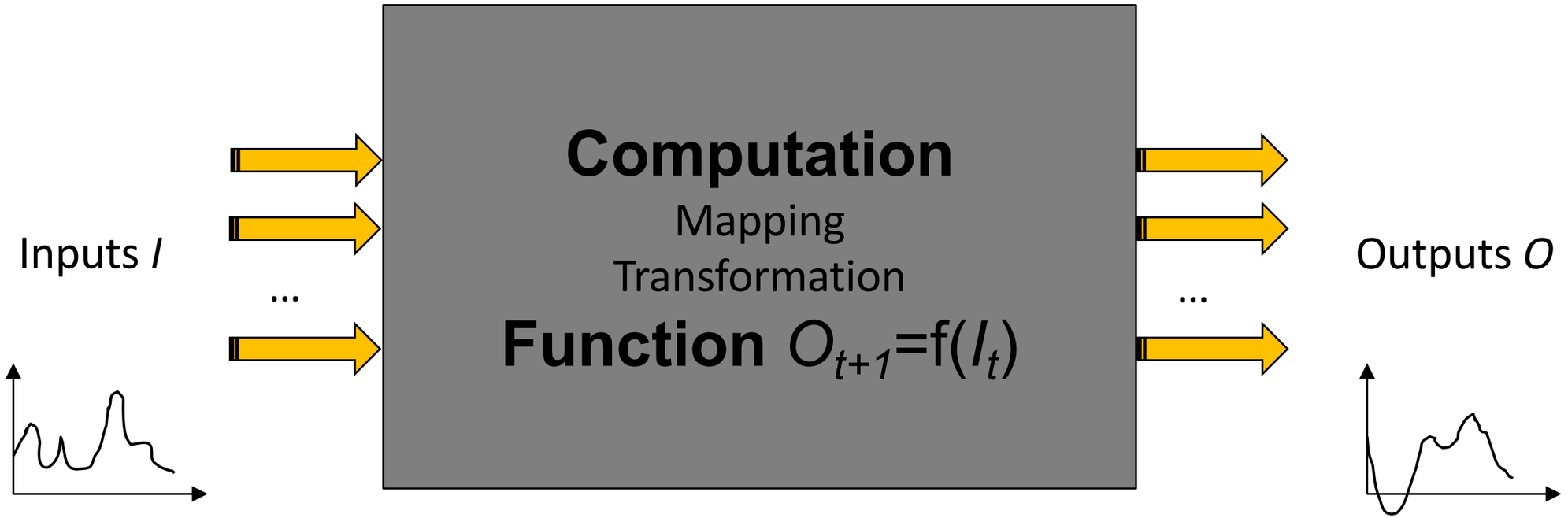
Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu


teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

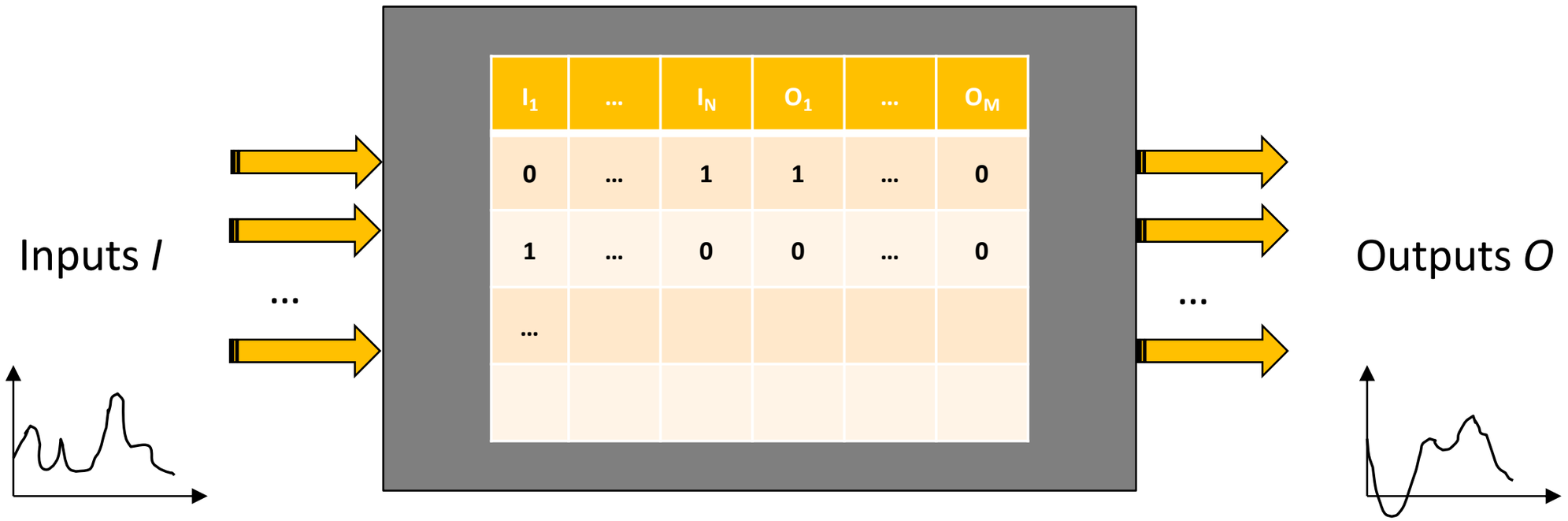
# What is a computer?

# What is computation? (1)




How can we "realize" this mapping?

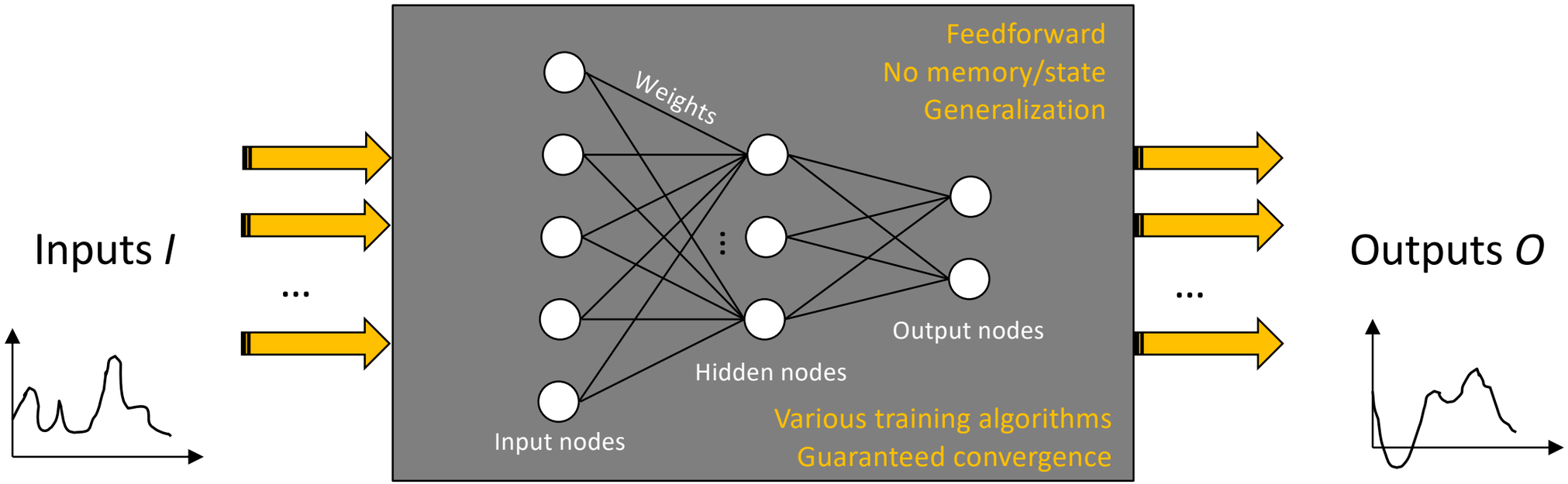
# What is computation? (2)




Look-up table

Simple, fast

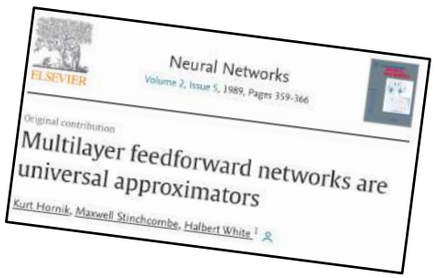
# What is computation? (3)



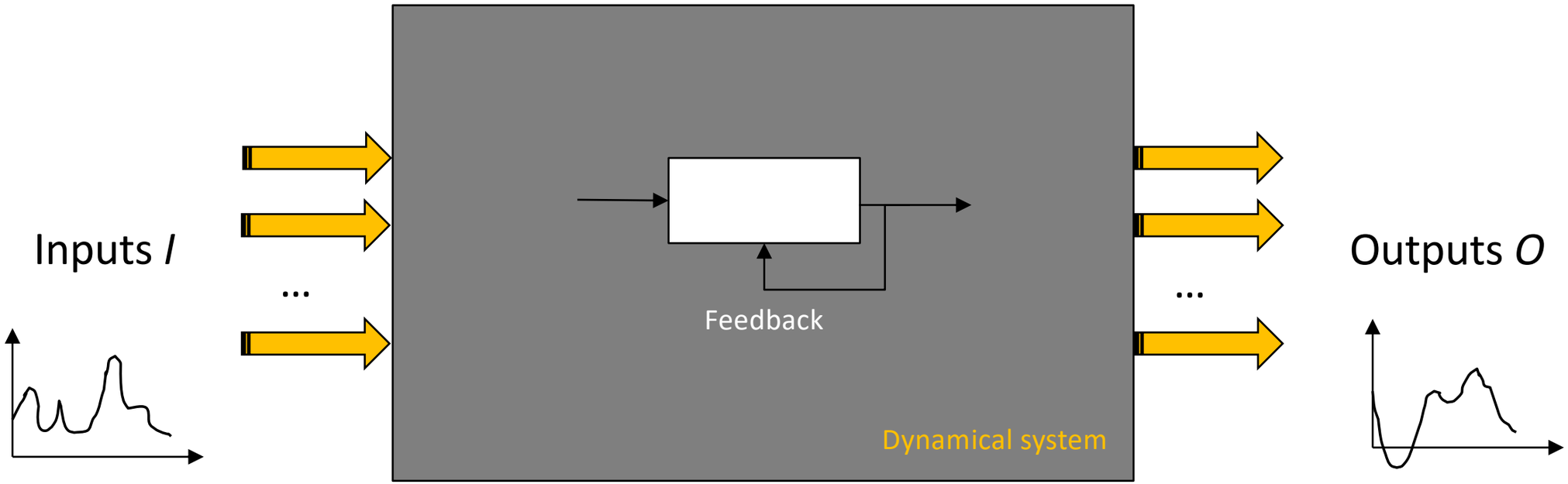

## Neural network

Universal function approximator

(Hornik, Stinchcombe, White, 1989, https://doi.org/10.1016/0893-6080(89)90020-8)



# What is computation? (4)



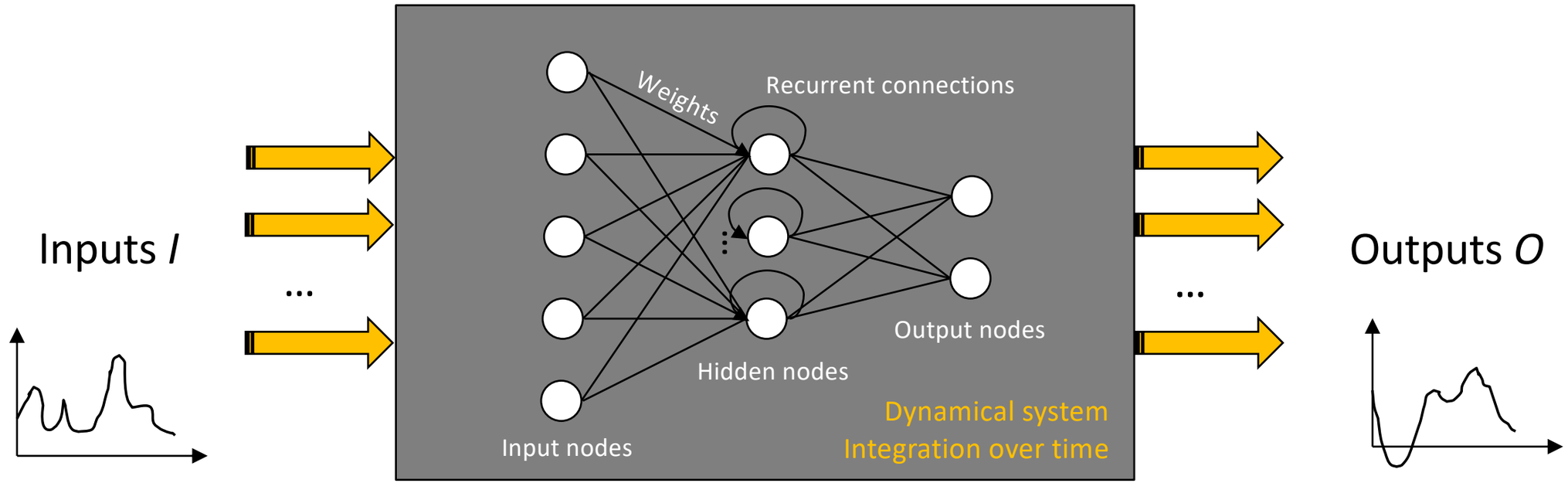

Memory is needed in a lot of cases

Temporal signal processing (audio, video)

Timeseries prediction

Control tasks

# What is computation? (5)



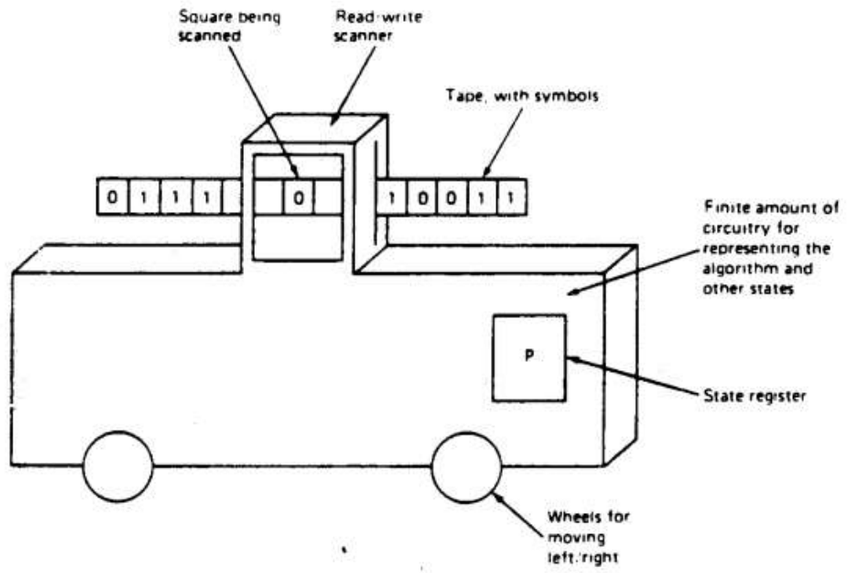

## Recurrent neural networks (RNN)

Training is challenging, various algorithms exist (backpropagation through time), convergence not guaranteed, many weights, slow learning. RNNs are also universal function approximators

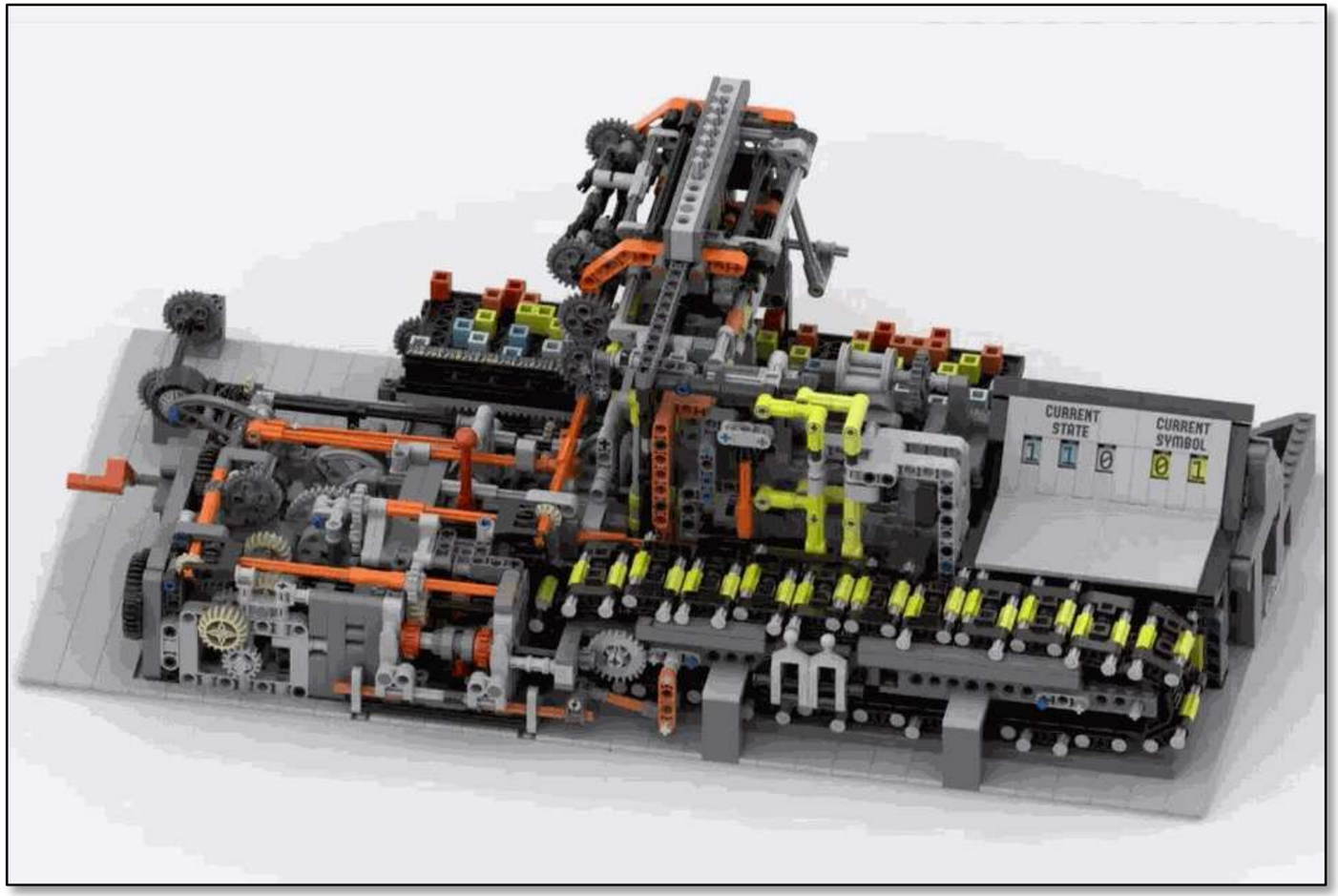
(Schaefer & Zimmerman, 2007, https://doi.org/10.1142/S0129065707001111)

# Turing Machine (TM)

**Figure B-2** An imaginary, physical Turing machine.




- Turing's aim was to define the simplest possible device that could perform any computation that could be performed by a human.
- TMs are a model of computability, they do not model the physical process of a computer.



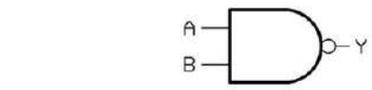

https://ideas.lego.com/projects/10a3239f-4562-4d23-ba8e-f4fc94eef5c7

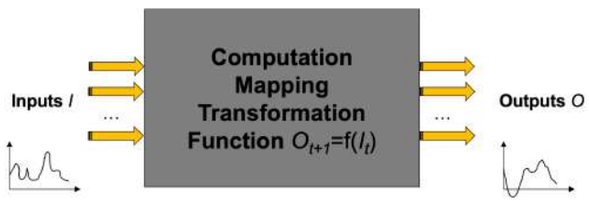
# Turing Machine (TM)

"What does a Turing machine look like? You can certainly imagine some crazy looking machine, but a better approach is to look in a mirror."
— Charles Petzold, The Annotated Turing

# Intrinsic vs designed computation (1)

- **Designed computation:**
  - top-down
  - we design devices and circuits for our purpose
  - information processing that is “useful”
- **Intrinsic computation:**
  - inherent dynamics of a dynamical system
  - not necessarily “useful”

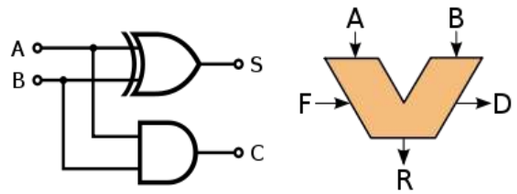



What does a falling apple ”compute?”

- Everything computes something, the question is what and how it can be harnessed for “useful” purposes.
- *“What natural substrates are complex enough to support controllable and useful information processing?”* (Crutchfield et al., 2010, https://doi.org/10.1063/1.3492712)

# Intrinsic vs designed computation (2)



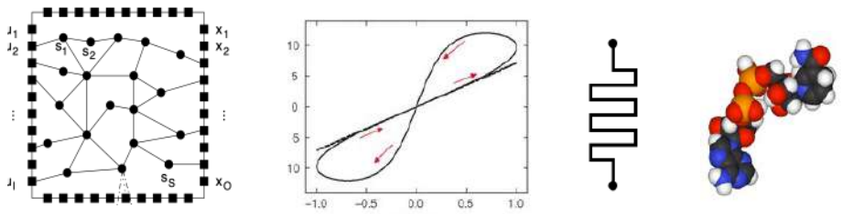

**Desired computation**

Computing architecture

**How to obtain a desired computation?**

**Intrinsic dynamics**

Device network



# Intrinsic vs designed computation (3)

"Evolvable hardware" variant

## Nanocell Logic Gates for Molecular Computing

James M. Tour, William L. Van Zandt, Christopher P. Husband, Summer M. Husband, Lauren S. Wilson, Paul D. Franzon, and David P. Nackashi

(Tour et al., 2002, http://doi.org/10.1109/TNANO.2002.804744)

- Network trained by switching NDR switches ON/OFF at post fabrication.
- Omnipotence is assumed (one knows and can control all switches)
- Tasks: logic functions, 1 bit-adder
- In simulation only

Self-assembled network of metallic particles, with reprogrammable negative differential resistance (NDR)

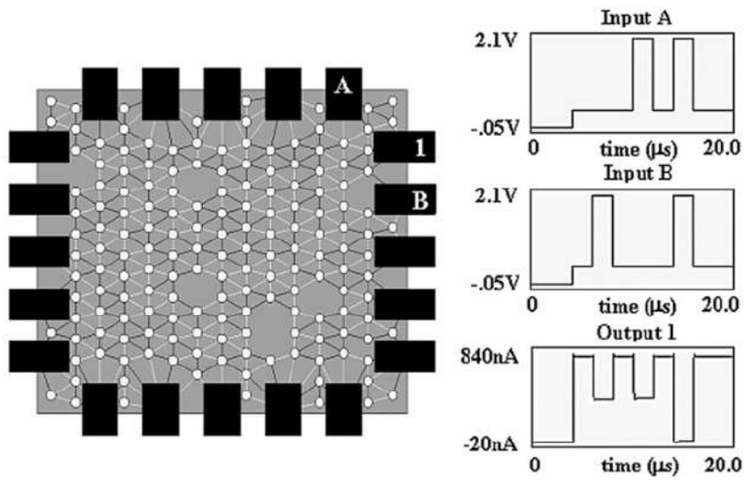
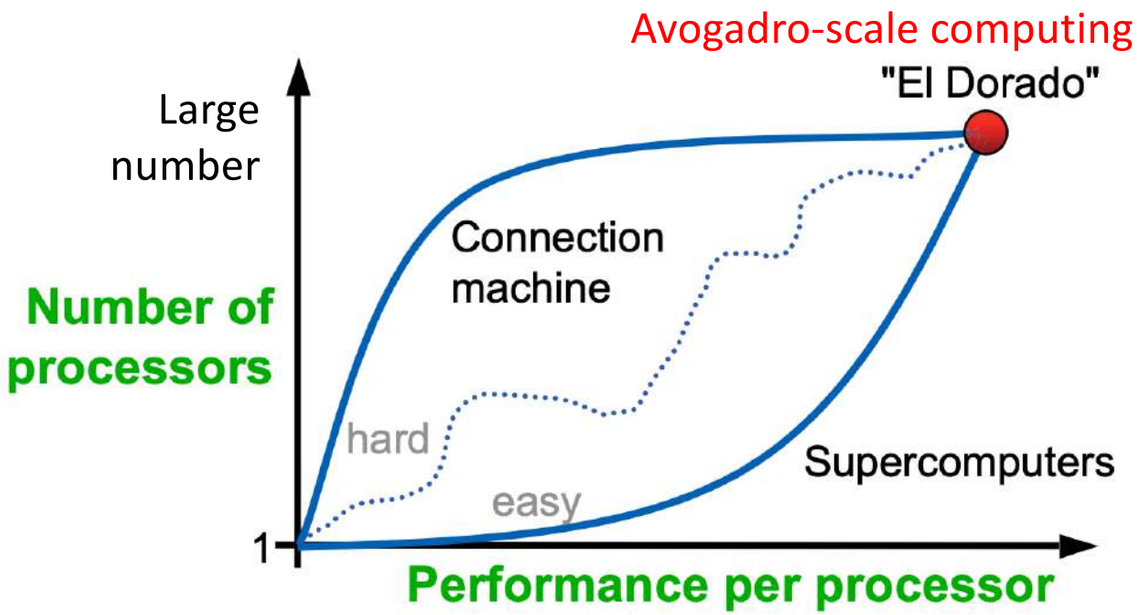


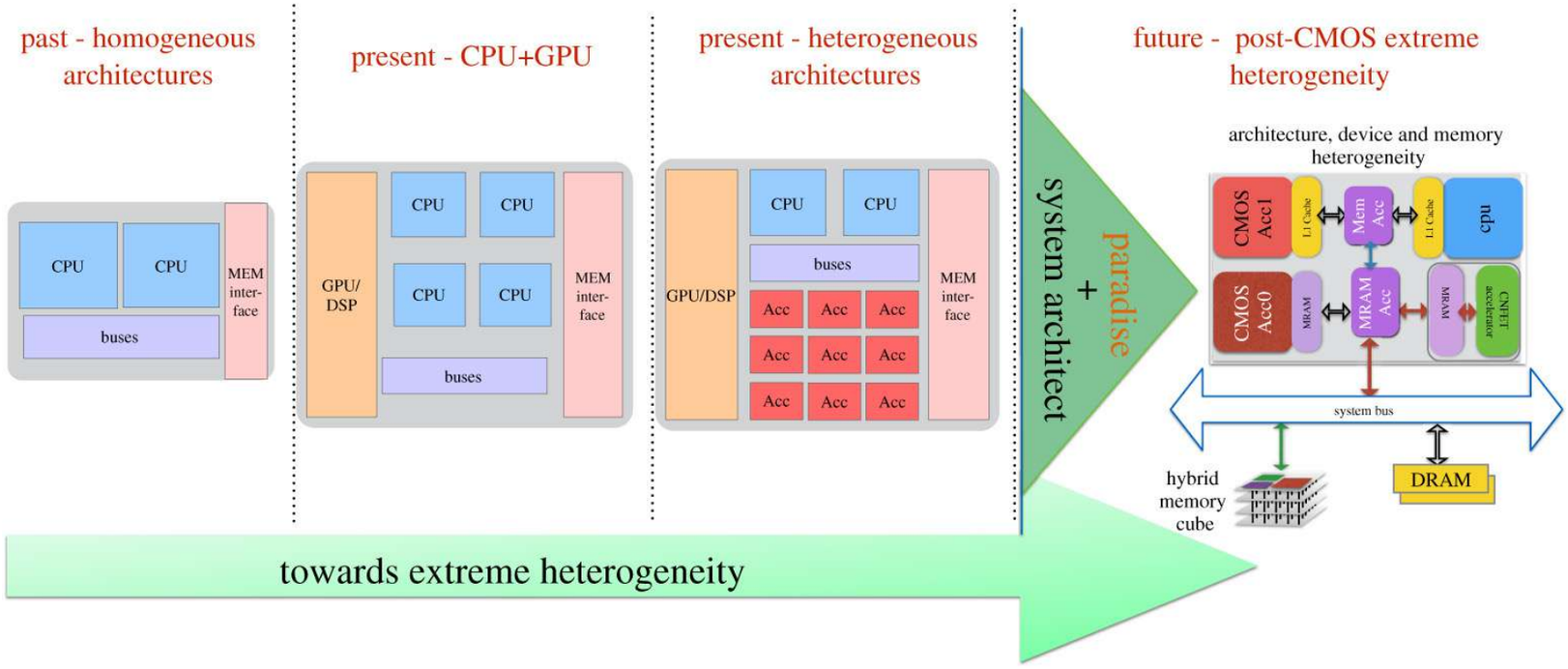

Fig. 5. NAND gate is demonstrated in this nanocell using the SPICE interface model and the $I(V)$ curve shown in Fig. 3. The plots show the $V(t)$ and $I(t)$ with the accompanying NAND Truth Table logic.

# What kind of computing machinery would an extraterrestrial build?

# The goals and drivers have changed



# The goals and drivers have changed



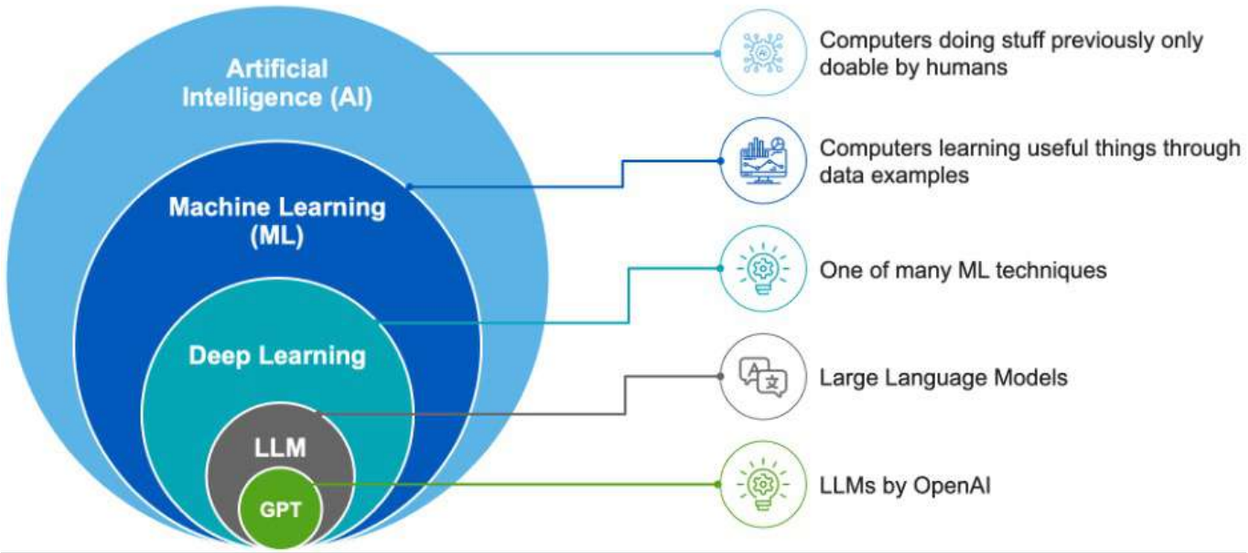

https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061

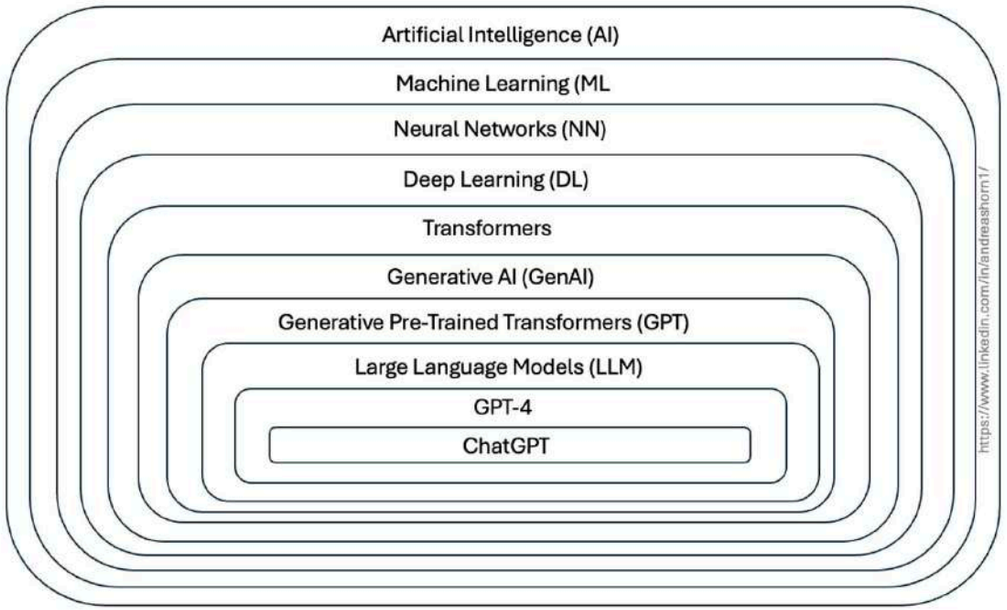
# AI vs ML

- **Machine Learning (ML)** is one technique used to achieve **Artificial Intelligence** (AI).
- All ML is AI, but not all AI is ML.
- AI is the destination (e.g., intelligent machines), while ML is one path to get there (learning from data).

- **Artificial Intelligence (AI):** Broader concept encompassing machines that can perform tasks requiring human intelligence. Ultimate goal: Artificial General Intelligence (AGI).
- **Machine Learning (ML):** Subset of AI focused specifically on algorithms that improve through experience. Example: neural networks.

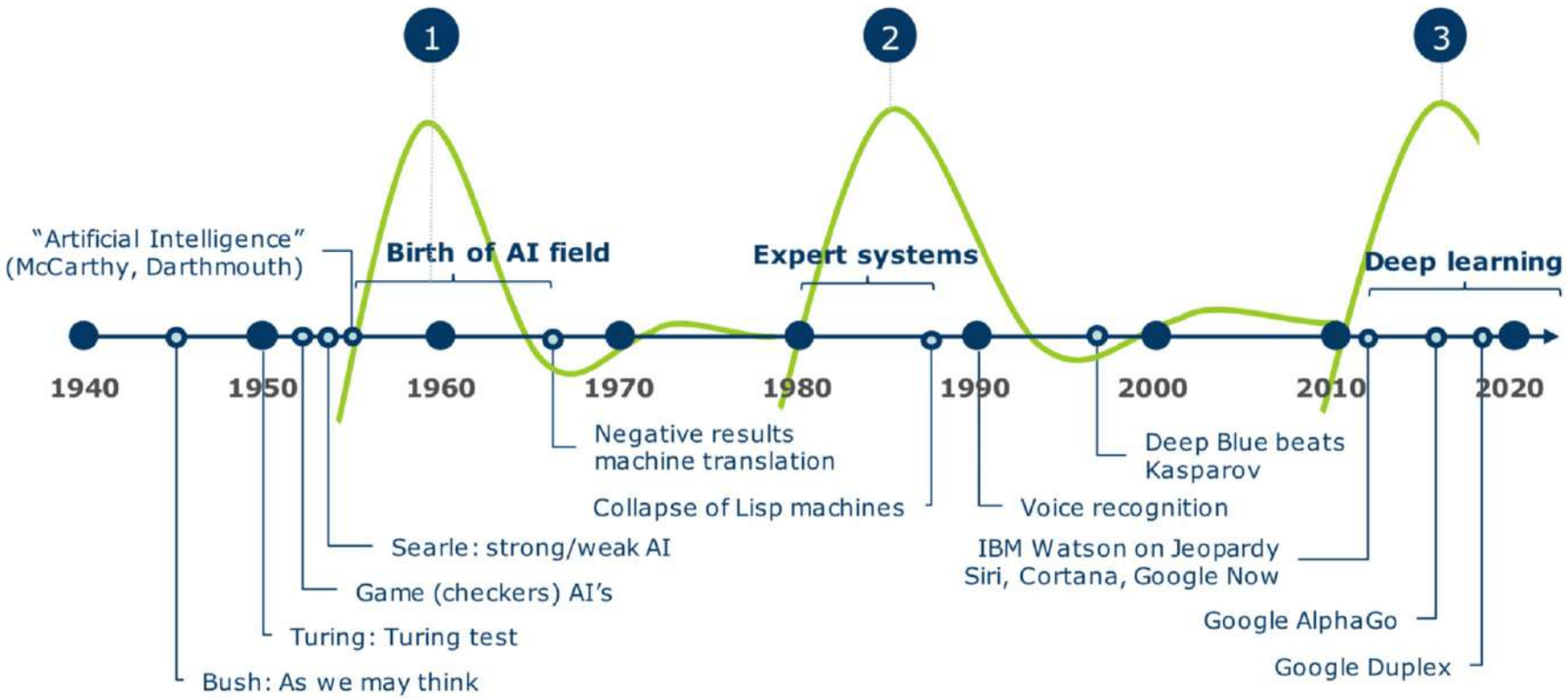
# AI vs ML



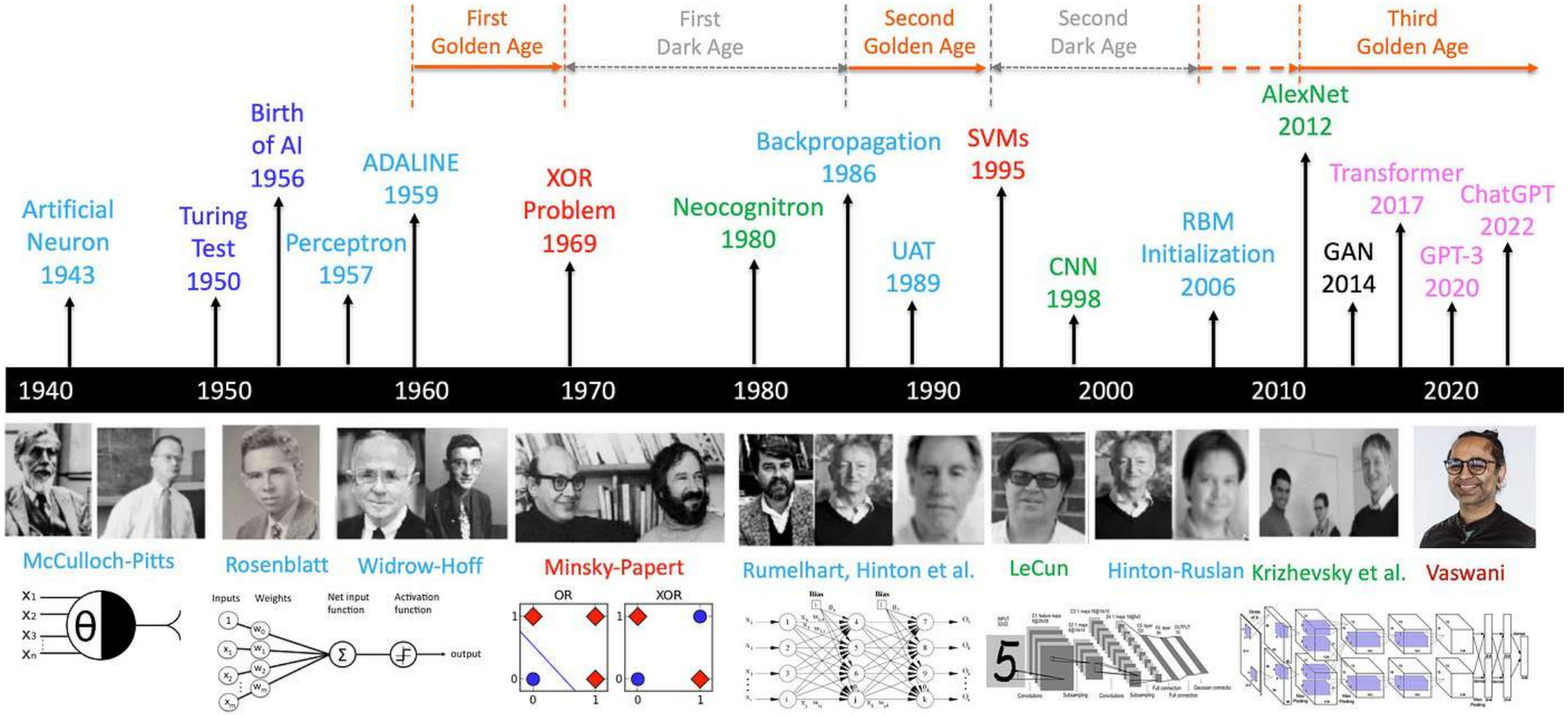

Source: Tignis

# A very brief history of AI



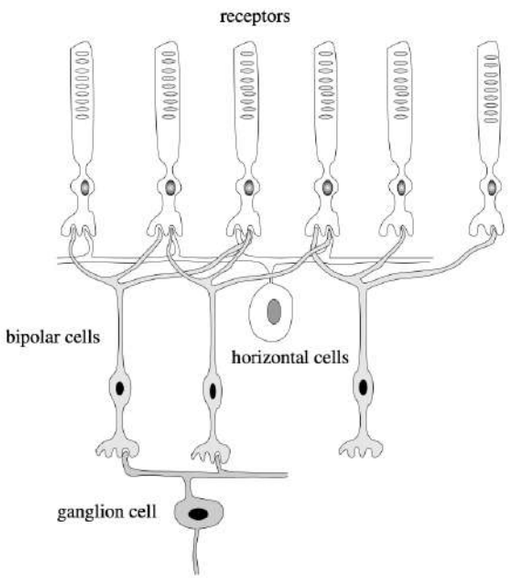

Strong AI = AGI = Artificial General Intelligence

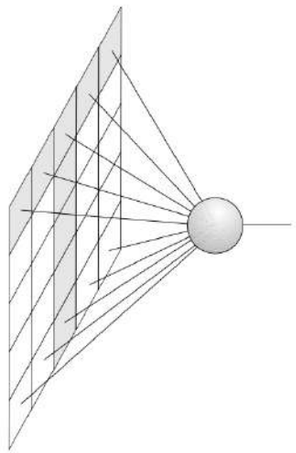
# A very brief history of AI with deep learning

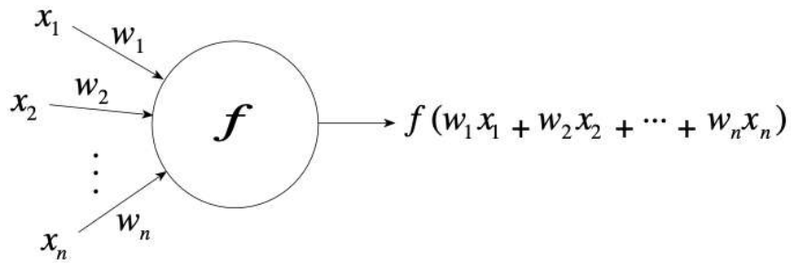


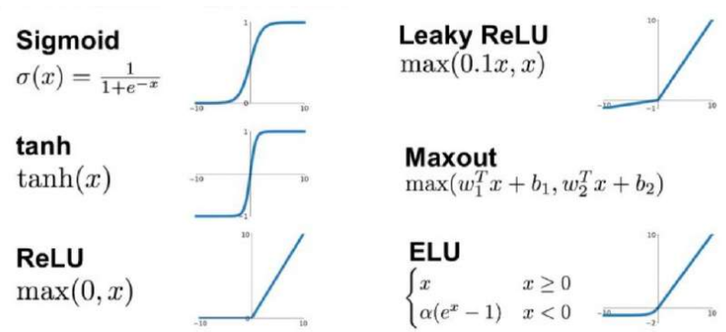

https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b

# An abstract neuron



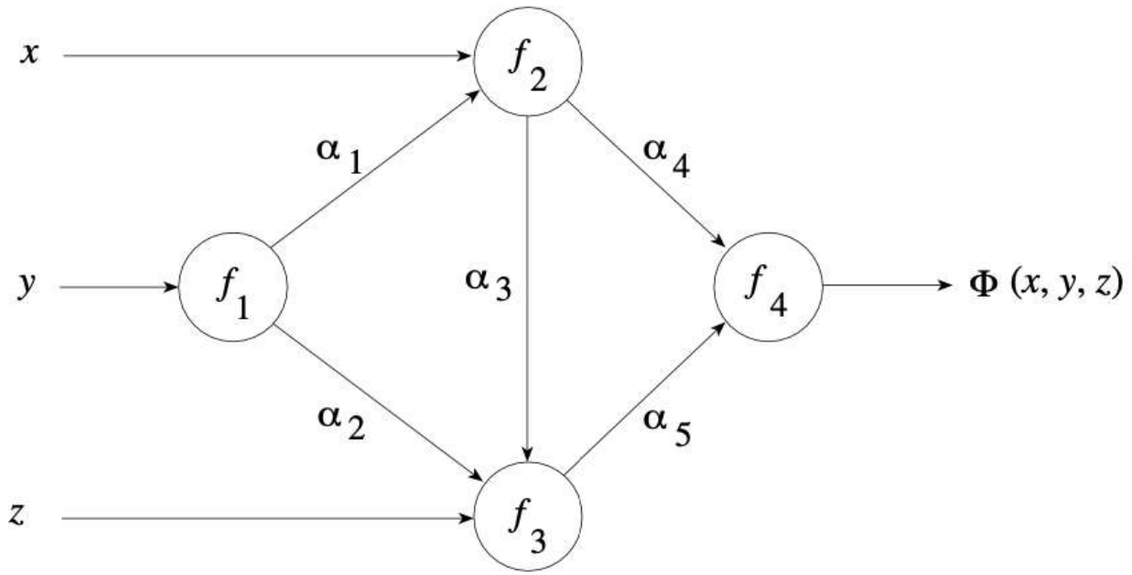




Rojas, 1986

# A neural network




Rojas, 1986

# Special types of neural networks

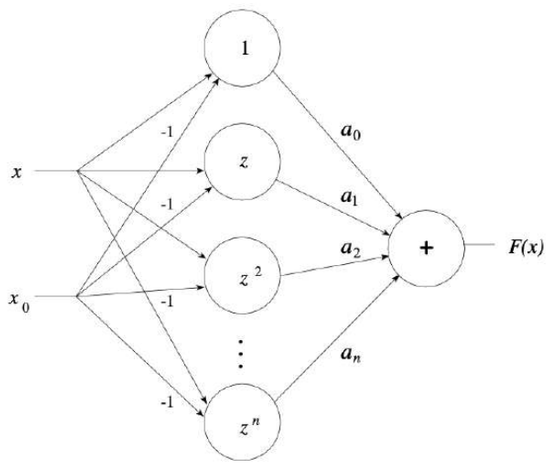



**Fig. 1.16.** A Taylor network

$$F(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + \cdots + a_n(x - x_0)^n + \cdots,$$

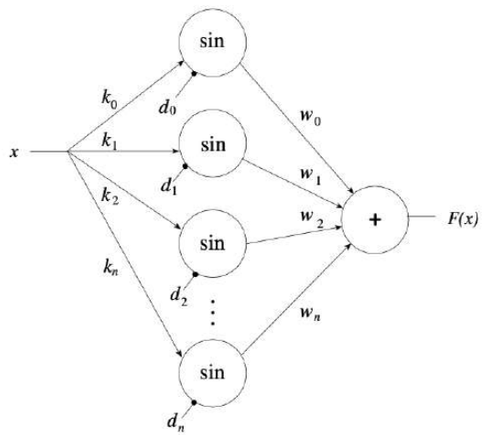


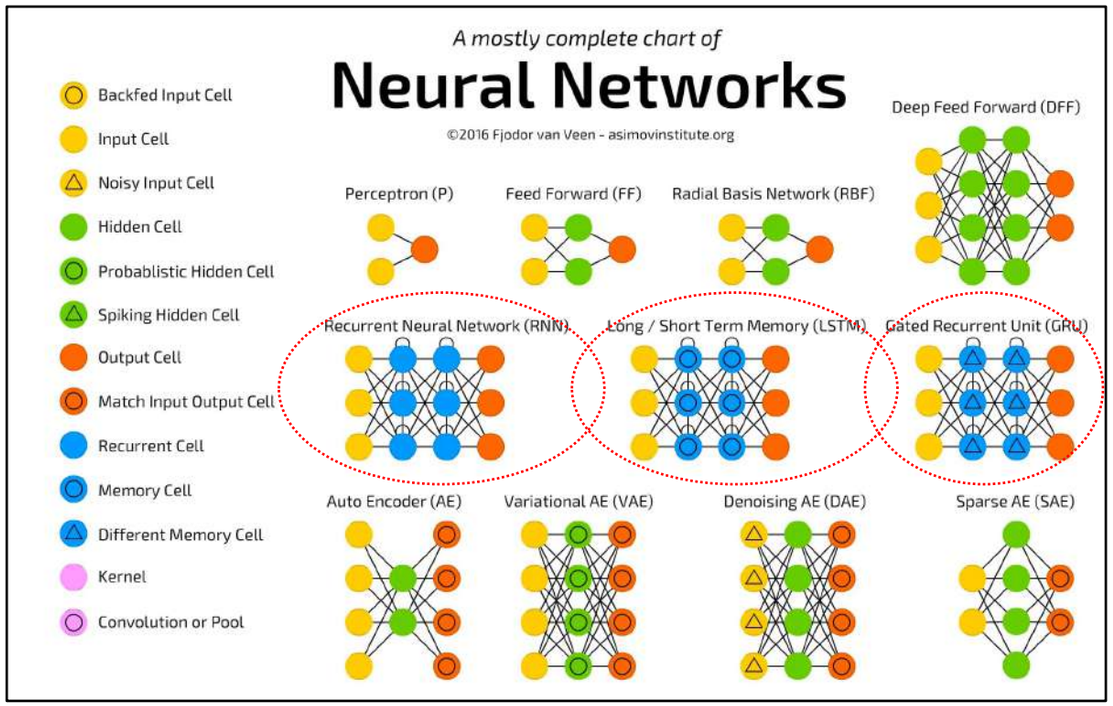

**Fig. 1.17.** A Fourier network

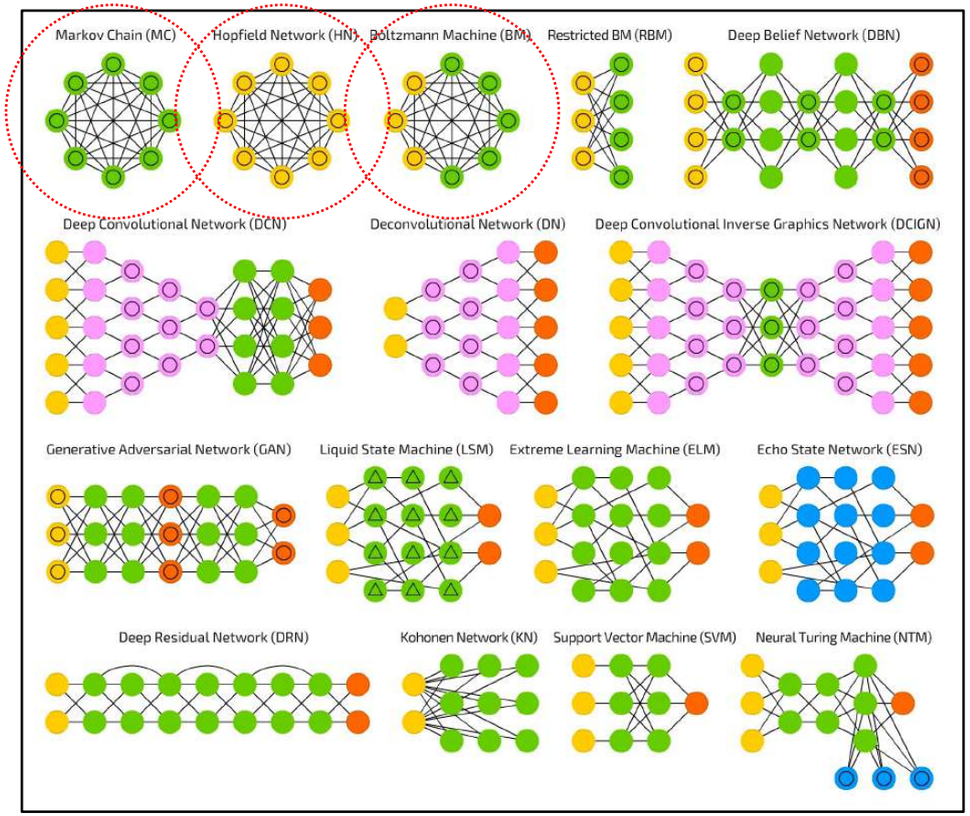
Figure 1.17 shows how a Fourier series can be implemented as a neural network. If the function $F$ is to be developed as a Fourier series it has the form

$$F(x) = \sum_{i=0}^{\infty} (a_i \cos(ix) + b_i \sin(ix)). \qquad (1.2)$$

Rojas, 1986

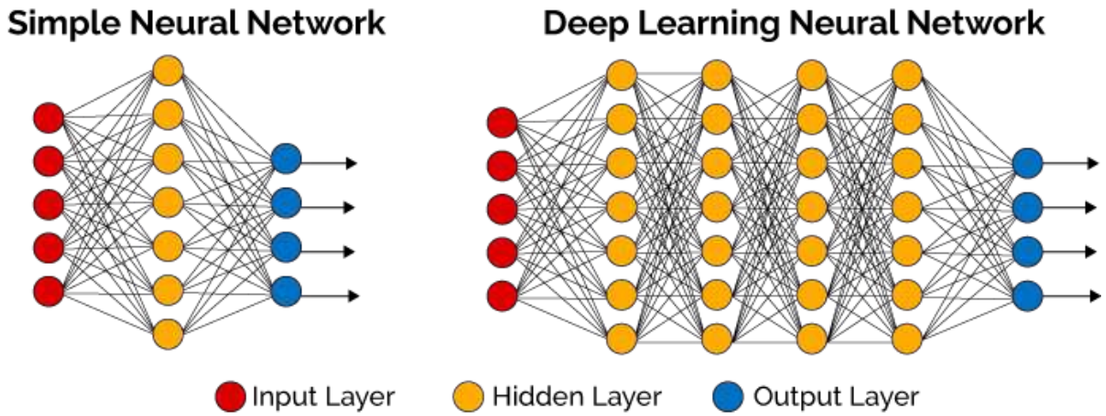
# Types of neural networks

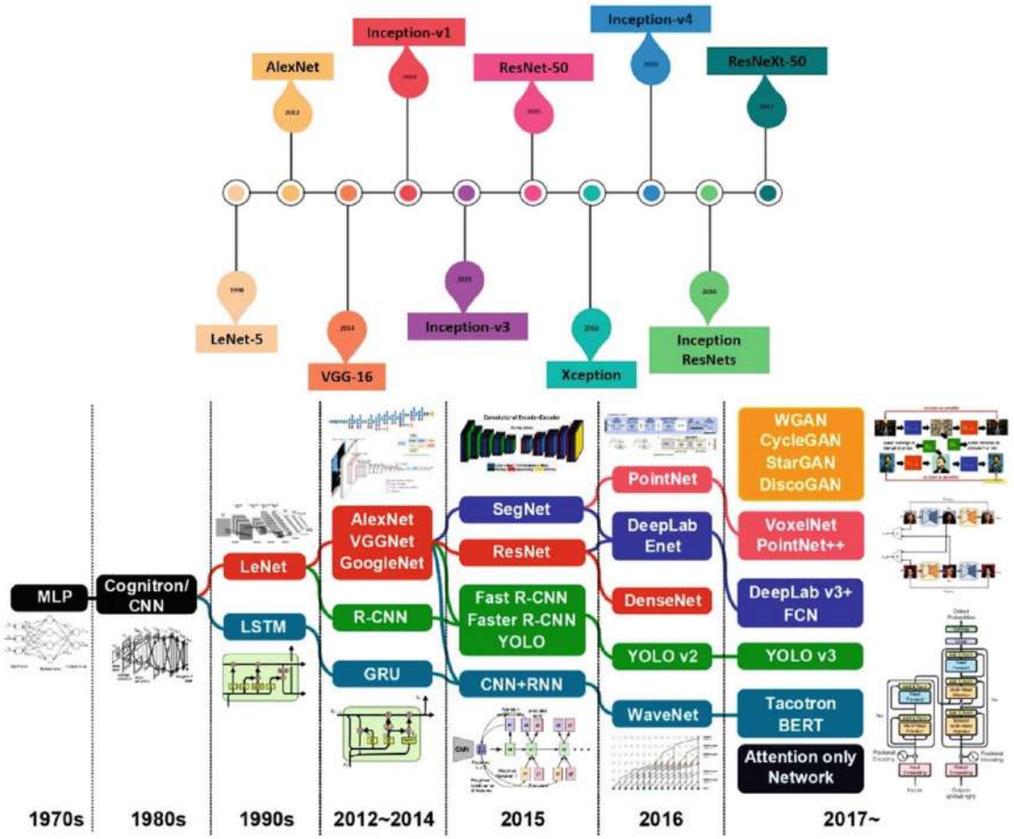


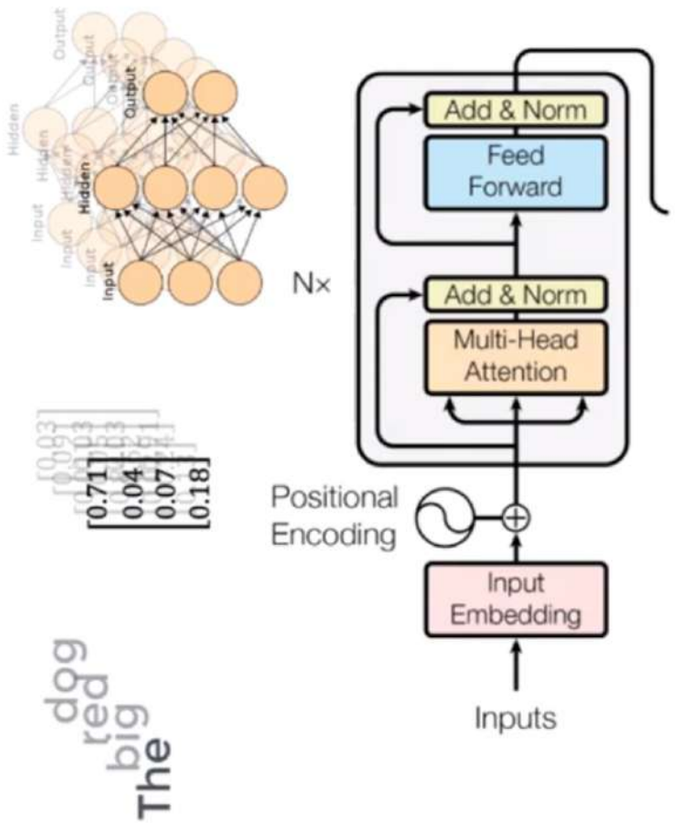

(Fjodor van Veen, 2016, asimovinstitute.org)

# Deep neural networks



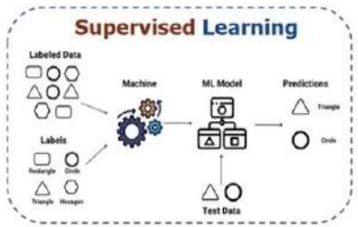



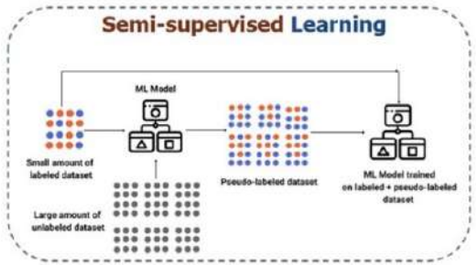
# Transformers are deep neural networks

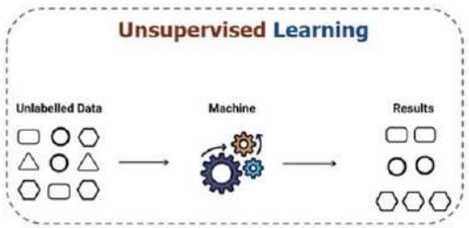


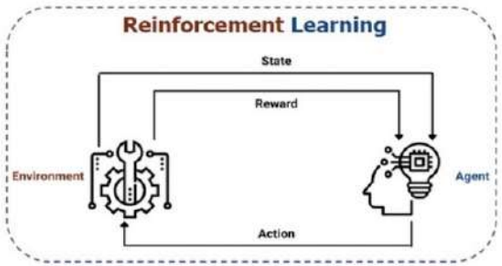

https://builtin.com/artificial-intelligence/transformer-neural-network

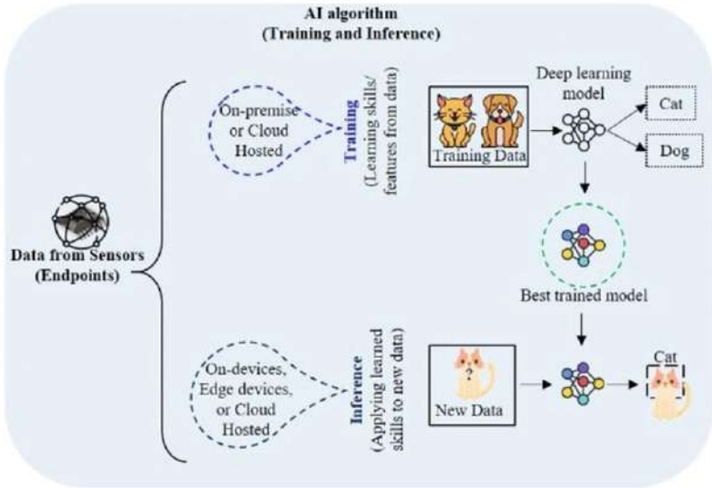
# Learning

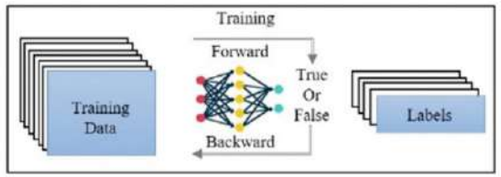


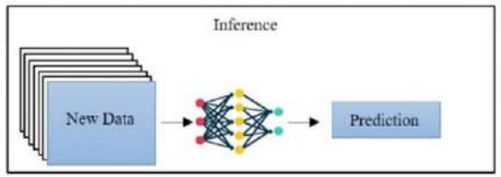



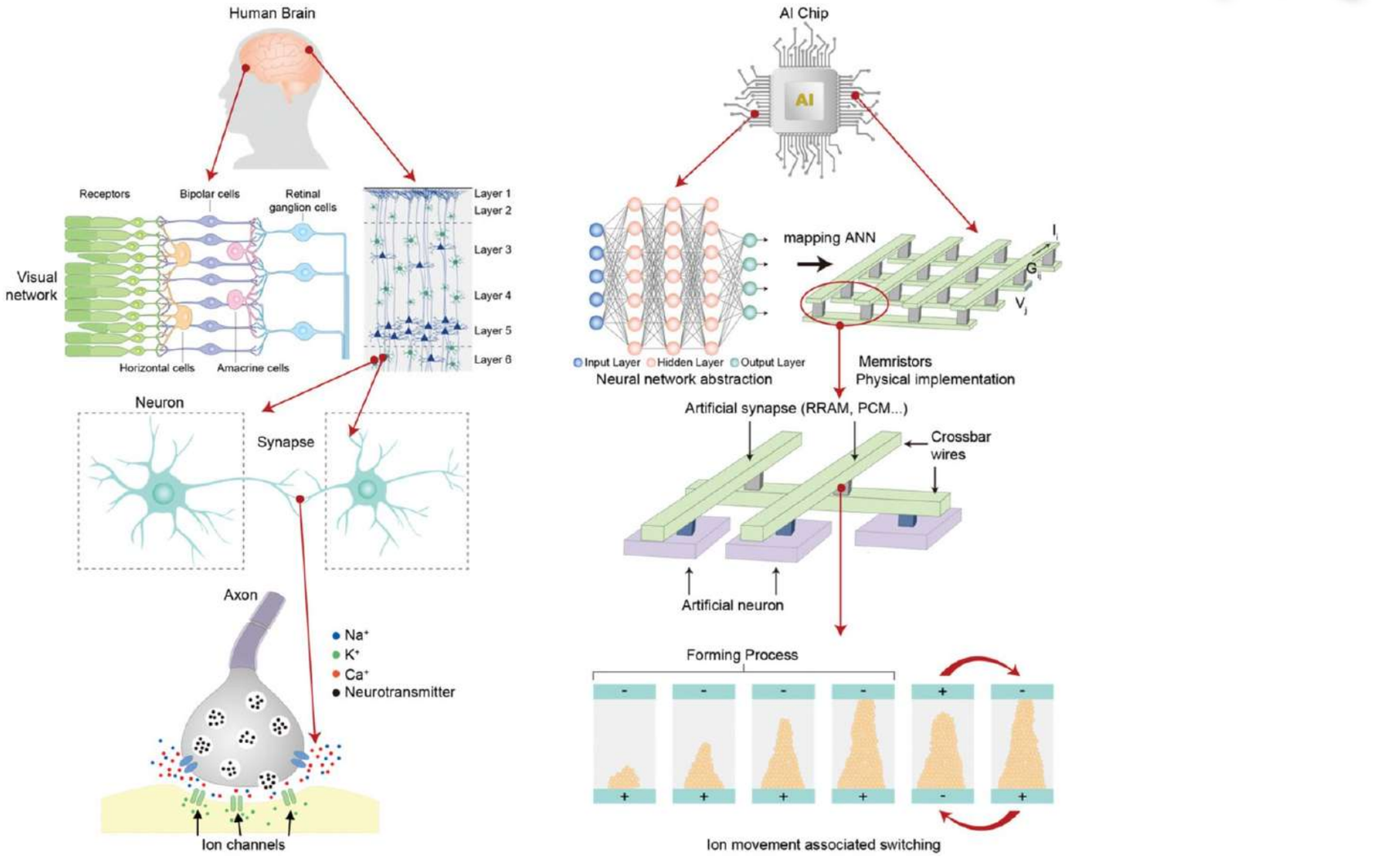





What is harder? Inference or training?

Mishra et al., 2023




Source: "Bridging Biological and Artificial Neural Networks with Emerging Neuromorphic Devices: Fundamentals, Progress, and Challenges"

# Neuromorphic Computing with Large Scale Spiking Neural Networks

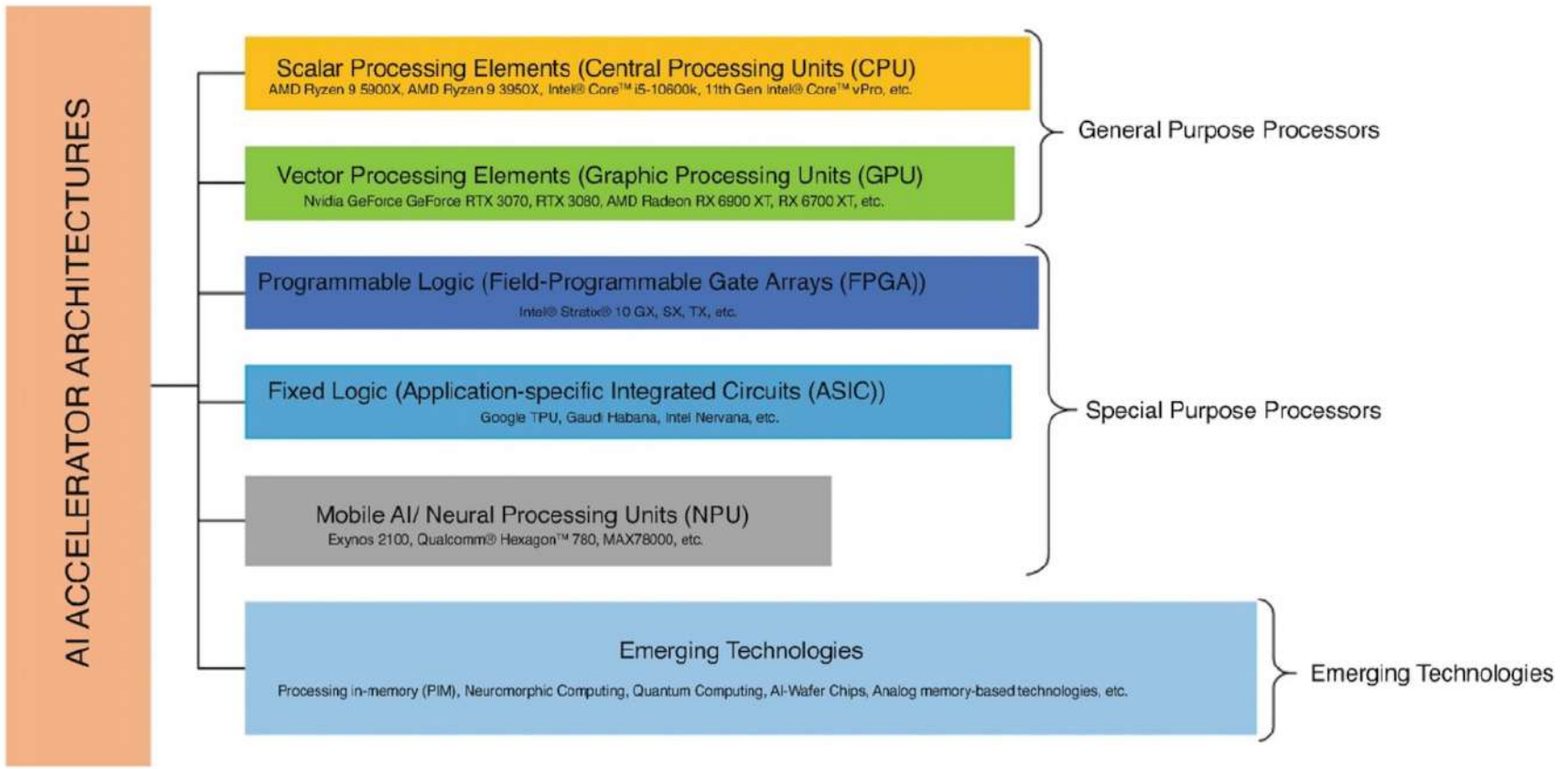
Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 March 2025 doi:10.20944/preprints202503.1505.v1



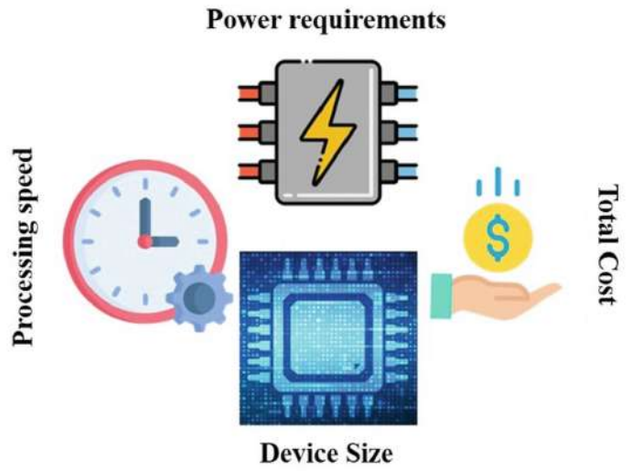
**Table 3.** Comparison of leading neuromorphic hardware platforms in terms of neuron capacity, power efficiency, and scalability.

| Hardware Platform | Technology | Neuron Capacity | Power Efficiency | Scalability |
|---|---|---|---|---|
| IBM TrueNorth | Digital | $10^6$ neurons | High | Moderate |
| Intel Loihi | Digital | $10^5$ neurons | Very High | High |
| SpiNNaker | Digital | $10^6$ neurons | Moderate | High |
| BrainScaleS | Analog | $10^4$ neurons | Low | Low |
| Tianjic | Hybrid | $10^5$ neurons | High | High |

# Hardware for AI/ML




**Metrics**




Mishra et al., 2023

# Hardware for AI/ML

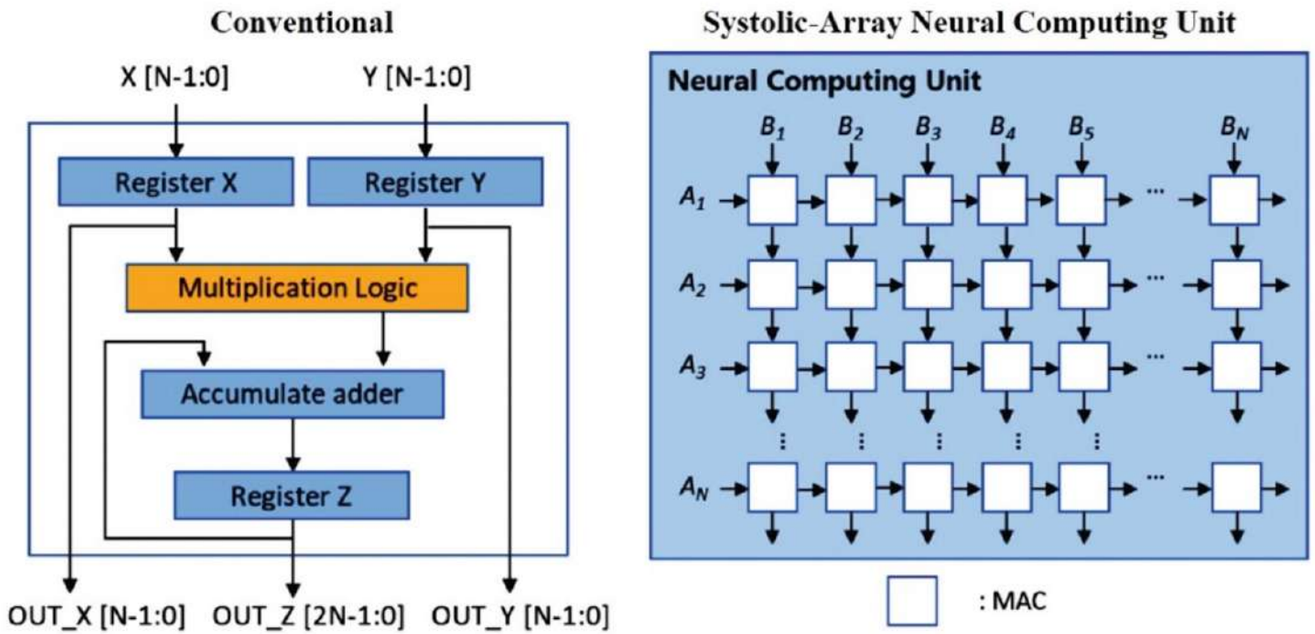
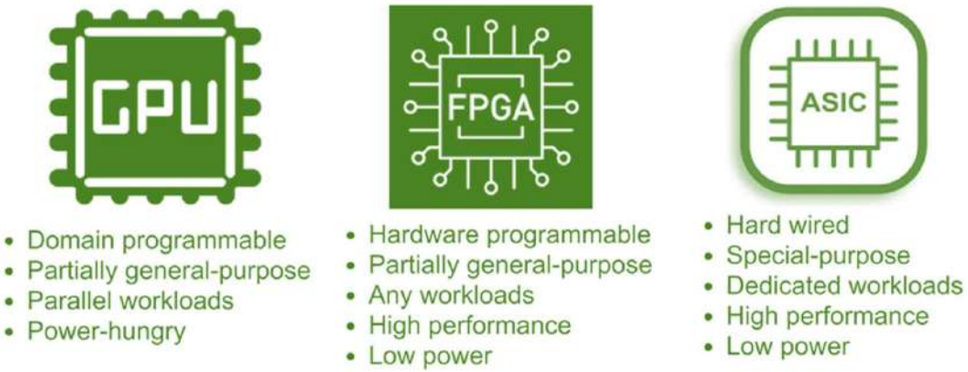


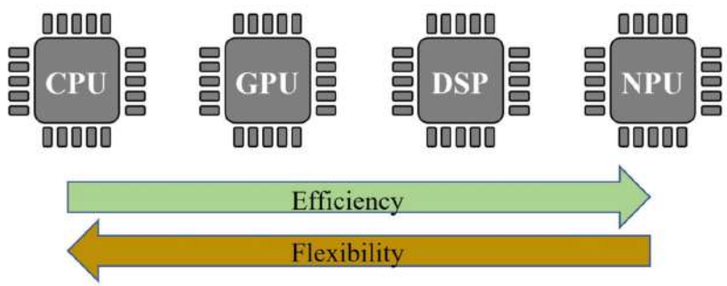

***Fig. 19*** A comparison of MAC operations performed on conventional and neural computing units.

Mishra et al., 2023

# Hardware for AI/ML

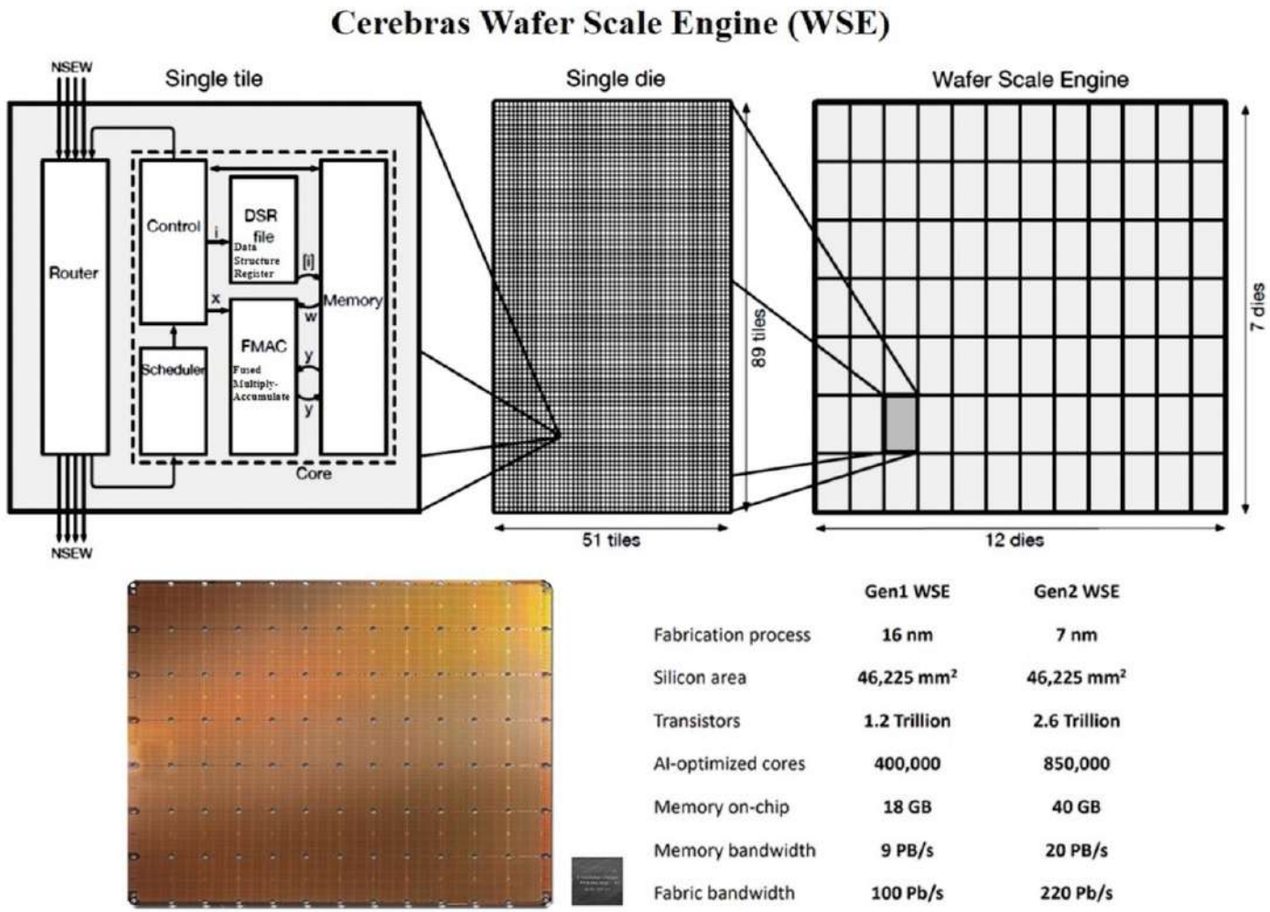
| Processor | Power Consumption | Strengths | Limitations |
|---|---|---|---|
| CPU | High | → Flexible<br>→ General-purpose processing<br>→ Complex instructions and tasks<br>→ System management | → Possible memory access bottlenecks<br>→ Few cores (4–16) |
| GPU | High | → Parallel cores (~1000 s of cores)<br>→ High-performance AI processing | → Power consumption<br>→ Large footprint |
| FPGA | Medium | → Configurable logic gates<br>→ Flexible<br>→ In-field re-programmability | → Programming complexity |
| ASIC | Low | → Custom logic designed with libraries<br>→ Faster processing<br>→ Small footprint | → Fixed function<br>→ Expensive custom design |
| Vision Processing Unit (VPU) | Ultra-low | → Dedicated image and vision co-processor<br>→ Small footprint | → Limited dataset and batch size<br>→ Limited network support |
| Tensor Processing Unit (TPU) | Low to medium | → Specialized tool support<br>→ Optimized for TensorFlow | → Proprietary design<br>→ Limited framework support |




Mishra et al., 2023

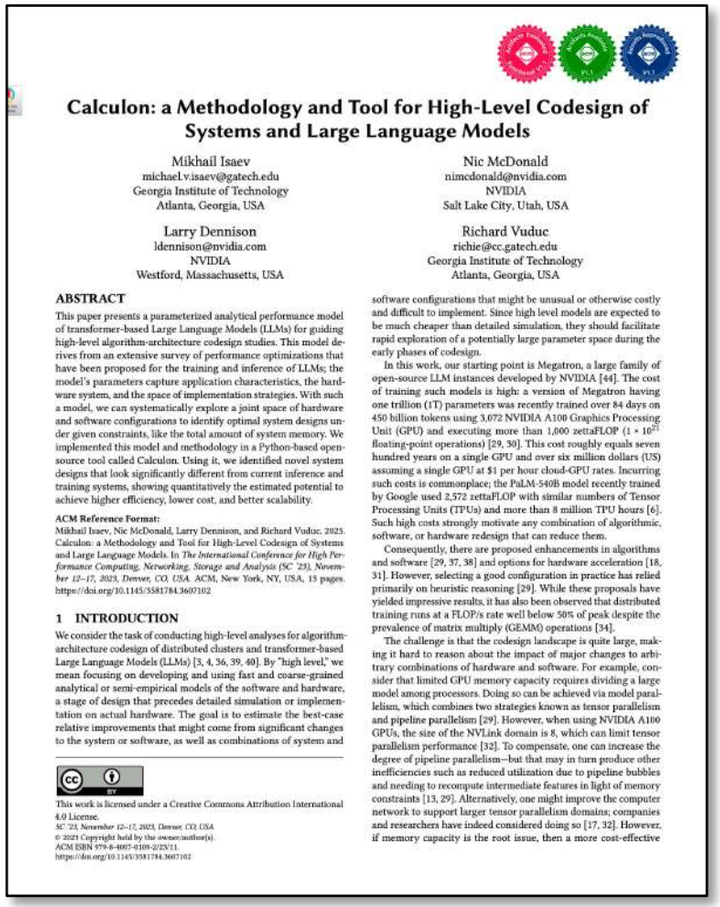
# Hardware for AI/ML




| | Gen1 WSE | Gen2 WSE |
|---|---|---|
| Fabrication process | 16 nm | 7 nm |
| Silicon area | 46,225 mm² | 46,225 mm² |
| Transistors | 1.2 Trillion | 2.6 Trillion |
| AI-optimized cores | 400,000 | 850,000 |
| Memory on-chip | 18 GB | 40 GB |
| Memory bandwidth | 9 PB/s | 20 PB/s |
| Fabric bandwidth | 100 Pb/s | 220 Pb/s |

Mishra et al., 2023

# Co-design & LLM



## Calculon: a Methodology and Tool for High-Level Codesign of Systems and Large Language Models

Mikhail Isaev
michael.v.isaev@gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

Nic McDonald
nimcdonald@nvidia.com
NVIDIA
Salt Lake City, Utah, USA

Larry Dennison
ldennison@nvidia.com
NVIDIA
Westford, Massachusetts, USA

Richard Vuduc
richie@cc.gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

**ABSTRACT**

This paper presents a parameterized analytical performance model of transformer-based Large Language Models (LLMs) for guiding high-level algorithm-architecture codesign studies. This model derives from an extensive survey of performance optimizations that have been proposed for the training and inference of LLMs; the model's parameters capture application characteristics, the hardware system, and the space of implementation strategies. With such a model, we can systematically explore a joint space of hardware and software configurations to identify optimal system designs under given constraints, like the total amount of system memory. We implemented this model and methodology in a Python-based open-source tool called Calculon. Using it, we identified novel system designs that look significantly different from current inference and training systems, showing quantitatively the estimated potential to achieve higher efficiency, lower cost, and better scalability.

**ACM Reference Format:**
Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard Vuduc. 2023. Calculon: a Methodology and Tool for High-Level Codesign of Systems and Large Language Models. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23), November 12–17, 2023, Denver, CO, USA.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3581784.3607102

### 1 INTRODUCTION

We consider the task of conducting high-level analyses for algorithm-architecture codesign of distributed clusters and transformer-based Large Language Models (LLMs) [3, 4, 36, 39, 40]. By "high level," we mean focusing on developing and using fast and coarse-grained analytical or semi-empirical models of the software and hardware, a stage of design that precedes detailed simulation or implementation on actual hardware. The goal is to estimate the best-case relative improvements that might come from significant changes to the system or software, as well as combinations of system and software configurations that might be unusual or otherwise costly and difficult to implement. Since high level models are expected to be much cheaper than detailed simulation, they should facilitate rapid exploration of a potentially large parameter space during the early phases of codesign.

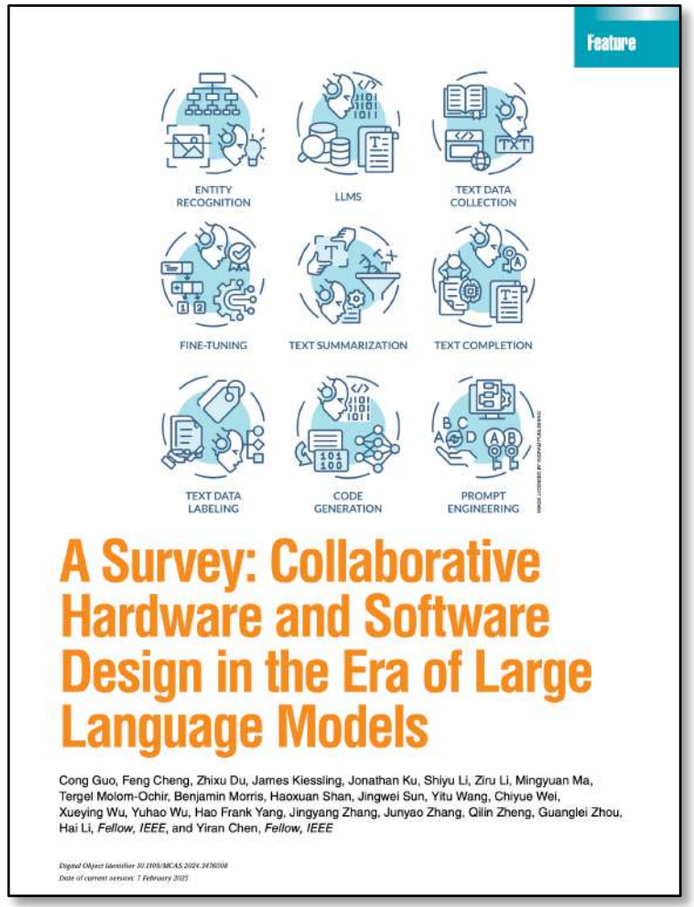
In this work, our starting point is Megatron, a large family of open-source LLM instances developed by NVIDIA [44]. The cost of training such models is high: a version of Megatron having one trillion (1T) parameters was recently trained over 84 days on 450 billion tokens using 3,072 NVIDIA A100 Graphics Processing Unit (GPU) and executing more than 1,000 zettaFLOP ($1 \times 10^{21}$ floating-point operations) [29, 30]. This cost roughly equals seven hundred years on a single GPU and over six million dollars (US) assuming a single GPU at $1 per hour cloud-GPU rates. Incurring such costs is commonplace; the PaLM-540B model recently trained by Google used 2,572 zettaFLOP with similar numbers of Tensor Processing Units (TPUs) and more than 8 million TPU hours [6]. Such high costs strongly motivate any combination of algorithmic, software, or hardware redesign that can reduce them.

Consequently, there are proposed enhancements in algorithms and software [29, 37, 38] and options for hardware acceleration [18, 31]. However, selecting a good configuration in practice has relied primarily on heuristic reasoning [29]. While these proposals have yielded impressive results, it has also been observed that distributed training runs at a FLOP/s rate well below 50% of peak despite the prevalence of matrix multiply (GEMM) operations [34].

The challenge is that the codesign landscape is quite large, making it hard to reason about the impact of major changes to arbitrary combinations of hardware and software. For example, consider that limited GPU memory capacity requires dividing a large model among processors. Doing so can be achieved via model parallelism, which combines two strategies known as tensor parallelism and pipeline parallelism [29]. However, when using NVIDIA A100 GPUs, the size of the NVLink domain is 8, which can limit tensor parallelism performance [32]. To compensate, one can increase the degree of pipeline parallelism—but that may in turn produce other inefficiencies such as reduced utilization due to pipeline bubbles and needing to recompute intermediate features in light of memory constraints [13, 29]. Alternatively, one might improve the computer network to support larger tensor parallelism domains; companies and researchers have indeed considered doing so [17, 32]. However, if memory capacity is the root issue, then a more cost-effective


*SC '23, November 12–17, 2023, Denver, CO, USA*

ACM ISBN 979-8-4007-0109-2/23/11.
https://doi.org/10.1145/3581784.3607102




## A Survey: Collaborative Hardware and Software Design in the Era of Large Language Models

Cong Guo, Feng Cheng, Zhixu Du, James Kiessling, Jonathan Ku, Shiyu Li, Ziru Li, Mingyuan Ma, Tergel Molom-Ochir, Benjamin Morris, Haoxuan Shan, Jingwei Sun, Yitu Wang, Chiyue Wei, Xueying Wu, Yuhao Wu, Hao Frank Yang, Jingyang Zhang, Junyao Zhang, Qilin Zheng, Guanglei Zhou, Hai Li, *Fellow, IEEE*, and Yiran Chen, *Fellow, IEEE*

*Digital Object Identifier 10.1109/MCAS.2024.3476358*
*Date of current version: 7 February 2025*

# (HW/SW) Co-design

- **Classical approach**: in college, different technologies are taught in different classes.
- **Hardware-software co-design:**
  - Started in the 1990s.
  - Its core idea is the concurrent designs of hardware and software components of complex electronic systems.
- **More broadly**:
  - The concurrent design of hardware and software across all layers of the compute stack.

- **What are possible metrics and trade-offs?**

# Deep co-design across the entire stack

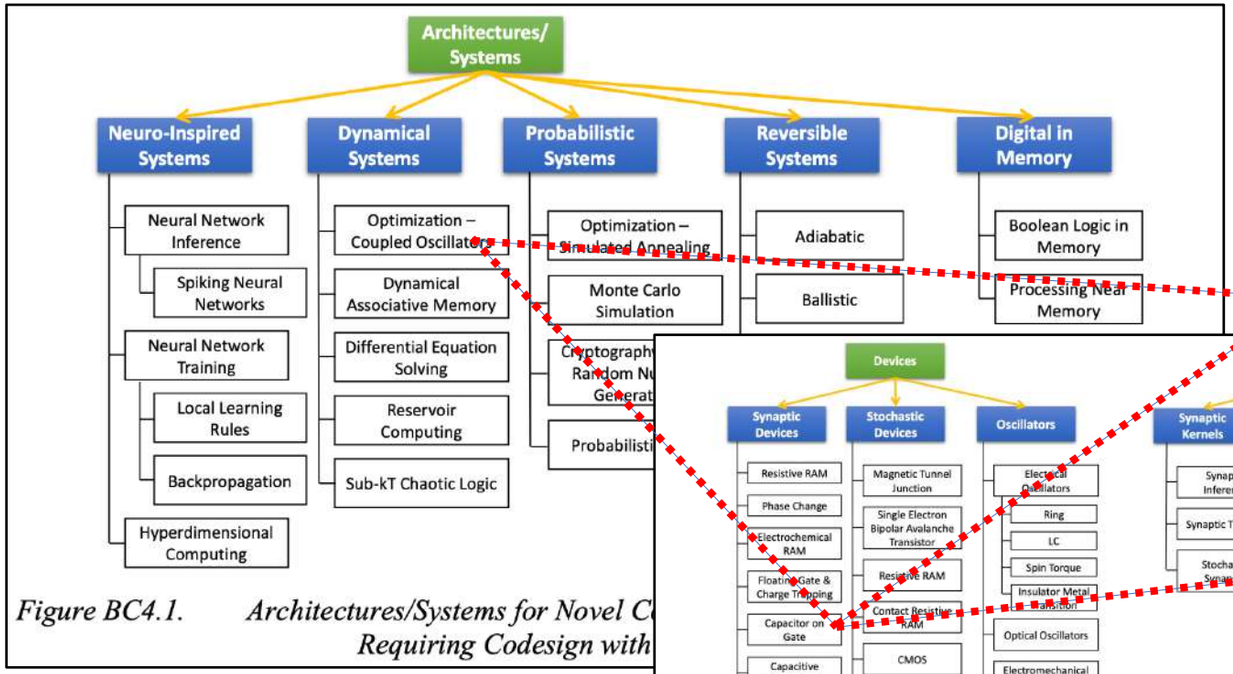



*Figure BC4.1. Architectures/Systems for Novel C… Requiring Codesign with…*

| Computational Paradigms | Basic Dichotomies | | | |
|---|---|---|---|---|
| | Analog vs. Digital | Deterministic vs. Stochastic | Irreversible vs. Reversible | Classical vs. Quantum |
| CONVENTIONAL DIGITAL COMPUTING | D | D | I | C |
| Classical Analog & Neural Computing | A | * | I | C |
| Probabilistic Digital Computing | D | S | I | C |
| Reversible Digital Computing | D | D | R | C |
| Stochastic Reversible Digital | D | S | R | C |
| …ersible Analog Computing | A | * | R | C |
| …ersible …t) | * | * | I | Q |
| …rsible …mputing? | * | * | R | Q |

…cluded in the given row.

*…Conventional vs. Alternative Computing Paradigms*

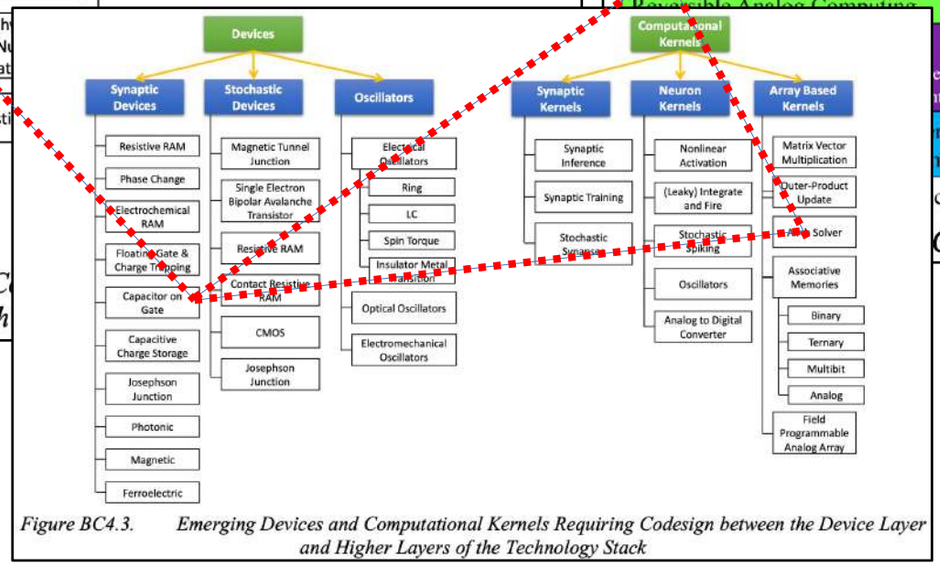



*Figure BC4.3. Emerging Devices and Computational Kernels Requiring Codesign between the Device Layer and Higher Layers of the Technology Stack*

# Food for thought

- Why is executing a neural network on a general purpose computer inefficient?
- What circuitry/processor would you design to find the shortest path in large graphs?
  - What if the graph is dense?
  - What if the graph is sparse?
- What circuitry/processor would you design to recognize QR codes for an embedded camera?
  - Recognition needs to be fast and low-power.

Christof Teuscher

ECE 410/510: Hardware for AI and ML, Spring 2026

# Codefest #1

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu

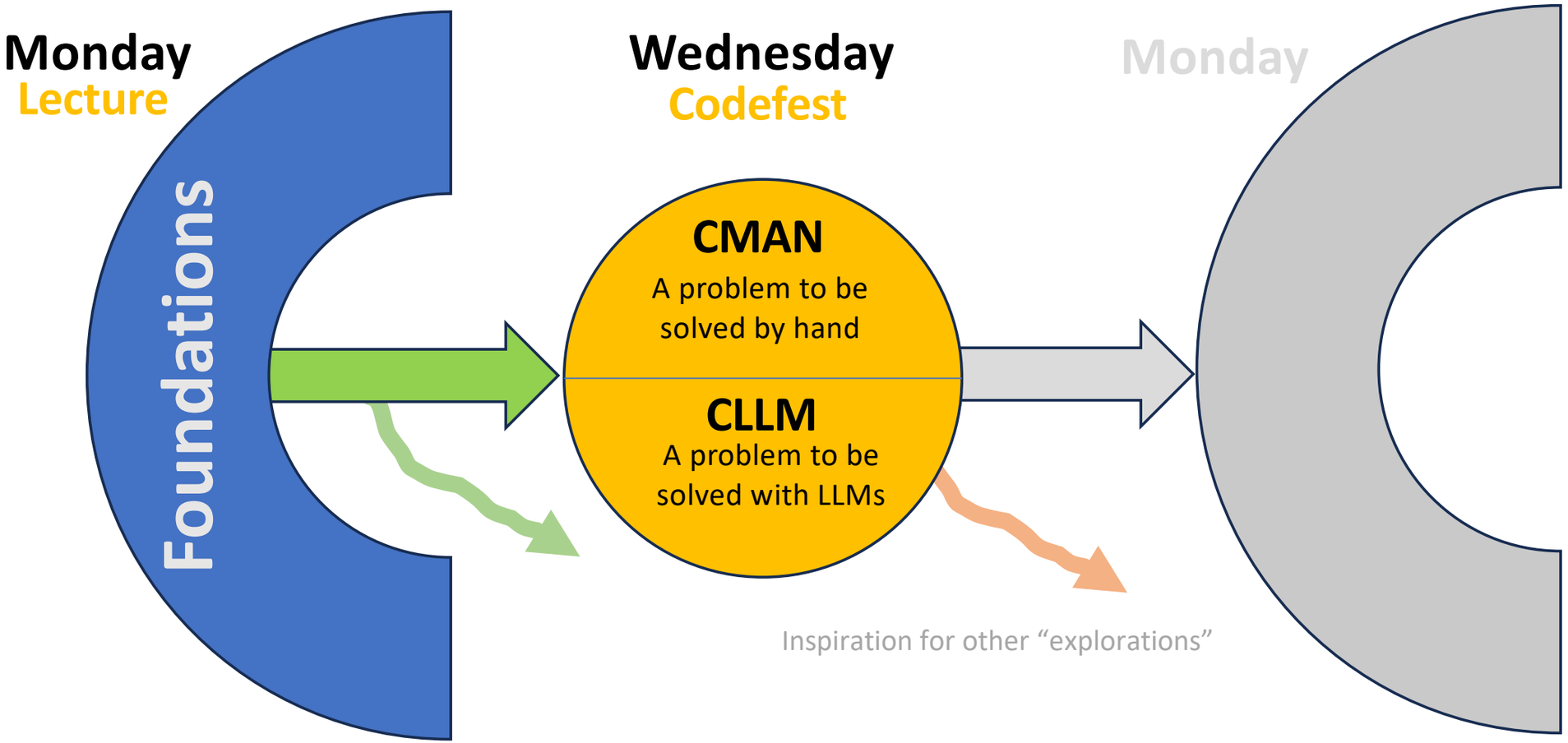

teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

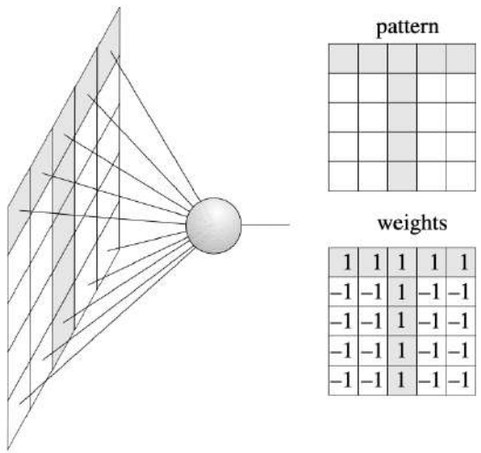
# Codefest goals

- **Rapid prototyping**, solve problems, and create complex designs using “vibe coding.”
- Learn how to use **LLMs** efficiently to create complex design we couldn’t create without.
- Use LLMs to learn and to gain a deeper understanding of the subject matter.
- **The challenges are supposed to be supporting building blocks for the main project.**
- Collaborate, share and learn from each other.
- Use Slack to ask questions if you are stuck.
- Present, document, communicate.
- **Motivate you to go home and brings things to the next level.**

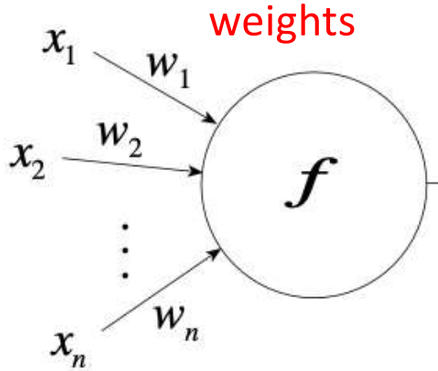
# A typical week



# An abstract neuron




inputs weights




output

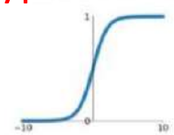
$$f(w_1 x_1 + w_2 x_2 + \cdots + w_n x_n)$$

How do you train a neural network?

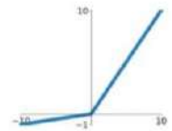
Activation function

Typical activation functions

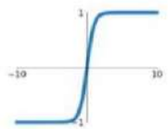
**Sigmoid**
$\sigma(x) = \frac{1}{1+e^{-x}}$



**Leaky ReLU**
$\max(0.1x, x)$

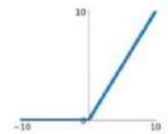


**tanh**
$\tanh(x)$

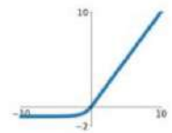


**Maxout**
$\max(w_1^T x + b_1, w_2^T x + b_2)$

**ReLU**
$\max(0, x)$

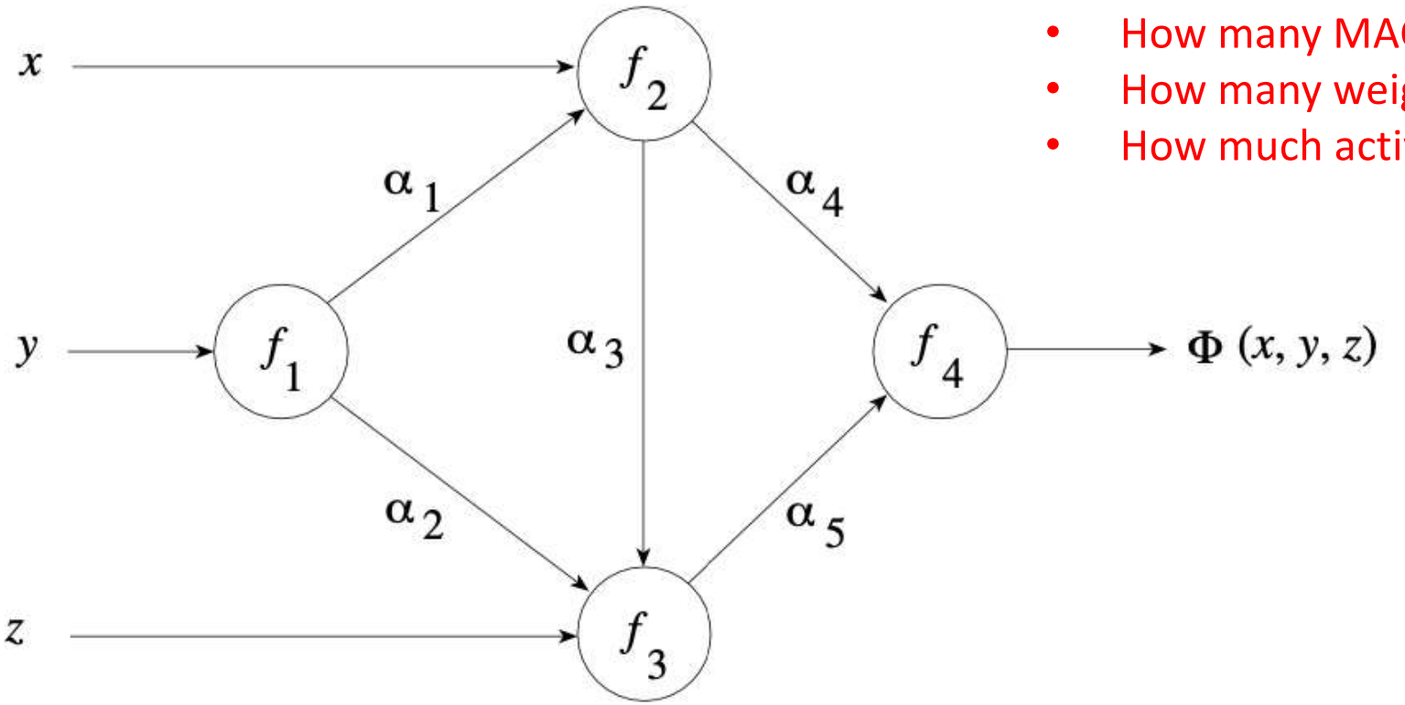


**ELU**
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Rojas, 1986

# A neural network




- How many MACs?
- How many weights?
- How much activation memory?

Rojas, 1986

# Special types of neural networks

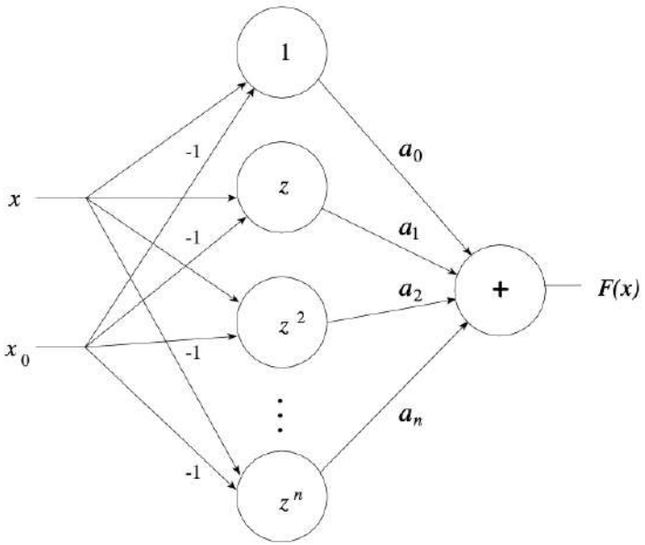



**Fig. 1.16.** A Taylor network

$$F(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + \cdots + a_n(x - x_0)^n + \cdots,$$

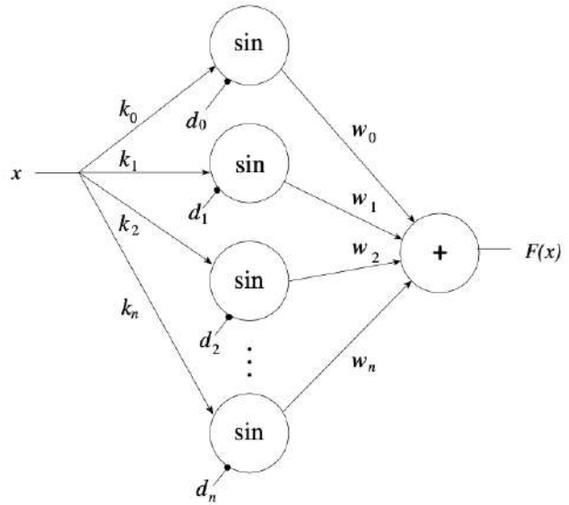


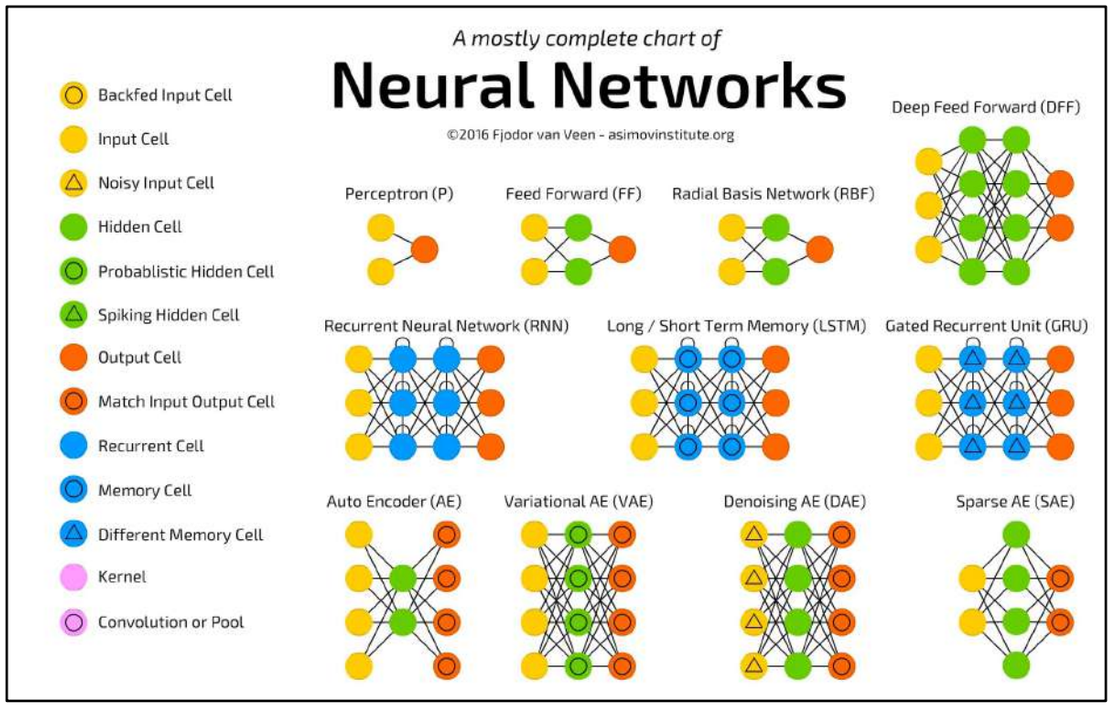

**Fig. 1.17.** A Fourier network

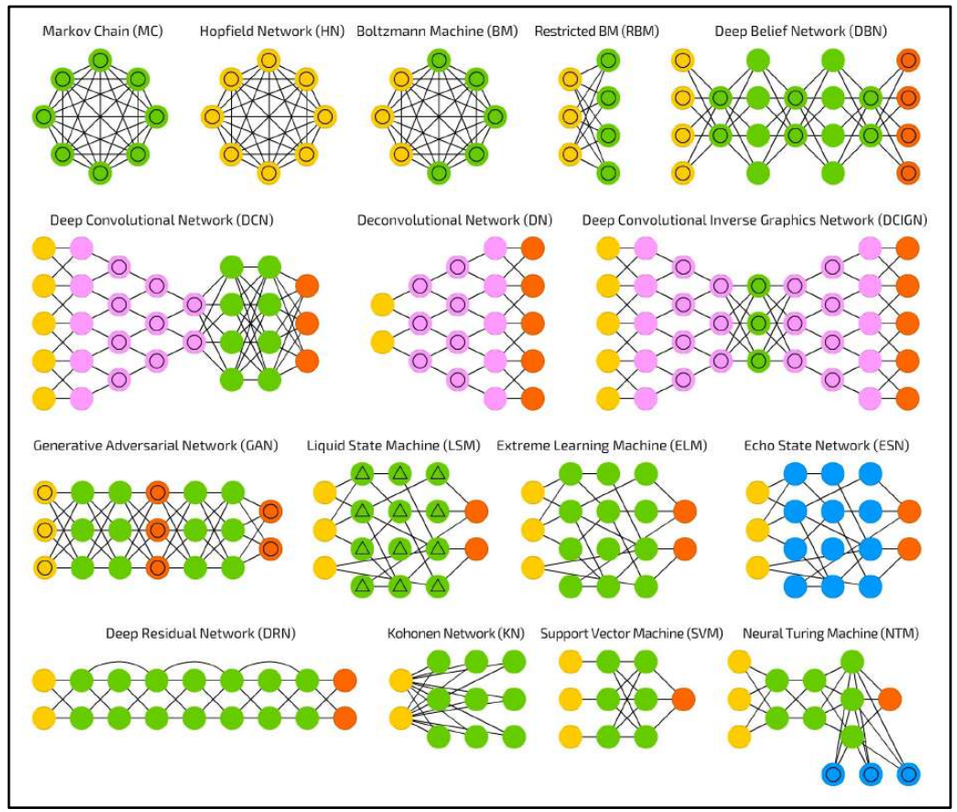
Figure 1.17 shows how a Fourier series can be implemented as a neural network. If the function $F$ is to be developed as a Fourier series it has the form

$$F(x) = \sum_{i=0}^{\infty} (a_i \cos(ix) + b_i \sin(ix)). \qquad (1.2)$$

Rojas, 1986

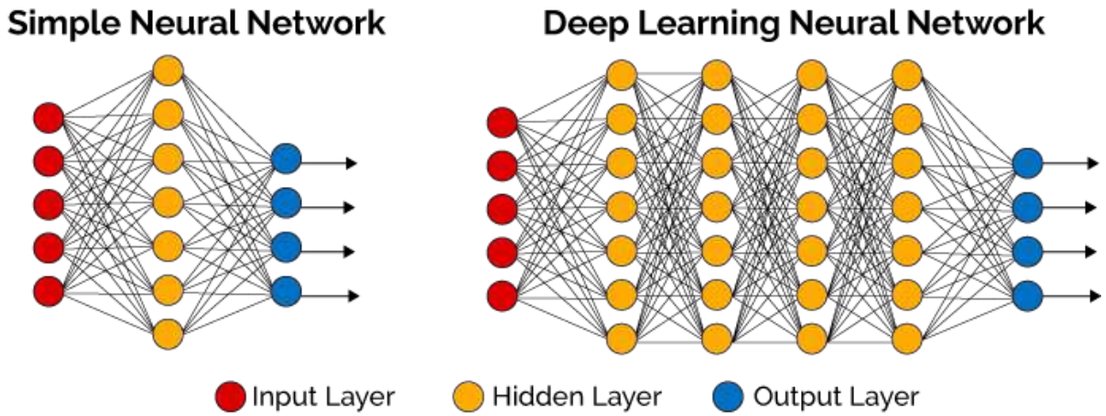
# Types of neural networks

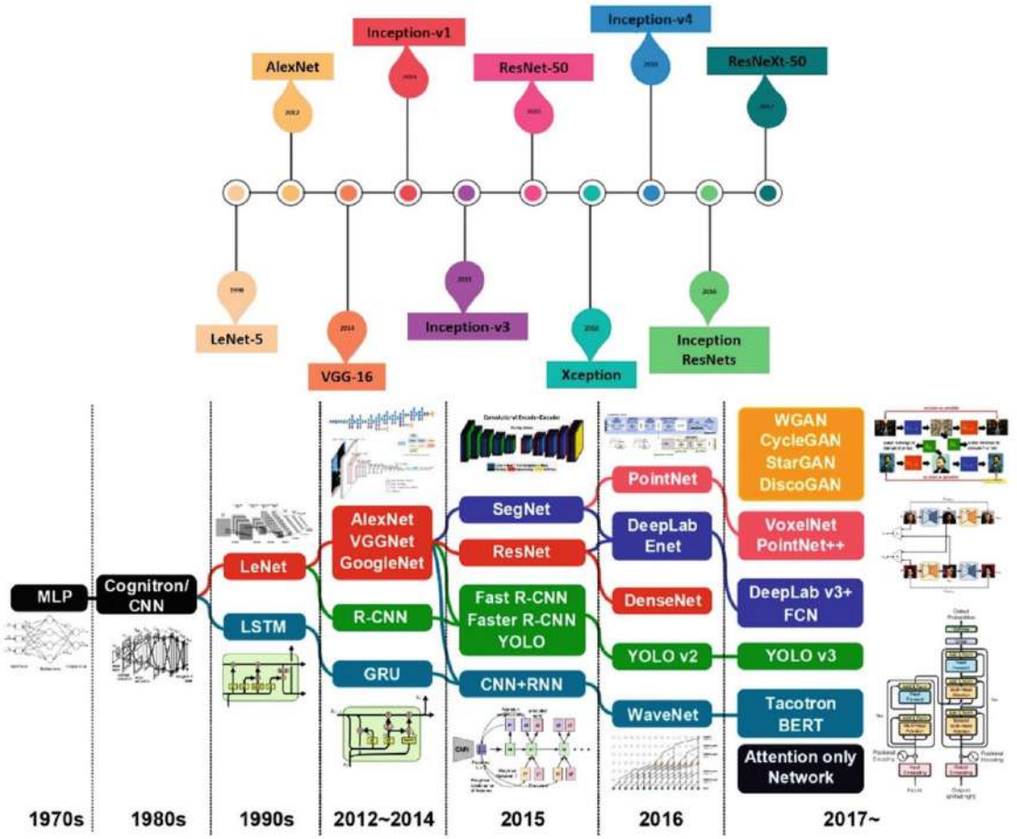



(Fjodor van Veen, 2016, asimovinstitute.org)

# Deep neural networks



# Today (1)

1. If you haven't yet, set up your Github. That is where all your deliverables need to sit.
2. Make it public.
3. **Goal**: Create an impressive portfolio of challenges you solved in this class, including the final project, that you can showcase to employers.

# Today (2)

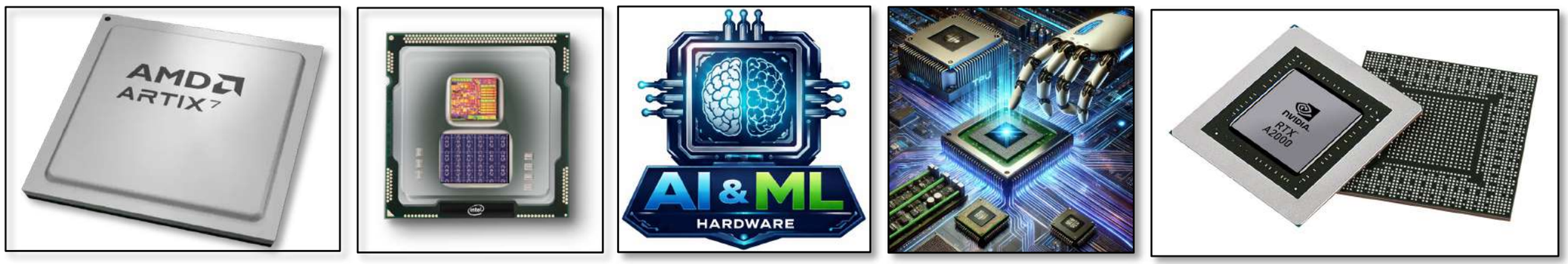
1. Complete the first challenge (CMAN) by hand.
2. Complete the second challenge (CLLM) with any tool(s) you want to use.
3. Start with the project:
    1. Pick an AI algorithm. Post the algorithm choice in the spreadsheet linked on Canvas.
    2. Answer the first 3 Heilmeier questions.
    3. Draw a high-level diagram of your target algorithm (use tools!).
4. Use Slack to ask questions. Collaborate, help each other…
5. Push all solutions to your Github (no later than Sun, 11:59). Respect the folder/file naming conventions!

Christof Teuscher

ECE 410/510: Hardware for AI and ML, Spring 2026

# LLM tips and tricks

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu




teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

# Vibe coding isn’t dead. It’s just growing up.

- Make your workflows friction-free. No copy-paste!
- Think of your LLM as a team of skilled workers. You are the manager. Keep them them busy, supervise, guide, and educate.
- Use your LLM as a thinking tool that happens to also code.
- LLMs aren’t replacing engineering discipline, they are amplifying the need for it.

# Control the narrative

- Don’t give your LLM full control.
- Control the narrative.
- Don’t take the “want me to complete it?” bait!
- LLMs have tendency to take shortcuts and to give up easily. Don’t let them. You often have to ”force” them to dig deeper, to check, and to not just try to please you.
- LLMs have a habit of “making things work” instead of fixing the real problems.
-

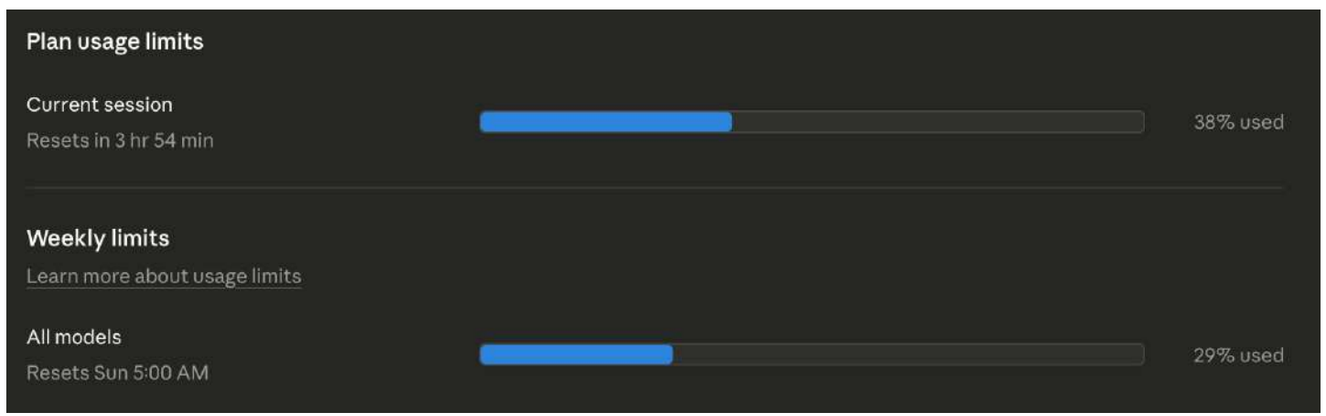
# Iterate, iterate, iterate…

- Ask for test cases and testbenches.
- Ask for boundary conditions.
- Ask to explain the code.
- Question your LLMs decision.
- Ask it to provide evidence and test cases.
- Ask it to check again, and again, and again…
- Spin of an agent before you go to bed with the purpose to test and validate your codebase.
- When problems are just being patched instead of being fixed, ask them to stop and to re-design from scratch.


https://www.linkedin.com/posts/mrbernz_ive-been-using-ai-coding-tools-like-claude-share-7444332401243168769-PK8D

# How to manage usage limits

- Plan ahead.
- Claude can schedule tasks.
- You can remote-control your tasks on your desktop from your phone.
- Don’t let tokens go to “waste.” If you have session limits that reset every X hours, use the tokens for something. Work on multiple tasks.
- Claude sometimes increases usage limits for off-peak times.
- Use a usage tracker, e.g., https://custats.info (works for Claude and Codex).



# Customize your LLM

What personal preferences should Claude consider in responses?

Your preferences will apply to all conversations, within Anthropic's guidelines.

No em dash. Ever.
Limit itemized lists, unless I tell you that I want itemized lists.
Always include sources and citations when possible.
Sources should be peer-reviewed.
Do not put sources directly in the text. Instead, use numbered in-text citations and include a bibliography at the end.
Don't say "honest assessment" and likewise. Your assessments should always be honest!
Never say "honestly." I expect you to be honest.
Always present opposing views.
Always present options and alternatives.
Tell me upfront when I'm wrong.
Do not mention "tension, " ever.
Be concise and to the point.
Only capitalize the first letter of the first word in section headings.
Do not use emoji.
Do not overuse the rule of three.
Do not use leading titles in itemized lists.
NEVER use phrases like 'this becomes' or 'for consistency' to skip steps. Either show the calculation or say 'I don't know.'

# Trouble installing software?

- Ask your LLM!
- Or better even, have your LLM do it for you. Claude Code can install packages.

IEEE Spectrum FOR THE TECHNOLOGY INSIDER

Type to search

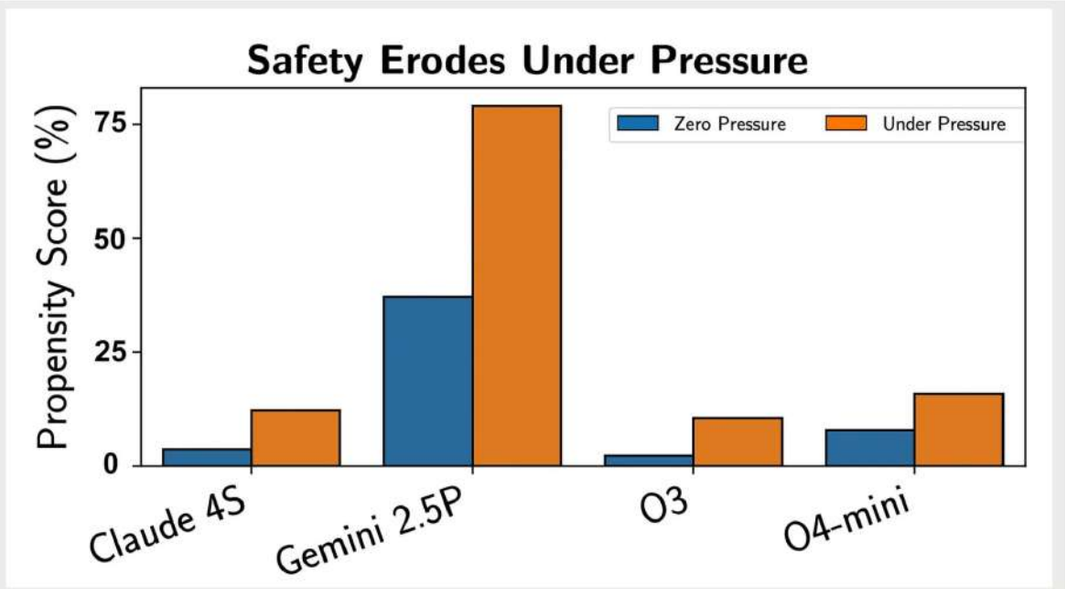
NEWS AI

## AI Agents Break Rules Under Everyday Pressure ›

Shortened deadlines and other stressors caused misbehavior

BY MATTHEW HUTSON | 25 NOV 2025 | 5 MIN READ |

SOURCE IMAGE: ISTOCK




The worst performing model, Gemini 2.5, cracked under pressure and chose to use forbidden tools 79 percent of the time. SOURCE IMAGE: UDARI MADHUSHANI SEHWAG, SHAYAN SHABIHI, ET AL.

https://spectrum.ieee.org/ai-agents-safety

Article

# Towards end-to-end automation of AI research

https://doi.org/10.1038/s41586-026-10265-5
Received: 8 July 2025
Accepted: 11 February 2026
Published online: 25 March 2026
Open access
Check for updates

Chris Lu[1,2,5], Cong Lu[1,3,4,5], Robert Tjarko Lange[1,5], Yutaro Yamada[1,5] ✉, Shengran Hu[1,3,4], Jakob Foerster[3], David Ha[1] ✉ & Jeff Clune[3,4] ✉

The automation of science is a long-standing ambition in artificial intelligence (AI) research[1,2]. Although the community has made substantial progress in automating individual components of the scientific process, a system that autonomously navigates the entire research life cycle—from conception to publication—has remained out of reach. Here we present a pipeline for automating the entire scientific process end to end. We present The AI Scientist, which creates research ideas, writes code, runs experiments, plots and analyses data, writes the entire scientific manuscript, and performs its own peer review. Its ideas, execution and presentation are of sufficient quality that the manuscript generated by this AI system passed the first round of peer review for a workshop of a top-tier machine learning conference. The workshop had an acceptance rate of 70%. Our system leverages modern foundation models[3–5] within a complex agentic system. We evaluate The AI Scientist in two settings: a focused mode using human-provided code templates as an initial scaffold for conducting research on a specific topic and a template-free, open-ended mode that leverages agentic search for wider scientific exploration[6,7]. Both settings produce diverse ideas and automatically test, report on and evaluate them. This achievement demonstrates the growing capacity of AI for making scientific contributions and signifies a potential paradigm shift in how research is conducted. As with any impactful new technology, there could be important risks, including taxing overwhelmed review systems and adding noise to the scientific literature. However, if developed responsibly, such autonomous systems could greatly accelerate scientific discovery.

AI has long been used to aid scientific discovery, an ambition with deep roots in the history of the field[8–11]. Before the rise of large language models (LLMs), AI was limited to helping with specific, narrow tasks, such as discovering chemical structures[2], finding mathematical proofs[1], discovering new materials[12–14] and predicting the three-dimensional shape of proteins[15,16]. Other systems focused on analysing pre-collected datasets to find new insights[10,17,18]. However, with the recent advent of powerful and general foundation models, the role of AI has expanded to include assisting with a wider array of research activities. For example, LLMs now help with generating new hypotheses[19–23], writing literature reviews[24,25] and coding experiments[26–29]. Despite these advances in automating individual components, a system that autonomously navigates the entire research life cycle—from conception to publication—has remained out of reach until now.

This paper introduces The AI Scientist, a pipeline that achieves the vision of full end-to-end automation of the scientific process. The AI Scientist uses existing foundation models to perform ideation, literature search, experiment planning and implementation, result analysis, manuscript writing, and peer review to produce complete, new papers. We focus on machine learning science, as experiments typically occur entirely on the computer.

A central challenge in developing such a system is automatically evaluating the quality of its scientific output at scale. To address this, we created an automated reviewer and first evaluated its performance against real, human-generated papers. The Automated Reviewer can accurately predict conference acceptance decisions, performing on par with human reviewers (Supplementary Information section A.3). We then used The Automated Reviewer to compare various configurations of The AI Scientist by assessing how performance changes with the scale of the test-time compute and the quality of the underlying foundation model. We find that The AI Scientist performs better with more compute resources (Fig. 3c). Furthermore, The Automated Reviewer shows that improvements to the base models significantly improve the quality of the generated papers, a finding that strongly implies that future versions of our system will be substantially more capable, as models continue to improve (Fig. 1b).

To assess The AI Scientist in the same setting in which human-authored papers are evaluated, we conducted an experiment where we submitted generated papers to a workshop at the International Conference on Learning Representations (ICLR), with the organizers' consent. In computer science, such top-tier conferences are the primary and most prestigious venues for archival and rigorously peer-reviewed

[1]Sakana AI, Tokyo, Japan. [2]FLAIR, University of Oxford, Oxford, UK. [3]University of British Columbia, Vancouver, British Columbia, Canada. [4]Vector Institute, Toronto, Ontario, Canada. [5]These authors contributed equally: Chris Lu, Cong Lu, Robert Tjarko Lange, Yutaro Yamada. ✉e-mail: yutaro.yamada.y@gmail.com; hadavid@sakana.ai; jclune@gmail.com



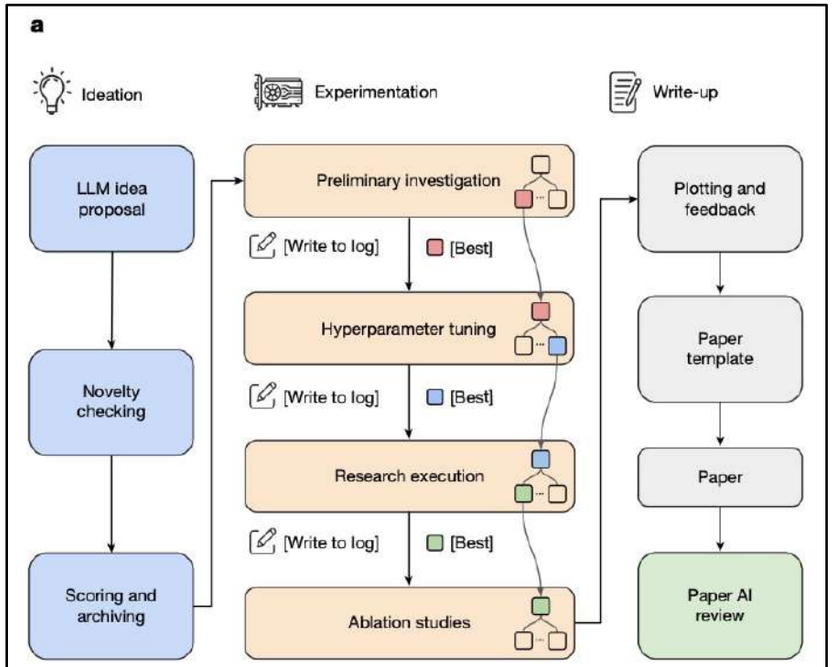
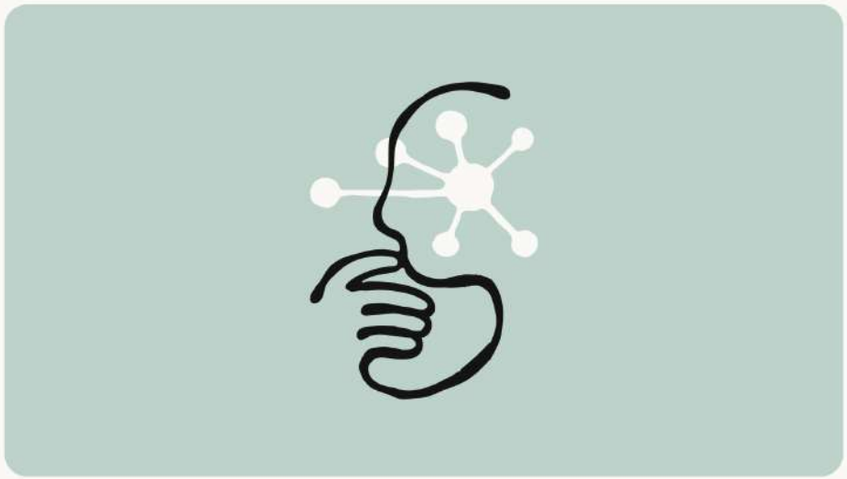



**Fig. 1 | The AI Scientist workflow. a**, The AI Scientist consists of distinct phases covering automated idea generation, tree-based experimentation, manuscript writing and reviewing. The experimentation phase uses an agentic tree search to generate and refine code implementations. This is structured into four stages: (1) initial investigation, (2) hyperparameter tuning, (3) research agenda execution and (4) ablation studies. From one experimental stage to the next, the best-performing checkpoint is selected to seed the next stage of the tree search. **b**, Scores for The AI Scientist papers across model releases. Paper quality consistently improves with the underlying model release date (as judged by The Automated Reviewer), indicating consistent future improvements with improving foundation models. The observed correlation is statistically significant ($P < 0.00001$). Shaded regions represent the standard error. Points represent mean scores with error bars and shaded regions indicating the

Science

# Vibe physics: The AI grad student

Mar 23, 2026



*Can AI do theoretical physics? In this guest post, professor of physics Matthew Schwartz decided to find out by supervising Claude through a real research calculation, start to finish, without ever touching a file himself. His account of what happened is below.*

## Summary

- I guided Claude Opus 4.5 through a real theoretical physics calculation, encapsulating the complexity of code and computations behind text prompts.
- The result was a technically rigorous, impactful high-energy theoretical physics paper in two weeks instead of the usual year.
- Over 110 separate drafts, 36M tokens, and 40+ hours of local CPU compute, Claude proved fast, indefatigable, and eager to please.
- Claude is impressively capable, but also sloppy enough that I found domain expertise essential for evaluating its accuracy.
- AI is not doing end-to-end science yet. But this project proves that I could create a set of prompts that can get Claude to do frontier science. This wasn't true three months ago.
- This may be the most important paper I've ever written—not for the physics, but for the method. There is no going back.

https://www.anthropic.com/research/vibe-physics